

گزارش کار تمرین سری سوم درس یادگیری ژرف دکتر تشنه‌لب

کسری عطار کاشانی

KASRA ATTAR KASHANI

بخش اول :

جواب سوال اول : (1)

در شبکه‌های پرسپترون چندلایه (MLP) برای دسته‌بندی تصویر چالش اصلی این است که MLP خودش هیچ مکانیزمی برای استخراج ویژگی ندارد و فقط با ورودی‌های برداری (برداری شده پیکسل‌ها) کار می‌کند. به همین دلیل:

1 - ساختار مکانی (همسایگی پیکسل‌ها) از بین می‌رود.

۲- طراح مجبور است به صورت دستی ویژگی‌های HOG، SIFT، گرادینان لبه و ... را استخراج کند و فقط بخش طبقه‌بند را به MLP بسپارد.

۳- تعداد پارامترها انفجاری می‌شود (مثلاً برای تصویر $224 \times 224 \times 3$ نخستین لایه FC حدود ۱۵۰ هزار وزن به‌ازای هر نورون می‌خواهد).

۴- شبکه نسبت به جابه‌جایی، تغییر مقیاس و چرخش تغییر میکند.

در CNN هر دو بخش استخراج ویژگی و دسته‌بندی در یک گراف end-to-end به‌طور همزمان با کاهش گرادیانی آموزش می‌بینند. لایه‌های کانولوشنی با اشتراک وزن (weight sharing) و پنجره محلی ویژگی‌های سطح پایین (لبه، گوشه)، سپس میانی (بافت، شکل ساده) و در انتها سطح بالا (شی) را به صورت سلسله‌مراتبی یاد می‌گیرند و بخش انتهایی با همین ویژگی‌ها طبقه‌بندی را انجام می‌دهد.

فرایند مناسب طراحی CNN :

- **شروع از معماری کوچک و عمق بخشی تدریجی:** ابتدا یک شبکه کم عمق (مثلاً ۲-۳ لایه کانولوشن) ساخته می‌شود تا مرز تطبیق با داده بررسی شود. بعد عمق و عرض (تعداد فیلتر) افزایش می‌یابد تا زمانی که مدل بتواند روی آموزش به دقت بالا برسد (نشانه ظرفیت کافی).
- **منظم سازی پس از کنترل تطبیق:** زمانی که فاصله Train Acc و Val Acc زیاد شد، Dropout، BatchNorm، Weight Decay و Data Augmentation شکاف کاهش داده می‌شود.
- **انتخاب اندازه کرنل، گام (stride)، padding و pooling** بر اساس ورودی و اندازه ناحیه پذیرنده مطلوب.
- **تنظیم هایپر پارامترهای آموزش:** نرخ یادگیری، batch size، بهینه‌ساز پس از تثبیت ساختار شبکه.

- پایش مداوم منحنی‌های Loss/Accuracy آموزش و اعتبارسنجی و تحلیل ماتریس درهم‌ریختگی برای کشف کلاس‌های ضعیف.

جواب سوال دوم: (2)

الف) تعریف GAP و تبدیل تنسور $8 \times 8 \times 256$ به بردار ویژگی

Global Average Pooling یعنی میانگین‌گیری مکانی روی هر کانال به صورت کامل. اگر تنسور ورودی به صورت زیر باشد:

$$X \in \mathbb{R}^{C \times H \times W}$$

که در اینجا ابعاد برابرند با:

$$C = 256, \quad H = W = 8$$

خروجی GAP یک بردار ویژگی به شکل زیر خواهد بود:

$$z \in \mathbb{R}^C$$

که عضو مربوط به کانال c در آن از رابطه زیر به دست می‌آید:

$$z_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W X_{c,i,j} = \frac{1}{8 \cdot 8} \sum_{i=1}^8 \sum_{j=1}^8 X_{c,i,j}$$

به این ترتیب هر نقشهٔ ویژگی (feature map) ابعاد 8×8 به یک عدد اسکالر فشرده می‌شود و در نتیجه 256 نقشه به یک بردار 256 بعدی تبدیل می‌گردد.

ب) ابعاد خروجی GAP

ابعاد خروجی پس از لایه GAP برابر است با:

$$\mathbb{R}^{256}$$

یا به طور دقیق‌تر ابعاد به صورت زیر است که پس از اعمال دستور squeeze به یک بردار تبدیل می‌شود:

$$256 \times 1 \times 1$$

یعنی ابعاد فضایی و مکانی کاملاً حذف می‌شوند و تنها بعد کانال باقی می‌ماند.

پ) مقایسه تعداد پارامترهای لایه FC در دوروش (بدون بایاس)

تعداد پارامتر FC	بعد خروجی	بعد ورودی FC	طراح
$16384 \times 10 = 163,840$	10	$256 \times 8 \times 8 = 16384$	طراح اول (Flatten → FC)
$256 \times 10 = 2,560$	10	256	طراح دوم (GAP → FC)

نسبت ساده سازی پارامترها به صورت زیر است:

$$\frac{163840}{2560} = 64$$

یعنی طراح دوم با 64 برابر پارامتر کمتر همان طبقه بندی 10 کلاسه را انجام می دهد.

ت) چرا GAP احتمال بیش برزش (overfitting) را کم می کند؟

بیش برزش عمدتاً وقتی رخ می دهد که شبکه ظرفیت بسیار بیشتری از حجم داده دارد. در روش Flatten + FC، لایه FC با 163,840 وزن می تواند الگوی دقیق هر تصویر آموزش (حتی نویز پس زمینه) را حفظ کند. GAP. حذف 64 برابر پارامتر، اولاً ظرفیت اضافی را برمی چیند و ثانیاً یک پیش فرض ساختاری مفید (structural prior) به مدل تحمیل می کند: «هر کانال نماینده یک مفهوم/کلاس است و فعالیت متوسط آن کانال بر کل تصویر، شواهد آن مفهوم است». این منظم سازی ساختاری باعث می شود مدل به جای حفظ کردن مختصات دقیق ویژگی ها، یاد بگیرد چه کانالی برای چه کلاسی فعال شود.

ث) مقاوم‌تر بودن GAP در برابر جابه‌جایی شی

در روش Flatten + FC هر وزن FC به یک مختصات مکانی مشخص از یک کانال خاص گره خورده است. اگر در تصویر آزمون شی چند پیکسل جابه‌جا شود فعال شدن همان کانال به مختصات دیگری منتقل می‌شود و وزنی که در فاز آموزش یاد گرفته شده بود روی پیکسلی خاموش قرار می‌گیرد که در نتیجه پاسخ لایه کاملاً تغییر می‌کند.

در لایه GAP چون ابتدا روی همه مختصات یک کانال میانگین گرفته می‌شود، جابه‌جایی شی داخل میدان دید همان کانال هیچ تغییری در خروجی زیر ایجاد نمی‌کند:

$$Z_c$$

این ویژگی یک تغییر ناپذیری تقریبی نسبت به انتقال فراهم می‌کند.

ج) چه زمانی GAP می‌تواند اطلاعات مفید را از بین ببرد؟

GAP هنگامی مضر می‌شود که مکان دقیق ویژگی بخشی از تصمیم باشد، مثلاً:

1. **تشخیص اشیا (object detection) و قطعه‌بندی (segmentation):** مختصات شی برای کشیدن جعبه یا ماسک ضروری است. میانگین مکانی این اطلاعات را پاک می‌کند.
2. **مسائلی که الگوی توزیع مکانی اهمیت دارد:** مثلاً تشخیص اینکه چهره در گوشه تصویر است یا وسط
3. **وقتی اندازه ناحیه مفید نسبت به کل تصویر بسیار کوچک است:** مثلاً یک تومور کوچک در عکس MRI که میانگین‌گیری بر روی پس‌زمینه خاموش، سیگنال ضعیف هدف را رقیق می‌کند.
4. **عدم معناگرایی کانال‌های انتخابی:** وقتی کانال‌های آخر هنوز ویژگی‌های سطح پایین یا میانی هستند و به اندازه کافی معنایی نشده‌اند.

جواب سوال سوم: (3)

مشکلی که برای آن پیشنهاد شد: پدیده degradation (افت دقت با عمیق تر شدن شبکه) و در سطح ریاضیاتی. محو شدن گرادیان (vanishing gradient) در شبکه های بسیار عمیق. در یک شبکه معمولی گرادیان نسبت به ورودی برابر است با حاصل ضرب ژاکوبین های همه لایه ها:

$$\frac{\partial L}{\partial x} = \prod_{l=1}^L J_l$$

اگر نرم هر ژاکوبین کوچک تر از یک باشد، این ضرب به صورت نمایی به صفر می رود. بلوک باقیمانده خروجی را به شکل زیر تعریف می کند:

$$y = F(x) + x$$

در نتیجه در فاز پس رو (Backward) خواهیم داشت:

$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + I$$

وجود ماتریس همانی تضمین می کند گرادیان حتی اگر مشتق تابع اصلی به صفر میل کند حداقل از مسیر میان بر (skip) به طور کامل به لایه های پایین دست برسد. مزایای دیگر: تسهیل یادگیری نگاشت همانی، هموارتر شدن لندسکیپ تابع هزینه، و امکان آموزش شبکه های با عمق بیش از 100 لایه.

طول اتصال باقیمانده کی بلند کی کوتاه؟

1- اتصال های کوتاه (مثلا روی 2 یا 3 لایه مجاور): زمانی بهتر است که شبکه بسیار عمیق است و خطر اصلی محو گرادیان است. اتصال کوتاه تضمین می کند گرادیان هر چند لایه یک تزریق تازه می گیرد و لایه های مجاور را وادار به یادگیری اصلاحات کوچک (residual learning) می کند.

2- اتصال های بلند (مثلا از ابتدای شبکه به انتها، مانند شبکه های U-Net): زمانی مفید است که انتقال جزئیات مکانی سطح پایین (مانند لبه های دقیق در قطعه بندی تصویر) به لایه های بسیار عمیق ضروری باشد یا بخواهیم ترکیب چند مقیاسی ویژگی ها (multi-scale fusion) داشته باشیم.

جواب سوال چهارم: (4)

هدف Max Pooling

1. زیرنمونه برداری مکانی down-sampling : کاهش ابعاد فضایی نقشه های ویژگی مثلا 2×2 با $\text{stride}=2$ ابعاد را نصف می کند.
2. افزایش ناحیه پذیرنده (receptive field) : نورون های بعدی ناحیه بزرگ تری از تصویر اصلی را پوشش می دهند.
3. ایجاد نوردایی محلی به انتقال : جابه جایی جزئی پیکسل ها خروجی را تغییر نمی دهد چون فقط بزرگ ترین مقدار انتخاب می شود.
4. برجسته سازی قوی ترین فعال سازی : بیشینه ها معمولا متناظر با وجود شواهد قطعی یک ویژگی هستند.

چرا Max Pooling از Average Pooling محبوب تر است؟

در بینایی، حضور یا عدم حضور یک ویژگی مهم تر از مقدار میانگین آن است؛ max به خوبی این حضور را آشکار می کند، در حالی که average ممکن است یک فعال سازی قوی را با اطراف خاموش رقیق کند. همچنین max نویز پس زمینه را فیلتر کرده و گرادینان خلوت (sparse gradient) ایجاد می کند که برای یادگیری ویژگی های متمایز مناسب تر است.

تفاوت Pooling و Convolution

ویژگی	Convolution	Pooling
داشتن وزن قابل یادگیری	بله	خیر (تابع ثابت)
ترکیب بین کانال ها	بله (مجموع وزنی کانال ها)	خیر (پردازش مجزای هر کانال)
تغییر تعداد کانال	بله	خیر
هدف	استخراج ویژگی های جدید	کاهش ابعاد و فشرده سازی
غیرخطی بودن	خطی (پیش از فعال ساز)	غیرخطی (ماهیت تابع max)

بخش دوم:

در این تمرین یک معماری یکسان CNN را روی سه دیتاست Fashion-MNIST، CIFAR-10 و Flower17 به دو شیوه مستقل پیاده‌سازی کردیم:

یک بار کاملاً From-Scratch با NumPy/CuPy (پیاده‌سازی دستی تمام لایه‌ها روی GPU) و یک بار با کتابخانه آماده PyTorch. هاپرپارامترها و تقسیم داده‌ها (70% / 10% / 20%) در هر دو نسخه یکسان نگه داشته شده است. در نهایت یک تحلیل حساسیت 8 گانه روی CIFAR-10 انجام می‌شود تا اثر پارامترهای مختلف بر دقت اعتبارسنجی بررسی شود.

پارامترهای شبکه (بین دو حالت کاملاً یکسان هستند)

پارامتر	مقدار
learning_rate	0.001
batch_size	64
num_epochs	10
dropout_rate	0.5
conv1_filters	32
conv2_filters	64
kernel_size	3
fc1_units	256
optimizer	Adam
Train, Val, Test Ratio	0.1/0.1/0.7
random_seed	42

معماری مشترک شبکه CNN

شماره	نوع لایه	ابعاد خروجی (به ازای ورودی $32 \times 32 \times 3$)
۱	Conv2D	$32 \times 32 \times 32$
۲	BatchNorm2D + ReLU	$32 \times 32 \times 32$
۳	MaxPool2D	$32 \times 16 \times 16$
۴	Conv2D	$64 \times 16 \times 16$
۵	BatchNorm2D + ReLU	$64 \times 16 \times 16$
۶	MaxPool2D	$64 \times 8 \times 8$
۷	Flatten	4096
۸	Dropout + FC1	256
۹	BatchNorm1D + ReLU + Dropout	256
۱۰	FC2	10 یا 17
۱۱	Softmax + Cross-Entropy	تابع هزینه (Loss)

روابط ریاضیاتی پیاده سازی شده در بخش پایه (From-Scratch) :

۱) کانولوشن (Convolution 2D) پیشرو:

$$Y[n, f, i, j] = b[f] + \sum_{c=1}^{C_{in}} \sum_{u=1}^K \sum_{v=1}^K W[f, c, u, v] \cdot X_{pad}[n, c, i \cdot s + u, j \cdot s + v]$$

۲) کانولوشن پسرو (محاسبه گرادیان‌ها):

$$\frac{\partial L}{\partial W[f, c, u, v]} = \sum_{n, i, j} \frac{\partial L}{\partial Y[n, f, i, j]} \cdot X_{pad}[n, c, i \cdot s + u, j \cdot s + v]$$

$$\frac{\partial L}{\partial b[f]} = \sum_{n, i, j} \frac{\partial L}{\partial Y[n, f, i, j]}$$

$$\frac{\partial L}{\partial X_{pad}[n, c, p, q]} = \sum_{f, u, v} \frac{\partial L}{\partial Y[n, f, \frac{p-u}{s}, \frac{q-v}{s}]} \cdot W[f, c, u, v]$$

۳) نرمال سازی دسته ای (Batch Normalization 2D) پیشرو:

$$\mu_c = \frac{1}{m} \sum_{n, i, j} X[n, c, i, j]$$

$$\sigma_c^2 = \frac{1}{m} \sum_{n, i, j} (X[n, c, i, j] - \mu_c)^2$$

$$\hat{X}_c = \frac{X - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}$$

$$Y = \gamma_c \hat{X}_c + \beta_c$$

۴) نرمال سازی دسته ای پسرو:

$$\frac{\partial L}{\partial \gamma_c} = \sum_{n, i, j} \frac{\partial L}{\partial Y} \cdot \hat{X}_c$$

$$\frac{\partial L}{\partial \beta_c} = \sum_{n, i, j} \frac{\partial L}{\partial Y}$$

$$\frac{\partial L}{\partial X} = \frac{1}{m\sqrt{\sigma_c^2 + \epsilon}} \left[m \cdot \frac{\partial L}{\partial \hat{X}_c} - \sum \frac{\partial L}{\partial \hat{X}_c} - \hat{X}_c \cdot \sum \left(\hat{X}_c \cdot \frac{\partial L}{\partial \hat{X}_c} \right) \right]$$

۵) لایه MaxPool2D پیشرو:

$$Y[n, c, i, j] = \max_{0 \leq u, v < P} X[n, c, i \cdot S + u, j \cdot S + v]$$

۶) لایه تمام‌متصل (Fully Connected) پیشرو و پسرو:

$$Y = XW + b$$

$$\frac{\partial L}{\partial W} = X^T \frac{\partial L}{\partial Y}$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^T$$

۷) فعال‌ساز ReLU پیشرو و پسرو:

$$y = \max(0, x)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \mathbf{1}[x > 0]$$

۸) تابع Dropout (نسخه Inverted در آموزش):

$$y = \frac{x \odot m}{1 - p}, \quad m \sim \text{Bernoulli}(1 - p)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \odot \frac{m}{1 - p}$$

۹) لایه Softmax و خطای Cross-Entropy:

$$p_{n,k} = \frac{\exp(z_{n,k} - \max_{k'} z_{n,k'})}{\sum_{k'} \exp(z_{n,k'} - \max_{k''} z_{n,k''})}$$

$$L = -\frac{1}{N} \sum_n \log(p_{n,y_n})$$

$$\frac{\partial L}{\partial z_{n,k}} = \frac{1}{N} (p_{n,k} - \mathbf{1}[k = y_n])$$

۱۰) به‌روزرسانی با بهینه‌ساز Adam:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

۱۱) مقداردهی اولیه وزن‌ها (He Initialization):

$$W \sim \mathcal{N}(0, \sigma^2)$$

$$\text{Conv: } \sigma = \sqrt{\frac{2}{C_{in} \cdot K \cdot K}}, \quad \text{FC: } \sigma = \sqrt{\frac{2}{\text{in_features}}}$$

توضیح Cell ها و خروجی‌ها

Cell 1 : پیاده‌سازی From-Scratch با NumPy + CuPy

در این بخش کدهای نوشته شده تمامی عملیات تانسوری را با هدف افزایش سرعت به ماژول‌های CuPy منتقل می‌کنند. نتایج استخراج شده در ۱۵ خروجی متوالی گواه دقت بالای عملیات دستی است.

- **برای Fashion-MNIST:** دقت روی مجموعه داده تست به 92% رسید که نشان دهنده تطابق عالی بدون علائم بیش‌برازش در نمودار Loss است.
- **برای CIFAR-10:** شبکه به دقت قابل قبول 75% روی داده‌های آزمون رسید. ماتریس درهم‌ریختگی نشان می‌دهد که تمرکز اصلی خطای مدل در تفکیک حیواناتی مانند سگ، گربه و گوزن از یکدیگر است.
- **برای Flower17:** با وجود دقت آموزش بالای 80% دقت تست در حدود 63% باقی ماند که نمودارها به طور واضح شکاف عمیق ناشی از Overfitting را از ایپاک هفتم به بعد نشان می‌دهند.
- **(کد مربوط به این بخش در < Cell 1 > فایل کد TASK2 در فایل ارائه هست)**

Cell 2 : همان معماری با PyTorch + autograd

این سلول با استفاده از ویژگی‌های پیشرفته‌تری نظیر StepLR Scheduler اجرا شده است.

- دقت نهایی CIFAR روی تست 76% بود که در حدود ۰.۶ درصد بهبود نسبت به نسخه-From Scratch را نشان می‌دهد.
- بیش‌برازش در Flower17 اینجا بسیار شدیدتر بود (دقت اعتبارسنجی روی 56% متوقف شد)، چرا که عملیات بهینه‌سازی PyTorch سریع‌تر همگرا شده و ظرفیت اضافی مدل روی داده‌های اندک سریع منجر به حفظ کردن نویزها می‌شود.

• (کد مربوط به این بخش در < Cell 2 > فایل کد TASK2 در فایل ارائه هست)

Cell 3: تحلیل حساسیت هایپرپارامترها روی CIFAR-10

در این ۸ آزمایش ساختارمند، اثر متغیرهای مختلف در ایپاک‌های اولیه بررسی شد:

- **نرخ یادگیری: (LR)** بهترین بازده روی 0.01 ثبت شد.
- **اندازه دسته: (Batch Size)** عدد 16 بیشترین کارایی را داشت زیرا نویز موجود در تخمین گرادیان، به مدل کمک می‌کند تا از مینی‌م‌های محلی خارج شود.
- **توابع فعال‌ساز:** برخلاف Sigmoid که در لایه‌های عمیق سریعاً دچار اشباع می‌شود، تابع ReLU بهترین همگرایی را از خود نشان داد.
- **سایر تنظیمات بهینه:** بهینه‌ساز RMSprop در دوره‌های کوتاه موفق‌تر بود، ضریب $L2 = 1e - 4$ برای منظم‌سازی کافی ارزیابی شد، و بهترین مقدار دراپ‌اوت 0.4 به دست آمد.

• (کد مربوط به این بخش در < Cell 3 > فایل کد TASK2 در فایل ارائه هست)

خروجی‌ها:

بخش SCRATCH:

بازگزاری FashionMNIST و train شبکه

Fashion-MNIST... بازگزاری

شکل داده: (1, 70000, 28, 28)

آموزش: 49000 | اعتبارسنجی: 7000 | آزمون: 14000

--- از صفر Fashion-MNIST آموزش ---

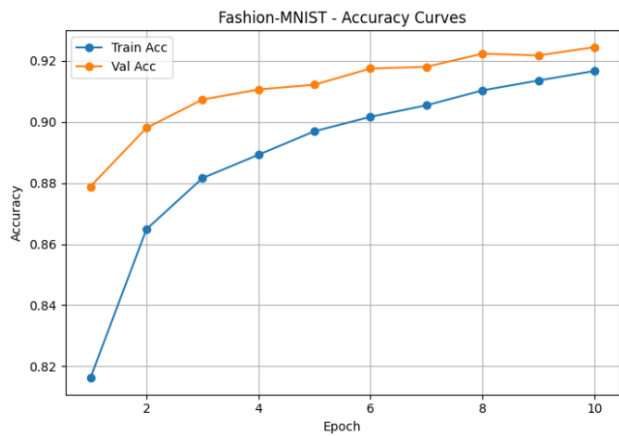
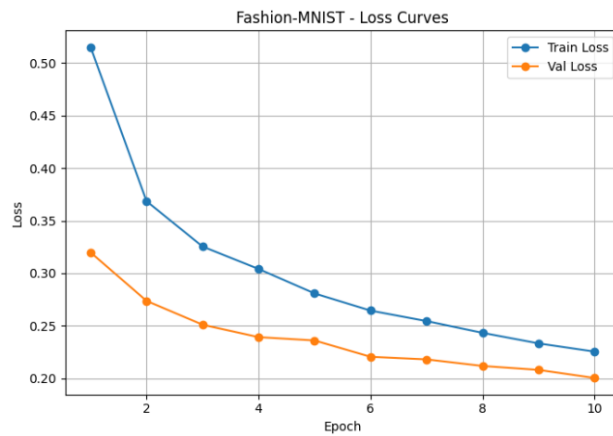
Epoch 01/10 (95.6s)	Train Loss: 0.5150	Train Acc: 81.62%	Val Loss: 0.3198	Val Acc: 87.89%
Epoch 02/10 (92.9s)	Train Loss: 0.3685	Train Acc: 86.48%	Val Loss: 0.2737	Val Acc: 89.80%
Epoch 03/10 (89.6s)	Train Loss: 0.3254	Train Acc: 88.16%	Val Loss: 0.2511	Val Acc: 90.73%
Epoch 04/10 (92.7s)	Train Loss: 0.3041	Train Acc: 88.92%	Val Loss: 0.2393	Val Acc: 91.06%
Epoch 05/10 (90.1s)	Train Loss: 0.2808	Train Acc: 89.69%	Val Loss: 0.2361	Val Acc: 91.21%
Epoch 06/10 (90.9s)	Train Loss: 0.2645	Train Acc: 90.16%	Val Loss: 0.2206	Val Acc: 91.74%
Epoch 07/10 (94.1s)	Train Loss: 0.2545	Train Acc: 90.54%	Val Loss: 0.2181	Val Acc: 91.80%
Epoch 08/10 (91.4s)	Train Loss: 0.2433	Train Acc: 91.03%	Val Loss: 0.2118	Val Acc: 92.23%
Epoch 09/10 (92.1s)	Train Loss: 0.2333	Train Acc: 91.35%	Val Loss: 0.2081	Val Acc: 92.17%
Epoch 10/10 (90.1s)	Train Loss: 0.2255	Train Acc: 91.66%	Val Loss: 0.2004	Val Acc: 92.44%

نتایج دقت در هر کلاس FashionMNIST

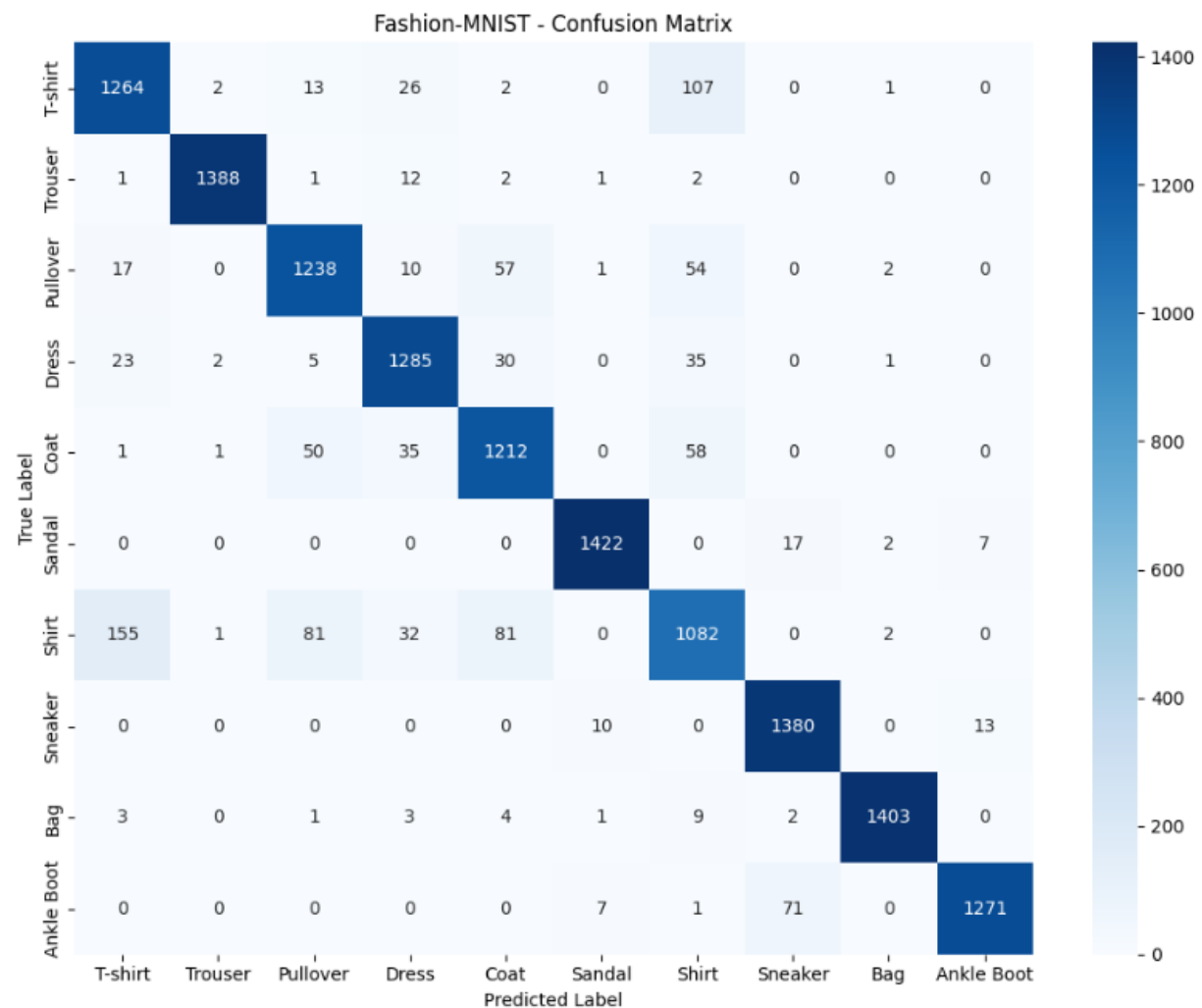
[Fashion-MNIST] Test Performance Evaluation:

	precision	recall	f1-score	support
T-shirt	0.86	0.89	0.88	1415
Trouser	1.00	0.99	0.99	1407
Pullover	0.89	0.90	0.89	1379
Dress	0.92	0.93	0.92	1381
Coat	0.87	0.89	0.88	1357
Sandal	0.99	0.98	0.98	1448
Shirt	0.80	0.75	0.78	1434
Sneaker	0.94	0.98	0.96	1403
Bag	0.99	0.98	0.99	1426
Ankle Boot	0.98	0.94	0.96	1350
accuracy			0.92	14000
macro avg	0.92	0.92	0.92	14000
weighted avg	0.92	0.92	0.92	14000

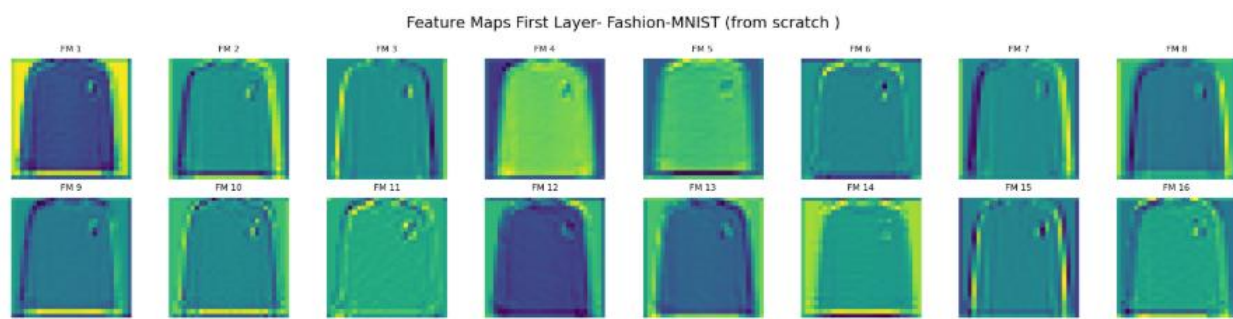
نمودار خطا و دقت FashionMNIST



Confusion Matrix FashionMNIST



First Layer Feature Maps for FashionMNIST



بازگزاری Cifar10 و train شبکه

CIFAR-10... بازگزاری

شکل داده: (3, 32, 32)

آموزش: 42000 | اعتبارسنجی: 6000 | آزمون: 12000

--- از صفر CIFAR-10 آموزش ---

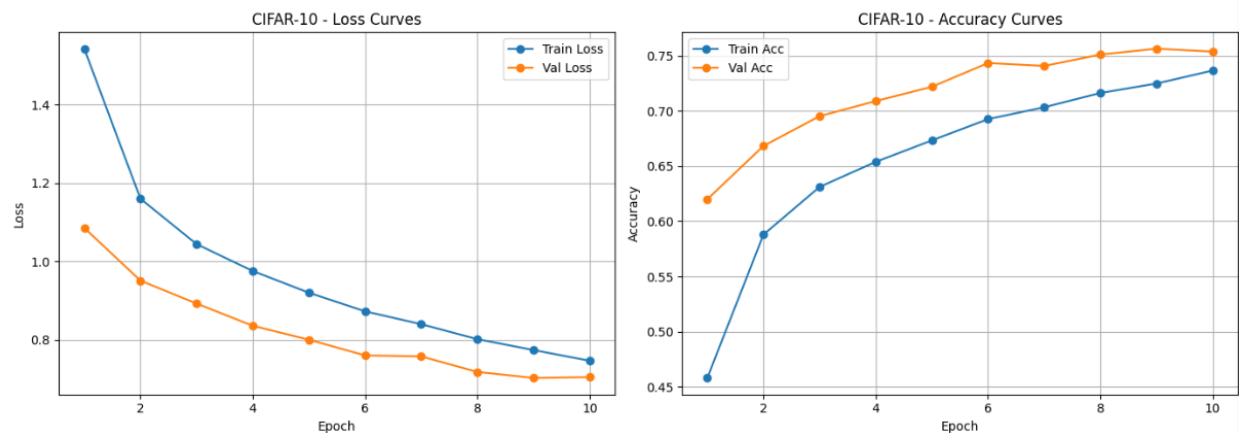
Epoch 01/10 (111.1s)	Train Loss: 1.5425	Train Acc: 45.80%	Val Loss: 1.0854	Val Acc: 62.00%
Epoch 02/10 (113.5s)	Train Loss: 1.1598	Train Acc: 58.80%	Val Loss: 0.9509	Val Acc: 66.83%
Epoch 03/10 (133.4s)	Train Loss: 1.0440	Train Acc: 63.12%	Val Loss: 0.8924	Val Acc: 69.53%
Epoch 04/10 (136.0s)	Train Loss: 0.9754	Train Acc: 65.39%	Val Loss: 0.8358	Val Acc: 70.90%
Epoch 05/10 (133.6s)	Train Loss: 0.9201	Train Acc: 67.34%	Val Loss: 0.8002	Val Acc: 72.20%
Epoch 06/10 (134.6s)	Train Loss: 0.8725	Train Acc: 69.27%	Val Loss: 0.7602	Val Acc: 74.35%
Epoch 07/10 (132.7s)	Train Loss: 0.8398	Train Acc: 70.34%	Val Loss: 0.7576	Val Acc: 74.08%
Epoch 08/10 (136.0s)	Train Loss: 0.8015	Train Acc: 71.63%	Val Loss: 0.7180	Val Acc: 75.12%
Epoch 09/10 (114.2s)	Train Loss: 0.7738	Train Acc: 72.49%	Val Loss: 0.7026	Val Acc: 75.65%
Epoch 10/10 (112.5s)	Train Loss: 0.7465	Train Acc: 73.67%	Val Loss: 0.7047	Val Acc: 75.38%

نتایج دقت در هر کلاس CIFAR10

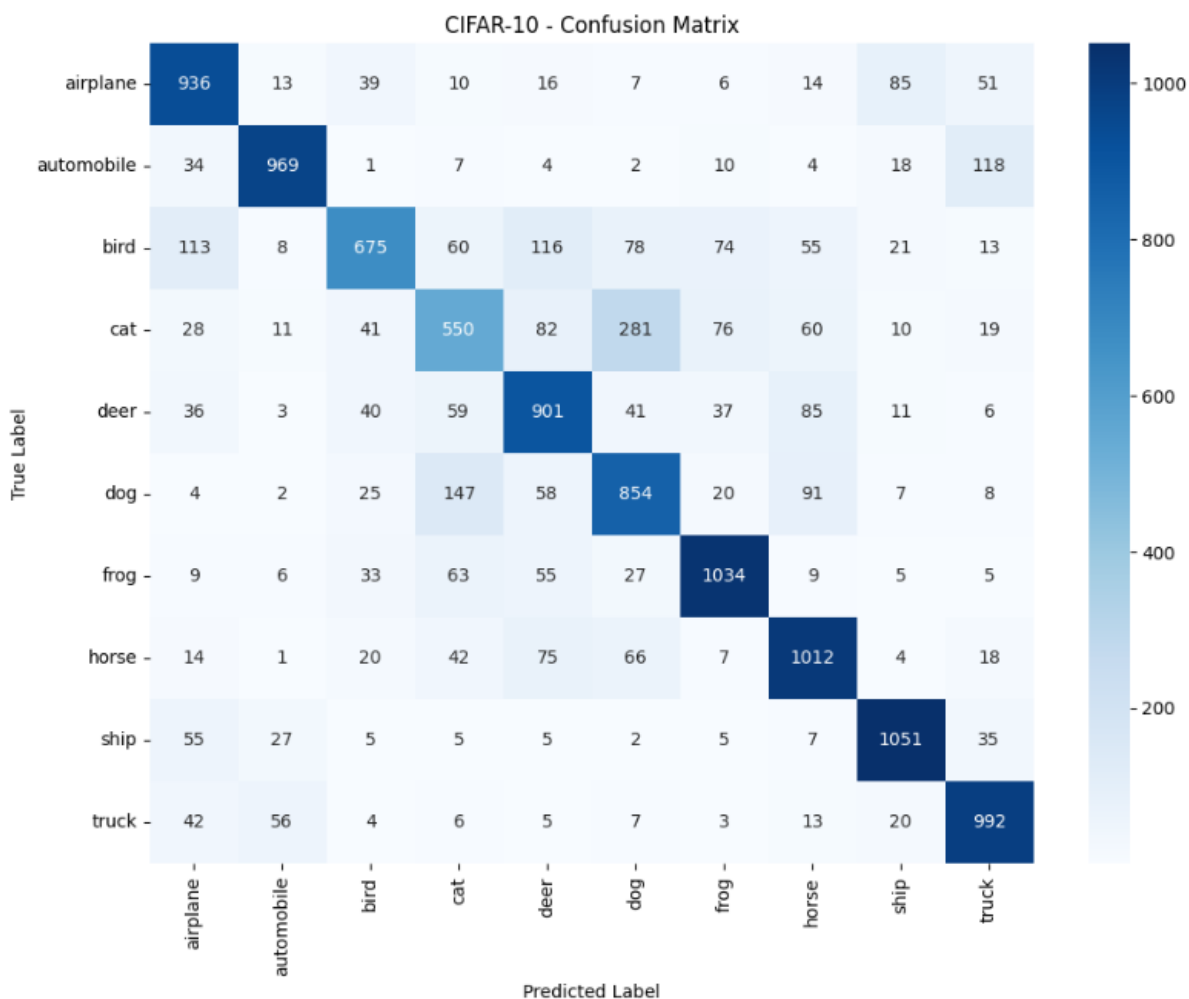
[CIFAR-10] Test Performance Evaluation:

	precision	recall	f1-score	support
airplane	0.74	0.80	0.76	1177
automobile	0.88	0.83	0.86	1167
bird	0.76	0.56	0.64	1213
cat	0.58	0.47	0.52	1158
deer	0.68	0.74	0.71	1219
dog	0.63	0.70	0.66	1216
frog	0.81	0.83	0.82	1246
horse	0.75	0.80	0.78	1259
ship	0.85	0.88	0.87	1197
truck	0.78	0.86	0.82	1148
accuracy			0.75	12000
macro avg	0.75	0.75	0.74	12000
weighted avg	0.75	0.75	0.74	12000

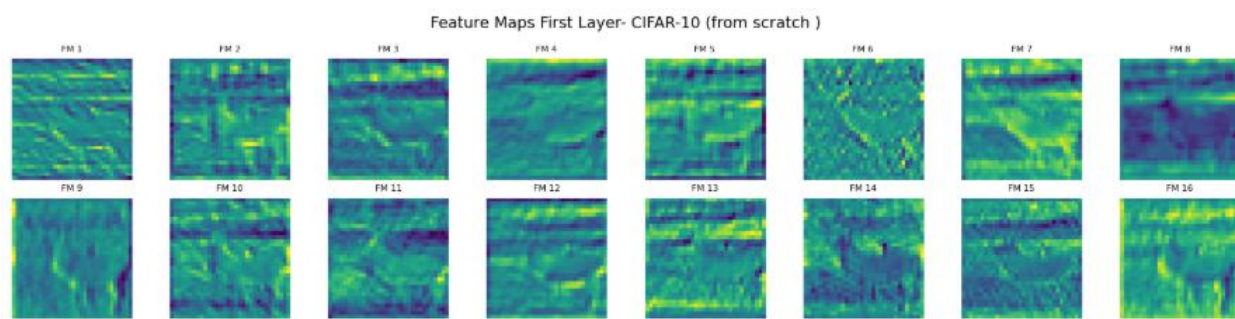
نمودار خطا و دقت CIFAR10



Confusion Matrix CIFAR10



First Layer Feature Maps for CIFAR10



بازگزاری Flower17 و train شبکه

Flower17... بازگزاری

شکل داده: (32, 32, 3, 1360)

آموزش: 951 | اعتبارسنجی: 136 | آزمون: 273

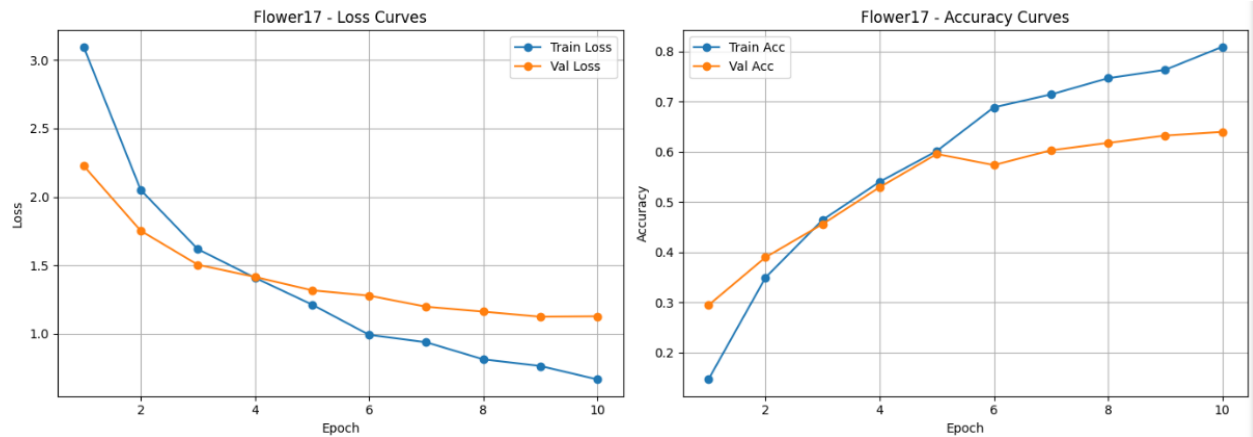
--- از صفر Flower17 آموزش ---

Epoch 01/10 (2.7s)	Train Loss: 3.0955	Train Acc: 14.62%	Val Loss: 2.2269	Val Acc: 29.41%
Epoch 02/10 (2.3s)	Train Loss: 2.0502	Train Acc: 34.93%	Val Loss: 1.7522	Val Acc: 38.97%
Epoch 03/10 (2.7s)	Train Loss: 1.6180	Train Acc: 46.43%	Val Loss: 1.5040	Val Acc: 45.59%
Epoch 04/10 (2.5s)	Train Loss: 1.4093	Train Acc: 54.02%	Val Loss: 1.4153	Val Acc: 52.94%
Epoch 05/10 (2.6s)	Train Loss: 1.2133	Train Acc: 60.16%	Val Loss: 1.3173	Val Acc: 59.56%
Epoch 06/10 (2.5s)	Train Loss: 0.9923	Train Acc: 68.86%	Val Loss: 1.2784	Val Acc: 57.35%
Epoch 07/10 (3.0s)	Train Loss: 0.9377	Train Acc: 71.43%	Val Loss: 1.1973	Val Acc: 60.29%
Epoch 08/10 (2.7s)	Train Loss: 0.8132	Train Acc: 74.67%	Val Loss: 1.1615	Val Acc: 61.76%
Epoch 09/10 (2.6s)	Train Loss: 0.7645	Train Acc: 76.34%	Val Loss: 1.1243	Val Acc: 63.24%
Epoch 10/10 (2.7s)	Train Loss: 0.6665	Train Acc: 80.92%	Val Loss: 1.1273	Val Acc: 63.97%

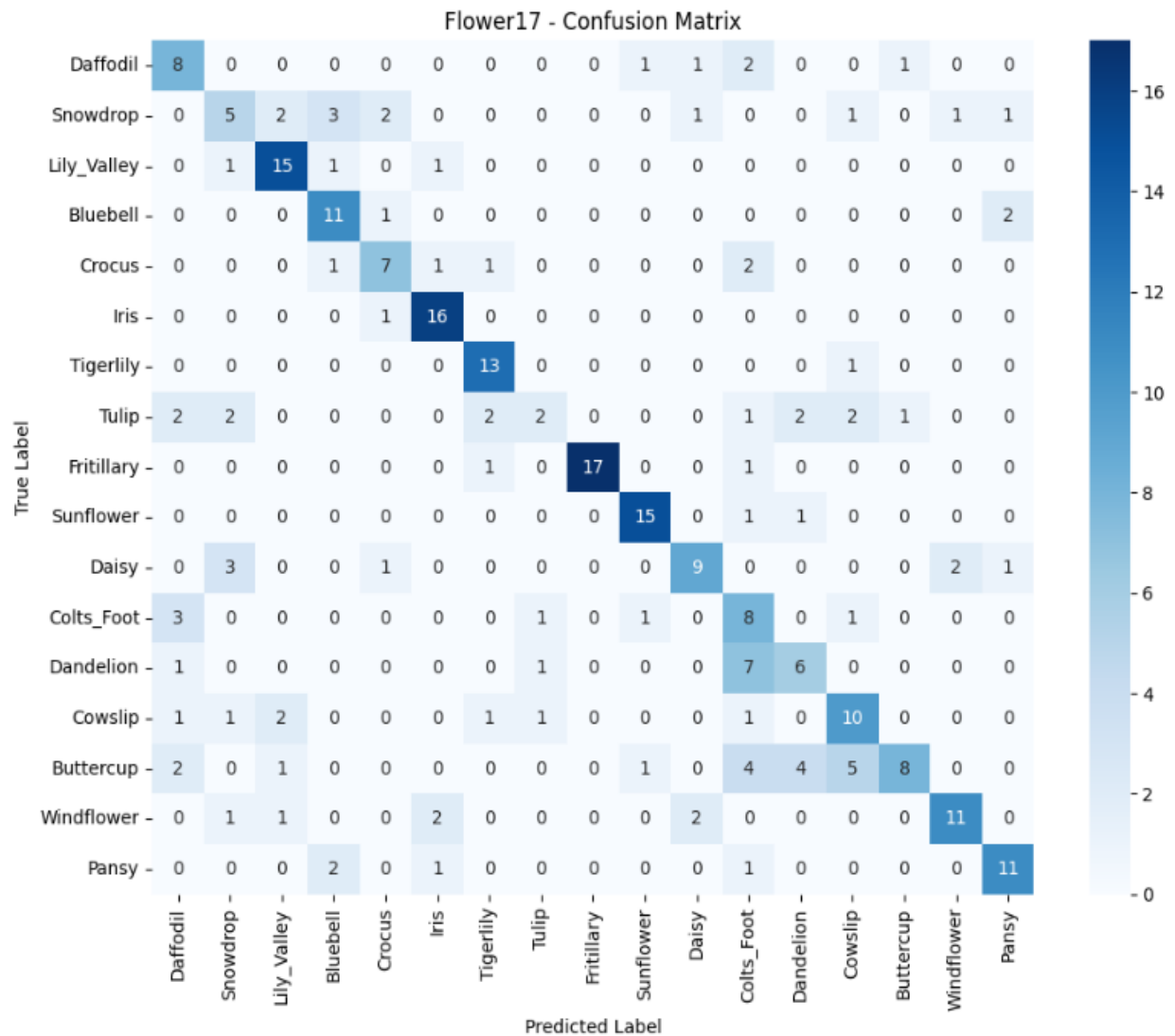
نتایج دقت در هر کلاس Flower17

[Flower17] Test Performance Evaluation:				
	precision	recall	f1-score	support
Daffodil	0.47	0.62	0.53	13
Snowdrop	0.38	0.31	0.34	16
Lily_Valley	0.71	0.83	0.77	18
Bluebell	0.61	0.79	0.69	14
Crocus	0.58	0.58	0.58	12
Iris	0.76	0.94	0.84	17
Tigerlily	0.72	0.93	0.81	14
Tulip	0.40	0.14	0.21	14
Fritillary	1.00	0.89	0.94	19
Sunflower	0.83	0.88	0.86	17
Daisy	0.69	0.56	0.62	16
Colts_Foot	0.29	0.57	0.38	14
Dandelion	0.46	0.40	0.43	15
Cowslip	0.50	0.59	0.54	17
Buttercup	0.80	0.32	0.46	25
Windflower	0.79	0.65	0.71	17
Pansy	0.73	0.73	0.73	15
accuracy			0.63	273
macro avg	0.63	0.63	0.62	273
weighted avg	0.65	0.63	0.62	273

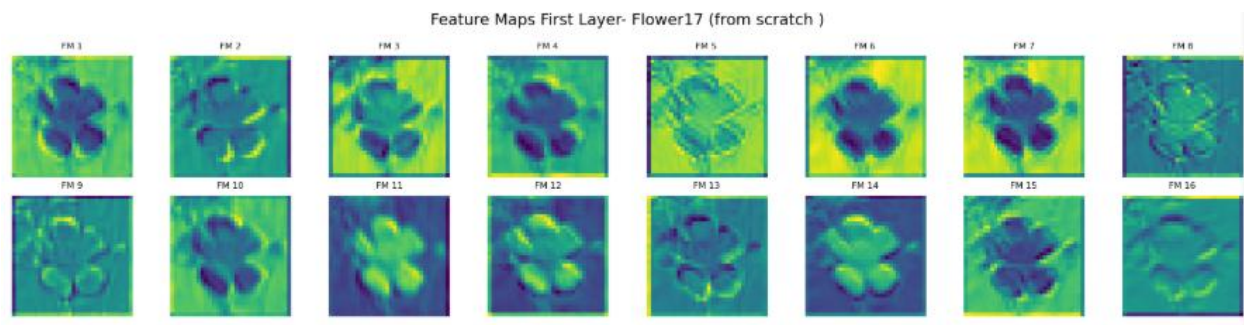
نمودار خطا و دقت CIFAR10



Confusion Matrix Flower17

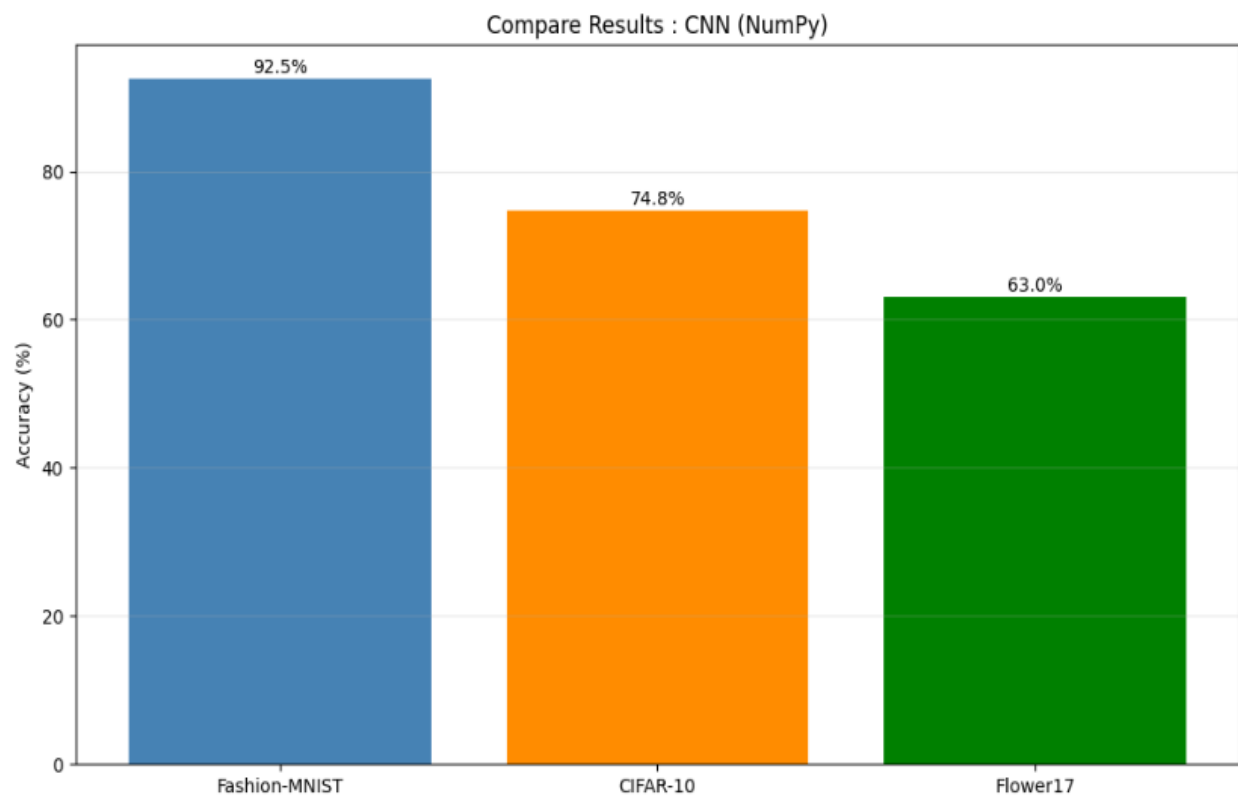


First Layer Feature Maps for Flower17



مقایسه نتایج بین 3 دیتاست

```
=====
Results: CNN (NumPy)
=====
Fashion-MNIST | Accuracy: 92.46% | F1: 0.9244
CIFAR-10      | Accuracy: 74.78% | F1: 0.7444
Flower17      | Accuracy: 63.00% | F1: 0.6151
=====
```

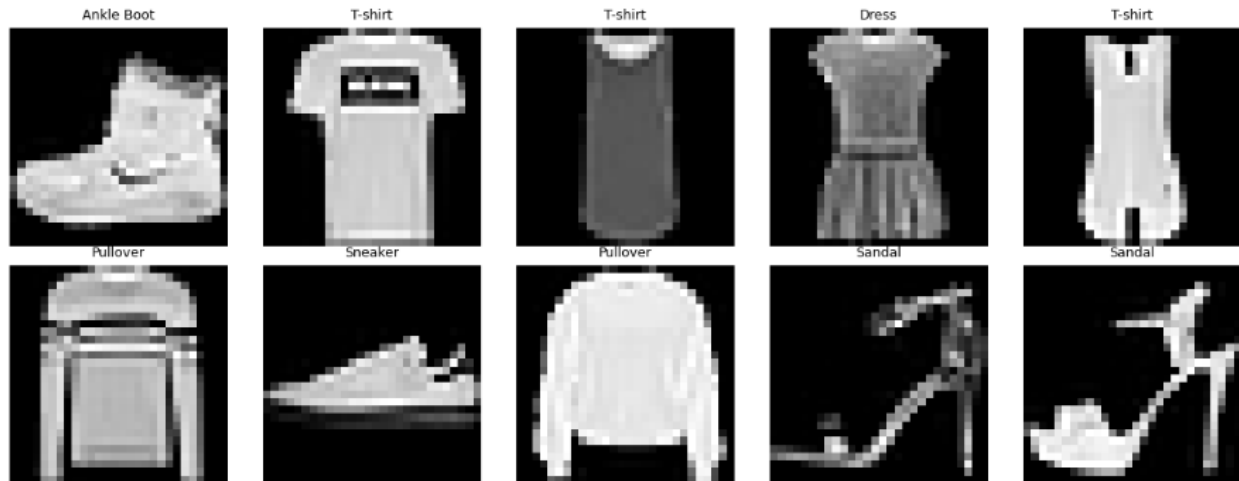


بخش PyTorch:

بارگذاری FashionMNIST و Sample Images

بارگذاری Fashion-MNIST از local...
تعداد کل: 70000 Fashion-MNIST
آموزش: 49000 | اعتبارسنجی: 7000 | آزمون: 14000

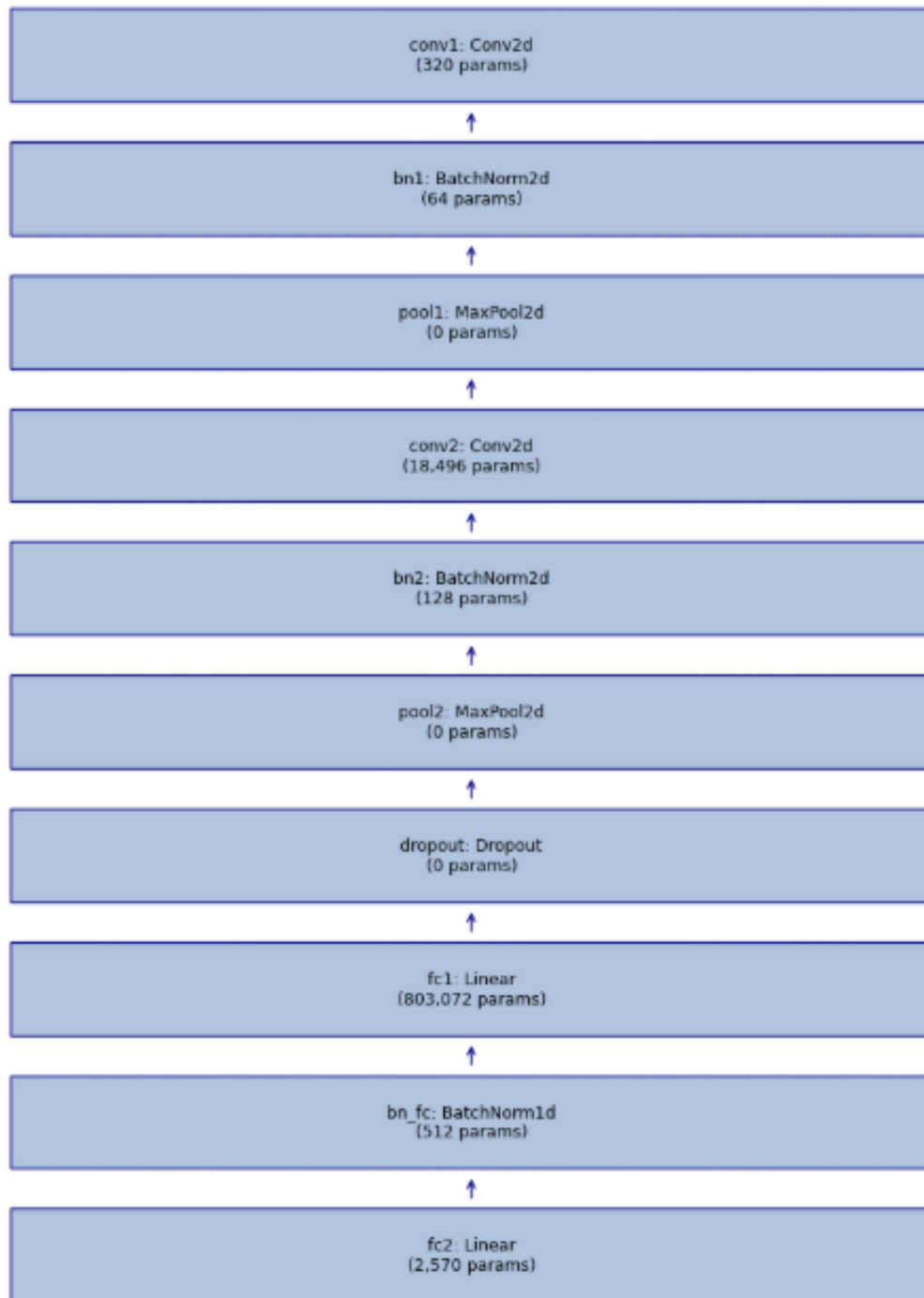
Sample Images - Fashion-MNIST



معماری مدل

```
.....  
فashion-MNIST معماری مدل  
CNN_PyTorch(  
    (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (pool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (pool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (dropout): Dropout(p=0.5, inplace=False)  
    (fc1): Linear(in_features=3136, out_features=256, bias=True)  
    (bn_fc): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (fc2): Linear(in_features=256, out_features=10, bias=True)  
)
```

Network Architecture - Fashion-MNIST



بازگزاری FashionMNIST و train شبکه

تعداد کل پارامترها: 825,162

--- Fashion-MNIST آموزش ---

Epoch 01/10		Train Loss: 0.4581		Train Acc: 83.78%		Val Loss: 0.2862		Val Acc: 89.50%
Epoch 02/10		Train Loss: 0.3407		Train Acc: 87.52%		Val Loss: 0.2562		Val Acc: 90.54%
Epoch 03/10		Train Loss: 0.3061		Train Acc: 88.71%		Val Loss: 0.2433		Val Acc: 91.01%
Epoch 04/10		Train Loss: 0.2824		Train Acc: 89.59%		Val Loss: 0.2377		Val Acc: 91.00%
Epoch 05/10		Train Loss: 0.2679		Train Acc: 90.14%		Val Loss: 0.2249		Val Acc: 91.53%
Epoch 06/10		Train Loss: 0.2532		Train Acc: 90.72%		Val Loss: 0.2267		Val Acc: 91.13%
Epoch 07/10		Train Loss: 0.2406		Train Acc: 91.01%		Val Loss: 0.2072		Val Acc: 92.14%
Epoch 08/10		Train Loss: 0.2164		Train Acc: 91.96%		Val Loss: 0.2008		Val Acc: 92.31%
Epoch 09/10		Train Loss: 0.2076		Train Acc: 92.26%		Val Loss: 0.1980		Val Acc: 92.61%
Epoch 10/10		Train Loss: 0.1996		Train Acc: 92.55%		Val Loss: 0.1951		Val Acc: 92.81%

نتایج دقت در هر کلاس (TRAIN)- FashionMNIST

===== Fashion-MNIST - Train =====

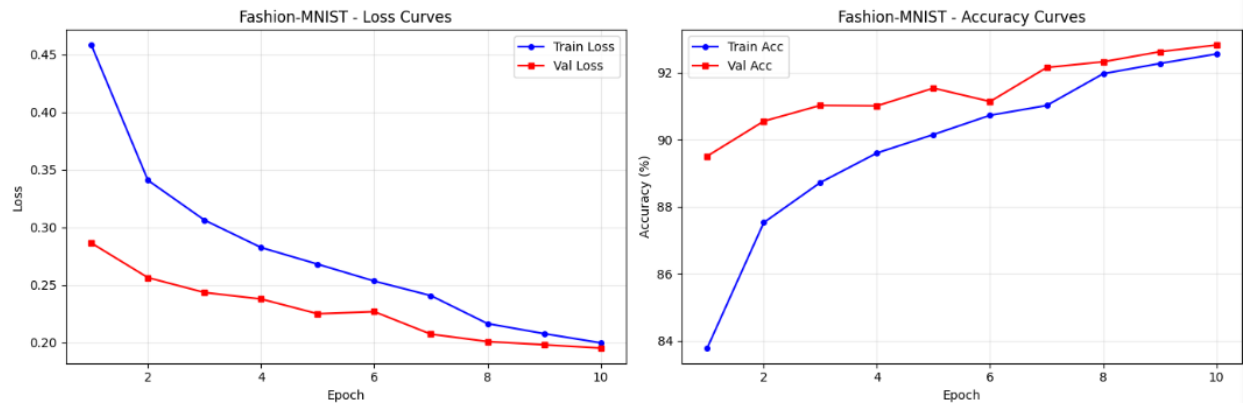
Accuracy: 95.58%
F1-score (Macro): 0.9556
F1-score (Weighted): 0.9556

گزارش کامل:

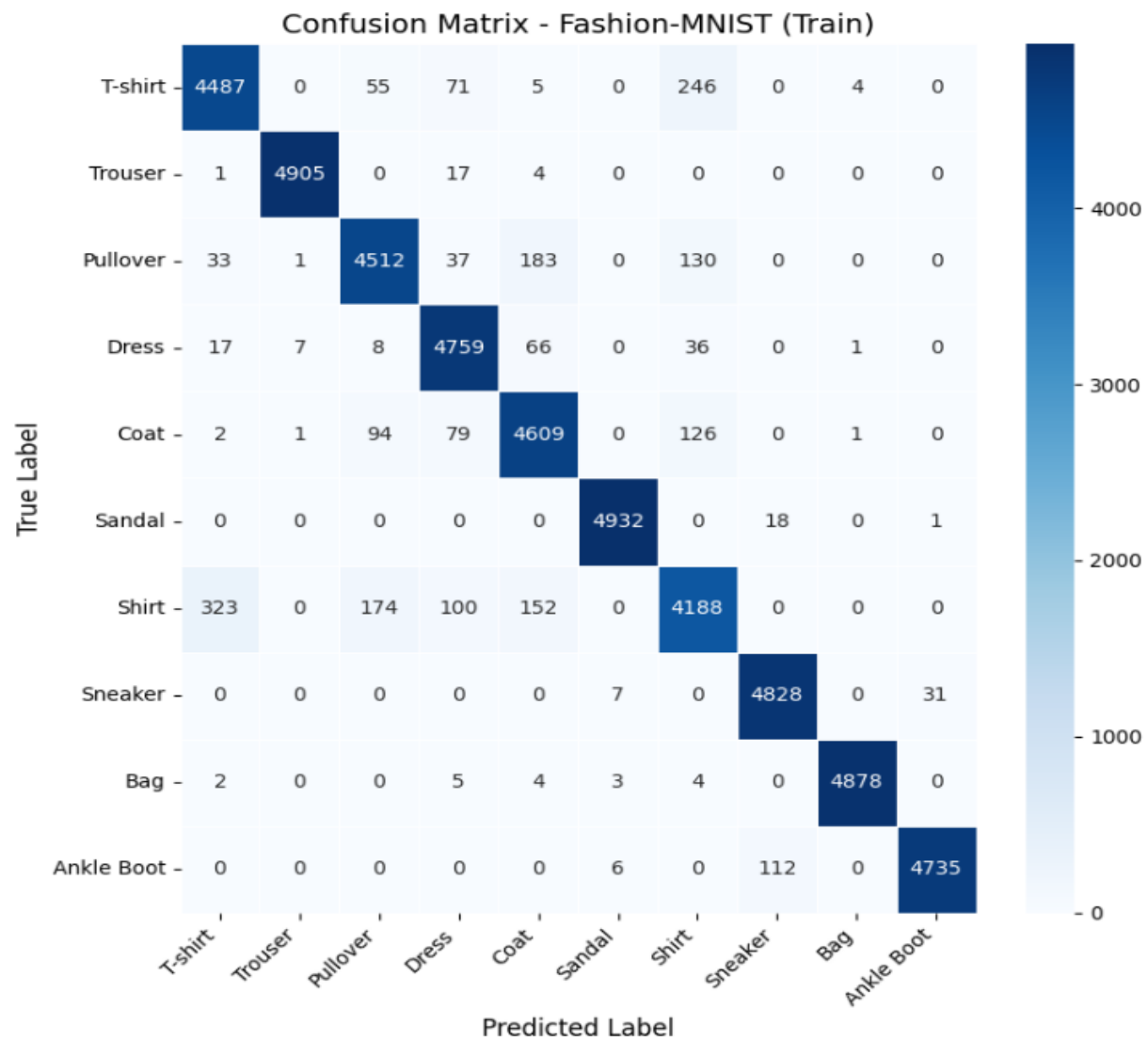
	precision	recall	f1-score	support
T-shirt	0.9223	0.9217	0.9220	4868
Trouser	0.9982	0.9955	0.9968	4927
Pullover	0.9317	0.9216	0.9266	4896
Dress	0.9390	0.9724	0.9554	4894
Coat	0.9176	0.9383	0.9278	4912
Sandal	0.9968	0.9962	0.9965	4951
Shirt	0.8854	0.8483	0.8665	4937
Sneaker	0.9738	0.9922	0.9829	4866
Bag	0.9988	0.9963	0.9975	4896
Ankle Boot	0.9933	0.9757	0.9844	4853
accuracy			0.9558	49000
macro avg	0.9557	0.9558	0.9556	49000
weighted avg	0.9557	0.9558	0.9556	49000

نمودار خطا و دقت FashionMNIST

PyTorch CNN - Fashion-MNIST



Confusion Matrix FashionMNIST (Train)



نتایج دقت در هر کلاس FashionMNIST (TEST)

===== Fashion-MNIST - Test =====

Accuracy: 92.82%

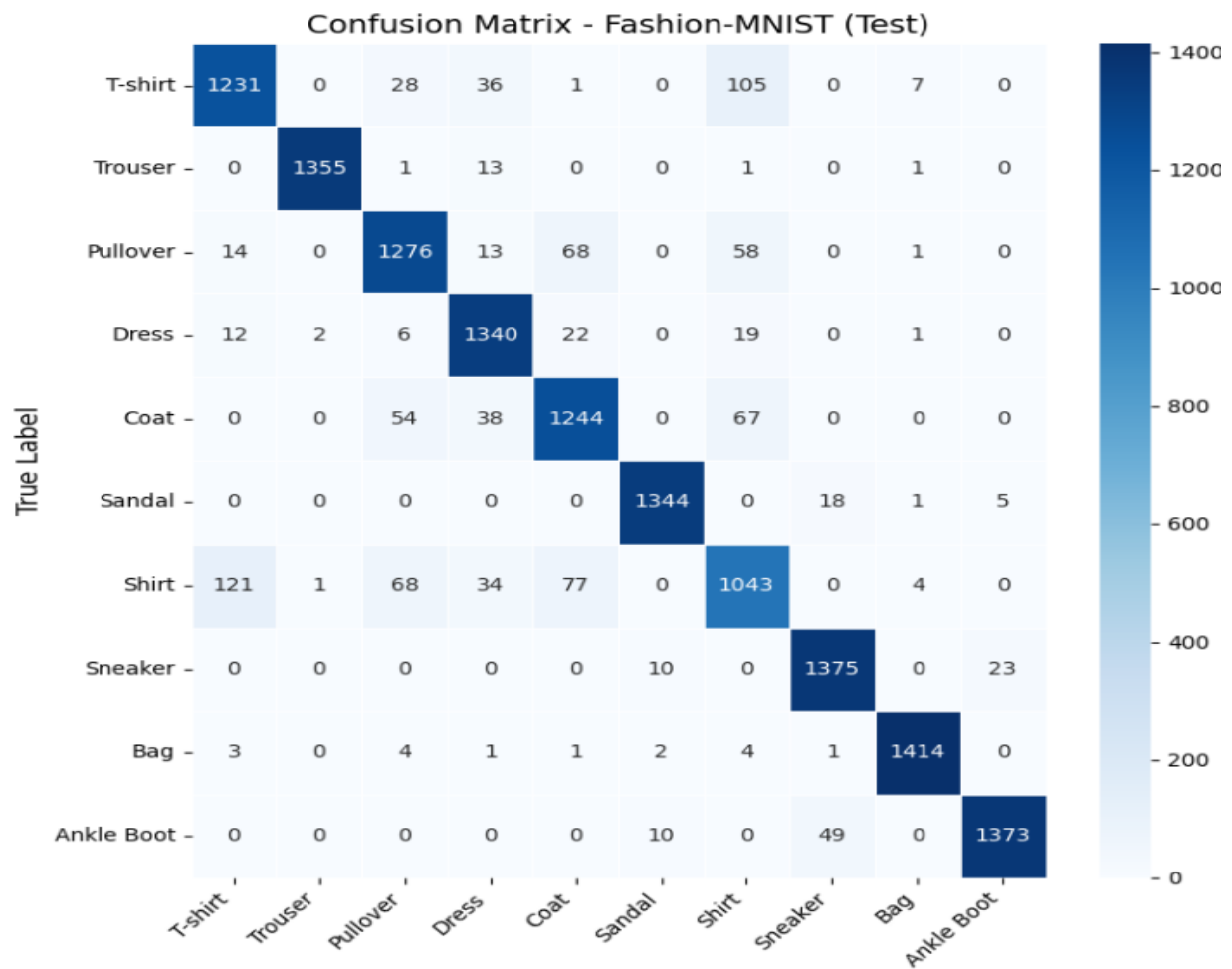
F1-score (Macro): 0.9276

F1-score (Weighted): 0.9280

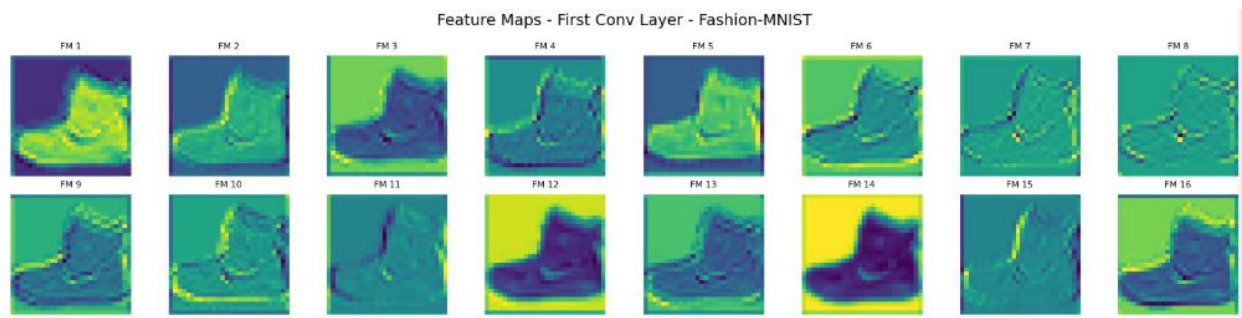
گزارش کامل:

	precision	recall	f1-score	support
T-shirt	0.8914	0.8743	0.8828	1408
Trouser	0.9978	0.9883	0.9930	1371
Pullover	0.8880	0.8923	0.8901	1430
Dress	0.9085	0.9558	0.9315	1402
Coat	0.8804	0.8867	0.8835	1403
Sandal	0.9839	0.9825	0.9832	1368
Shirt	0.8042	0.7737	0.7887	1348
Sneaker	0.9529	0.9766	0.9646	1408
Bag	0.9895	0.9888	0.9892	1430
Ankle Boot	0.9800	0.9588	0.9693	1432
accuracy			0.9282	14000
macro avg	0.9276	0.9278	0.9276	14000
weighted avg	0.9280	0.9282	0.9280	14000

Confusion Matrix FashionMNIST (TEST)



First CONV Layer Feature Maps for FashionMNIST

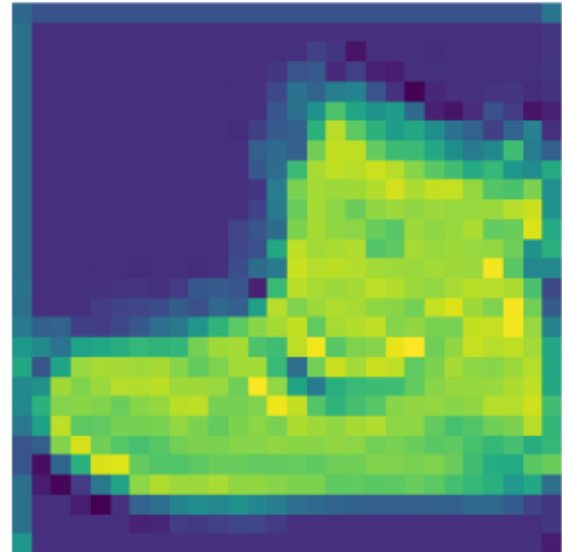


Original vs Feature Map - Fashion-MNIST

Original Image



Feature Map 1



✓ Fashion-MNIST model saved → best_model_fashion_mnist.pth

خروجی های مربوط به CIFAR10 – PyTorch

local... از CIFAR-10 بارگذاری

تعداد کل: 60000 - CIFAR-10

آموزش: 42000 | اعتبارسنجی: 6000 | آزمون: 12000

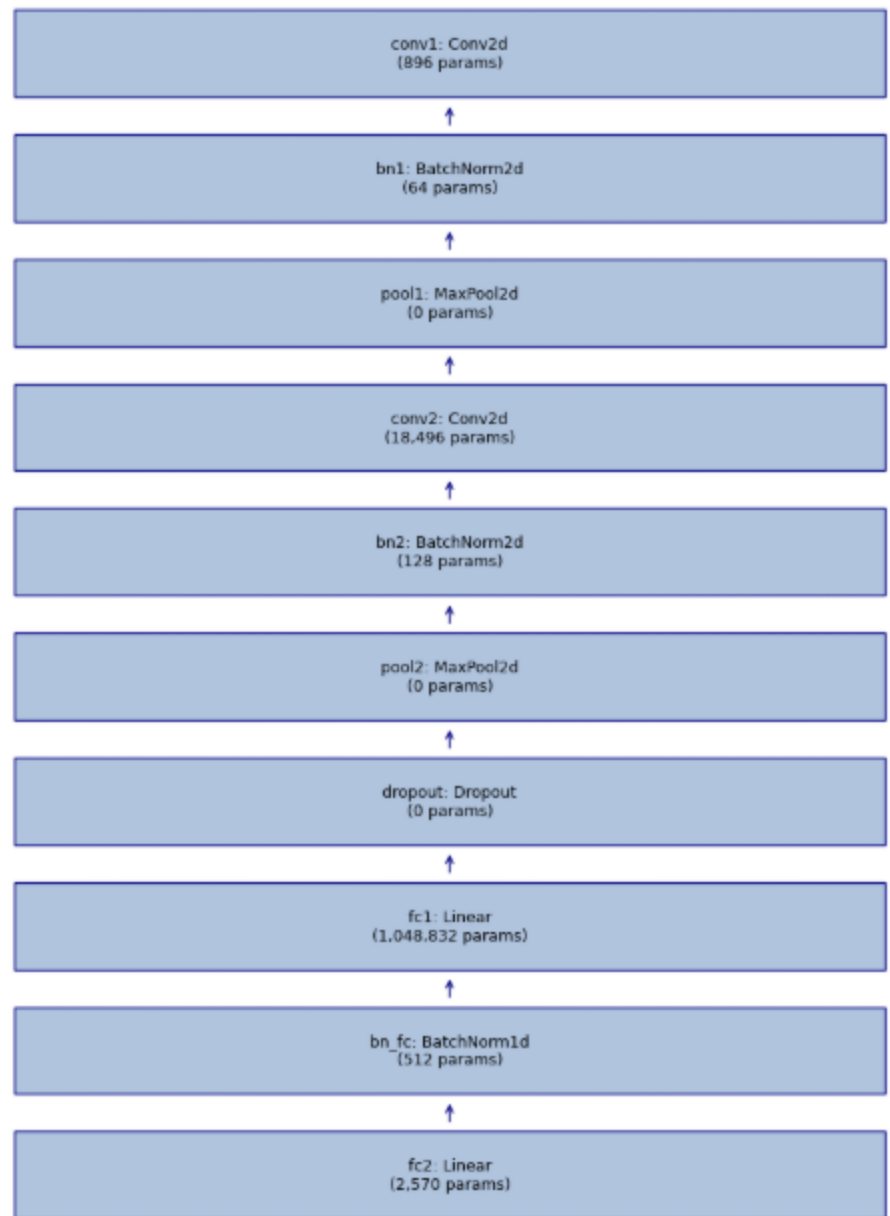
Sample Images - CIFAR-10



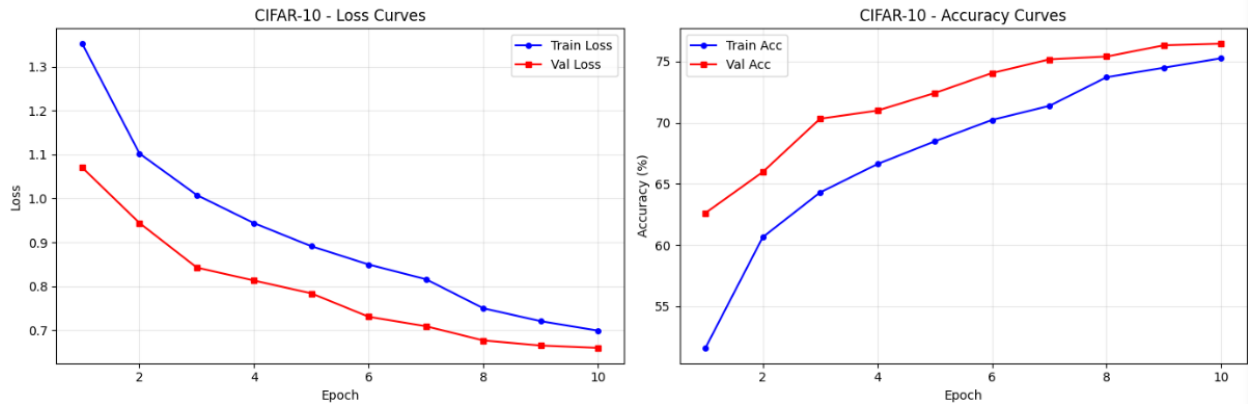
--- آموزش CIFAR-10 ---

Epoch 01/10	Train Loss: 1.3523	Train Acc: 51.59%	Val Loss: 1.0704	Val Acc: 62.62%
Epoch 02/10	Train Loss: 1.1023	Train Acc: 60.64%	Val Loss: 0.9442	Val Acc: 65.97%
Epoch 03/10	Train Loss: 1.0078	Train Acc: 64.30%	Val Loss: 0.8428	Val Acc: 70.32%
Epoch 04/10	Train Loss: 0.9438	Train Acc: 66.61%	Val Loss: 0.8132	Val Acc: 70.98%
Epoch 05/10	Train Loss: 0.8912	Train Acc: 68.47%	Val Loss: 0.7840	Val Acc: 72.42%
Epoch 06/10	Train Loss: 0.8496	Train Acc: 70.22%	Val Loss: 0.7307	Val Acc: 74.05%
Epoch 07/10	Train Loss: 0.8161	Train Acc: 71.37%	Val Loss: 0.7092	Val Acc: 75.17%
Epoch 08/10	Train Loss: 0.7501	Train Acc: 73.71%	Val Loss: 0.6769	Val Acc: 75.40%
Epoch 09/10	Train Loss: 0.7208	Train Acc: 74.49%	Val Loss: 0.6653	Val Acc: 76.32%
Epoch 10/10	Train Loss: 0.6992	Train Acc: 75.26%	Val Loss: 0.6599	Val Acc: 76.45%

Network Architecture - CIFAR-10



PyTorch CNN - CIFAR-10



===== CIFAR-10 - Train =====

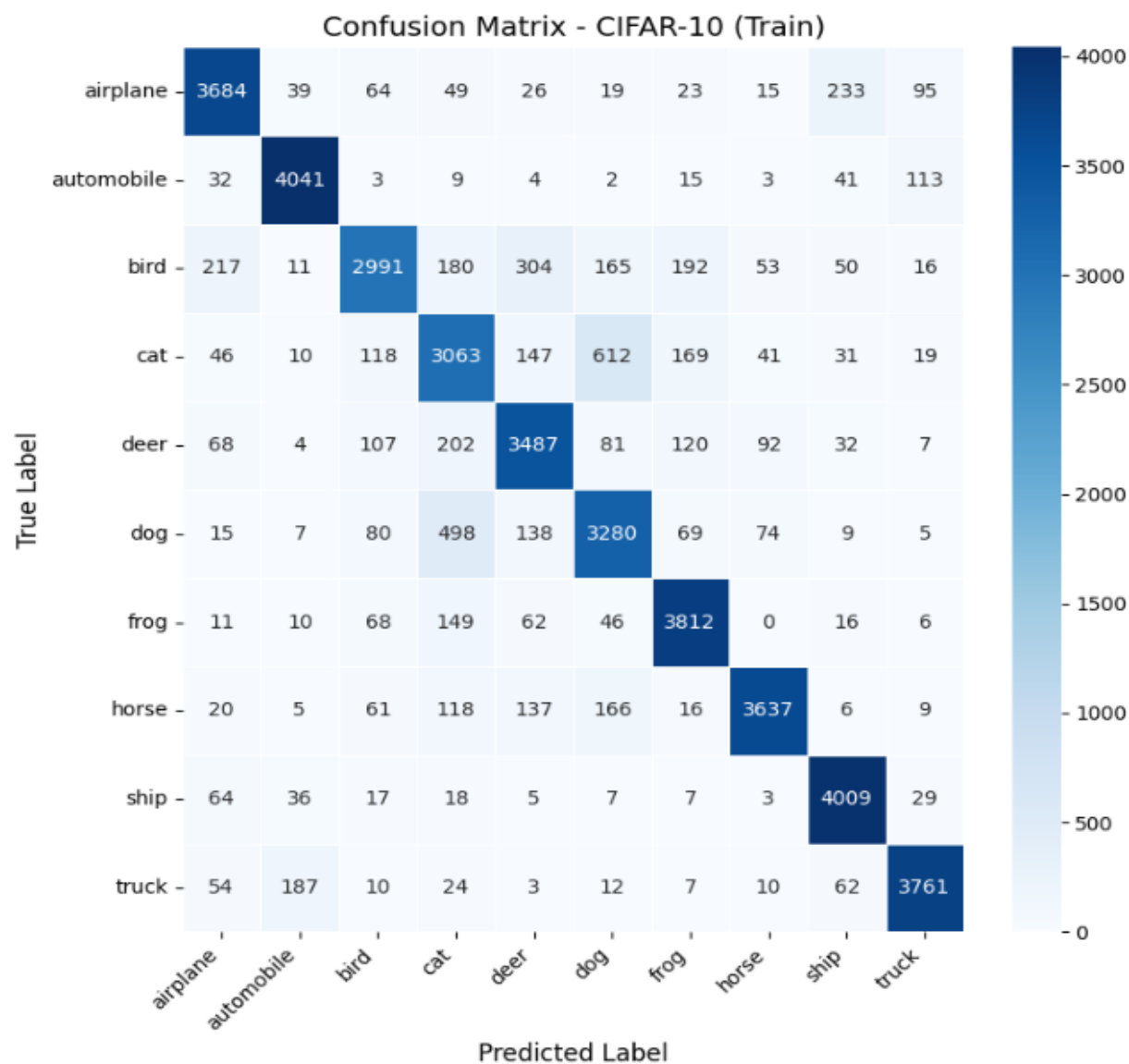
Accuracy: 85.15%

F1-score (Macro): 0.8512

F1-score (Weighted): 0.8511

گزارش کامل:

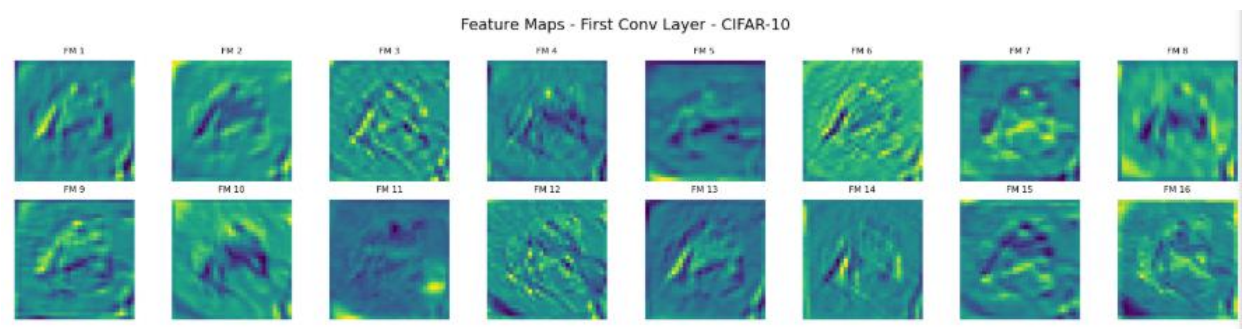
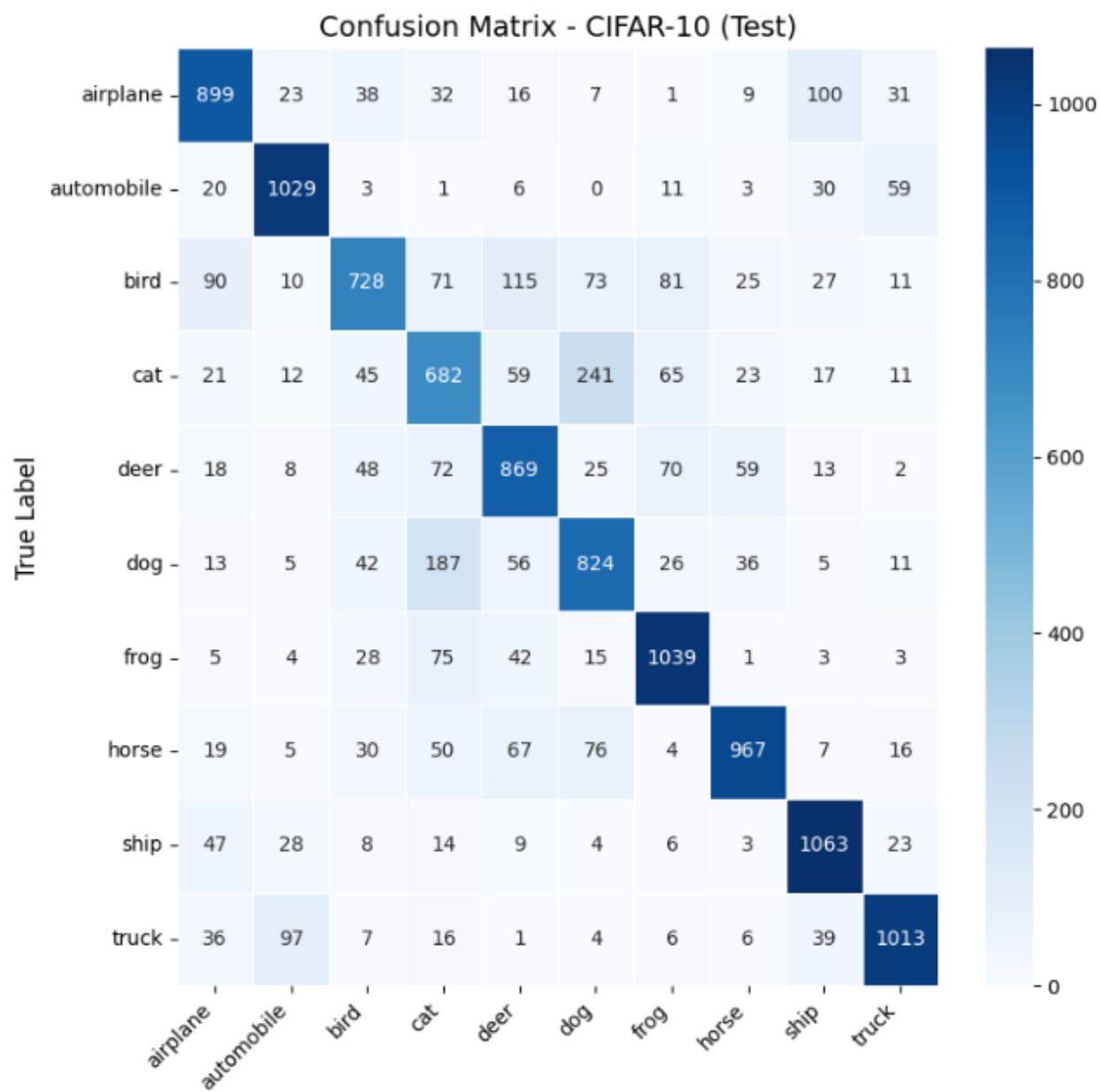
	precision	recall	f1-score	support
airplane	0.8749	0.8674	0.8711	4247
automobile	0.9290	0.9479	0.9383	4263
bird	0.8500	0.7157	0.7771	4179
cat	0.7107	0.7197	0.7152	4256
deer	0.8085	0.8302	0.8192	4200
dog	0.7472	0.7856	0.7659	4175
frog	0.8605	0.9120	0.8855	4180
horse	0.9259	0.8711	0.8977	4175
ship	0.8931	0.9557	0.9233	4195
truck	0.9264	0.9107	0.9184	4130
accuracy			0.8515	42000
macro avg	0.8526	0.8516	0.8512	42000
weighted avg	0.8524	0.8515	0.8511	42000



==== CIFAR-10 - Test =====
Accuracy: 75.94%
F1-score (Macro): 0.7582
F1-score (Weighted): 0.7584

نگاراش کامل:

	precision	recall	f1-score	support
airplane	0.7697	0.7777	0.7737	1156
automobile	0.8428	0.8855	0.8636	1162
bird	0.7451	0.5914	0.6594	1231
cat	0.5683	0.5799	0.5741	1176
deer	0.7008	0.7340	0.7170	1184
dog	0.6493	0.6838	0.6661	1205
frog	0.7937	0.8551	0.8233	1215
horse	0.8542	0.7792	0.8150	1241
ship	0.8152	0.8822	0.8473	1205
truck	0.8585	0.8269	0.8424	1225
accuracy			0.7594	12000
macro avg	0.7598	0.7596	0.7582	12000
weighted avg	0.7604	0.7594	0.7584	12000

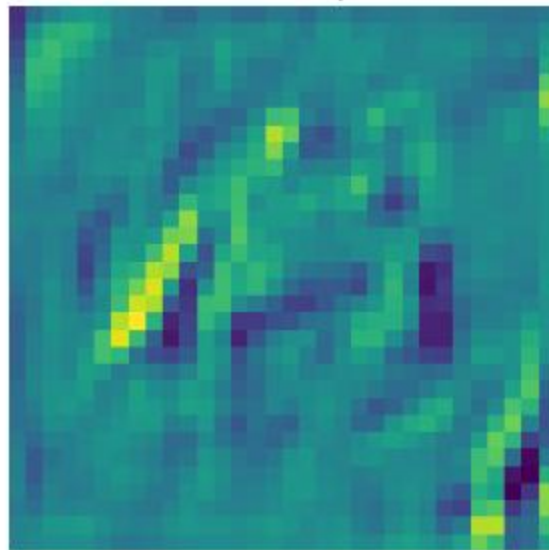


Original vs Feature Map - CIFAR-10

Original Image



Feature Map 1



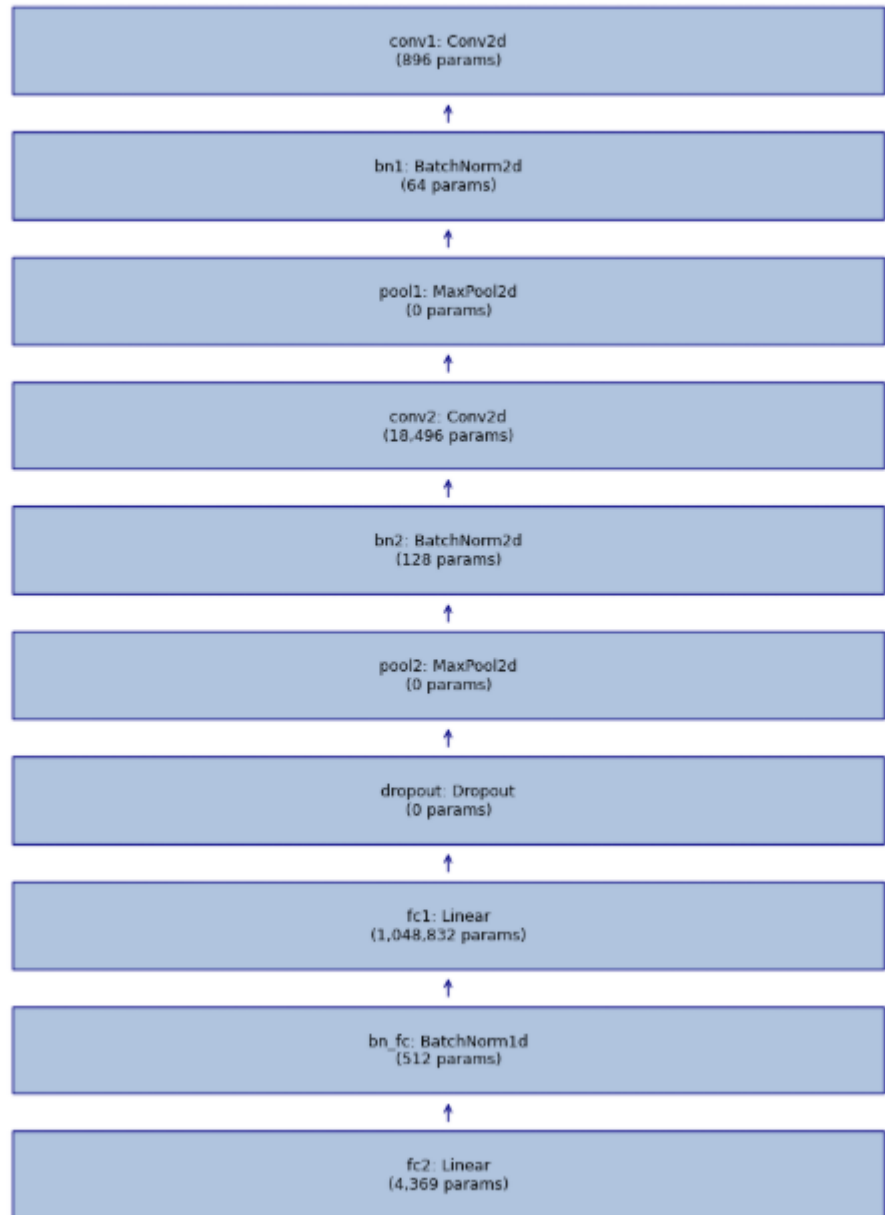
خروجی های مربوط به PyTorch – FLOWER17

Sample Images - Flower17



نمونه شد Flower17.
تعداد کل پارامترها (Flower17): 1,073,297

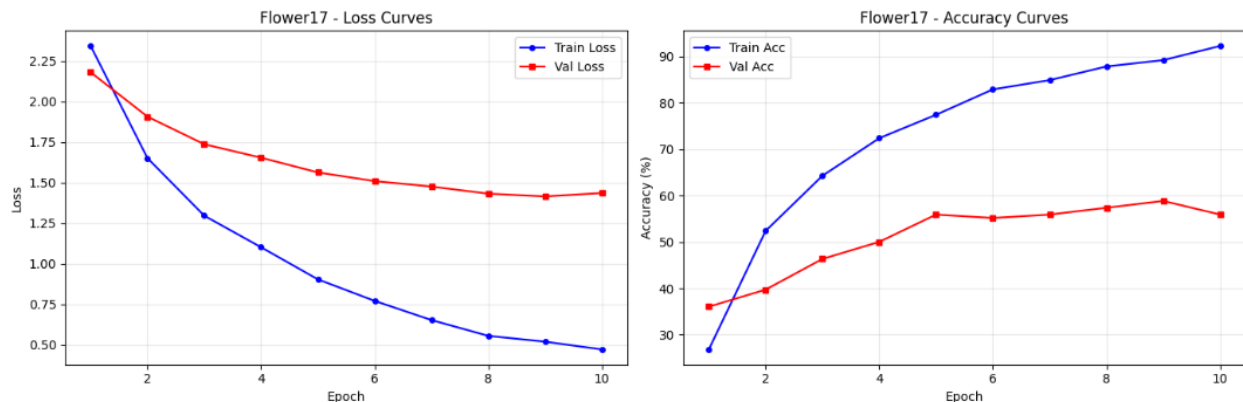
Network Architecture - Flower17



--- آموزش Flower17 ---

Epoch 01/10	Train Loss: 2.3417	Train Acc: 26.81%	Val Loss: 2.1797	Val Acc: 36.03%
Epoch 02/10	Train Loss: 1.6504	Train Acc: 52.37%	Val Loss: 1.9061	Val Acc: 39.71%
Epoch 03/10	Train Loss: 1.2957	Train Acc: 64.25%	Val Loss: 1.7361	Val Acc: 46.32%
Epoch 04/10	Train Loss: 1.1013	Train Acc: 72.34%	Val Loss: 1.6533	Val Acc: 50.00%
Epoch 05/10	Train Loss: 0.9033	Train Acc: 77.39%	Val Loss: 1.5624	Val Acc: 55.88%
Epoch 06/10	Train Loss: 0.7707	Train Acc: 82.86%	Val Loss: 1.5089	Val Acc: 55.15%
Epoch 07/10	Train Loss: 0.6525	Train Acc: 84.86%	Val Loss: 1.4753	Val Acc: 55.88%
Epoch 08/10	Train Loss: 0.5552	Train Acc: 87.80%	Val Loss: 1.4314	Val Acc: 57.35%
Epoch 09/10	Train Loss: 0.5194	Train Acc: 89.17%	Val Loss: 1.4146	Val Acc: 58.82%
Epoch 10/10	Train Loss: 0.4714	Train Acc: 92.22%	Val Loss: 1.4354	Val Acc: 55.88%

PyTorch CNN - Flower17

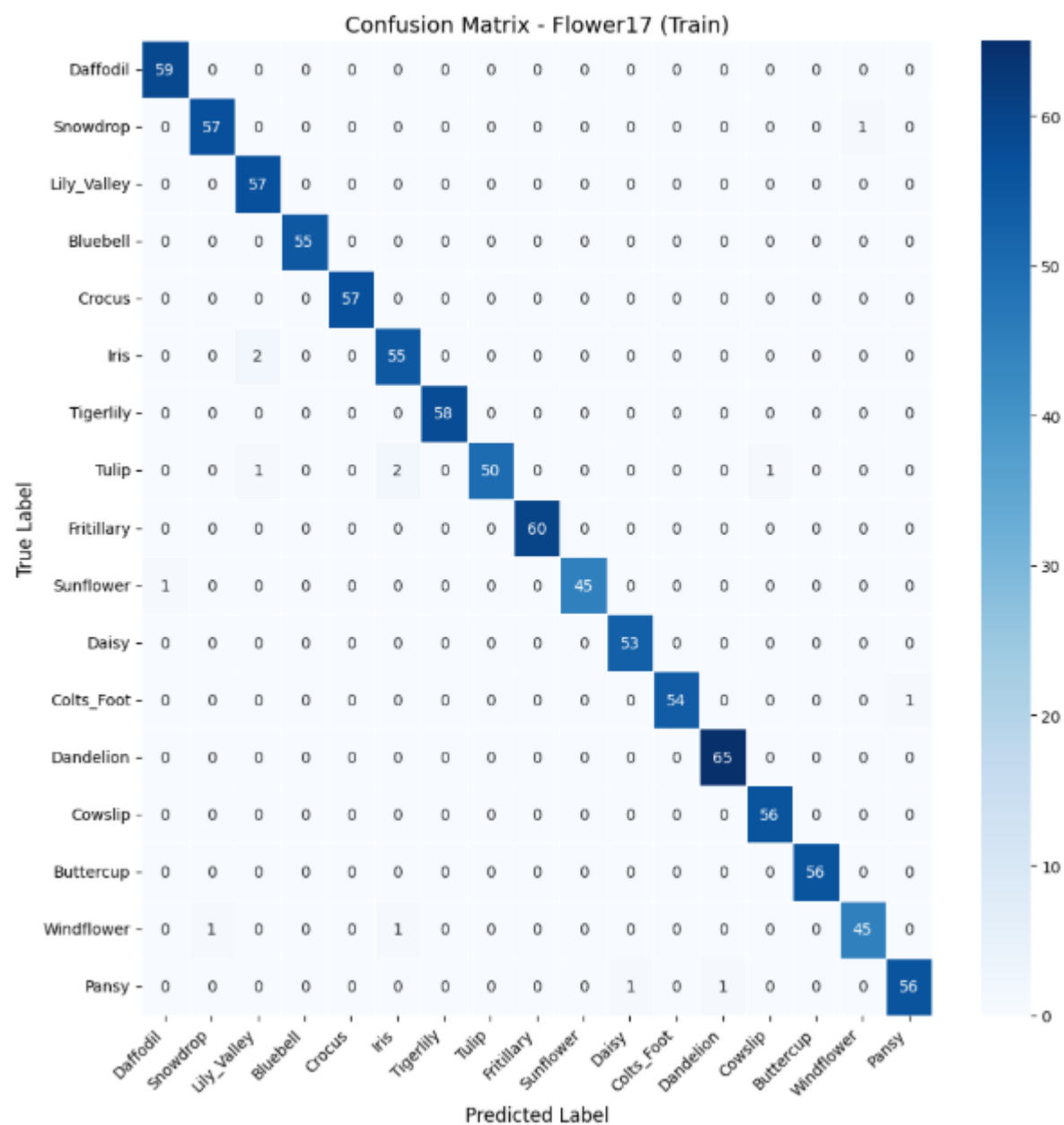


===== Flower17 - Train =====

Accuracy: 98.63%
 F1-score (Macro): 0.9860
 F1-score (Weighted): 0.9863

گزارش کامل:

	precision	recall	f1-score	support
Daffodil	0.9833	1.0000	0.9916	59
Snowdrop	0.9828	0.9828	0.9828	58
Lily_Valley	0.9500	1.0000	0.9744	57
Bluebell	1.0000	1.0000	1.0000	55
Crocus	1.0000	1.0000	1.0000	57
Iris	0.9483	0.9649	0.9565	57
Tigerlily	1.0000	1.0000	1.0000	58
Tulip	1.0000	0.9259	0.9615	54
Fritillary	1.0000	1.0000	1.0000	60
Sunflower	1.0000	0.9783	0.9890	46
Daisy	0.9815	1.0000	0.9907	53
Colts_Foot	1.0000	0.9818	0.9908	55
Dandelion	0.9848	1.0000	0.9924	65
Cowslip	0.9825	1.0000	0.9912	56
Buttercup	1.0000	1.0000	1.0000	56
Windflower	0.9783	0.9574	0.9677	47
Pansy	0.9825	0.9655	0.9739	58
accuracy			0.9863	951
macro avg	0.9867	0.9857	0.9860	951
weighted avg	0.9866	0.9863	0.9863	951



===== Flower17 - Test =====

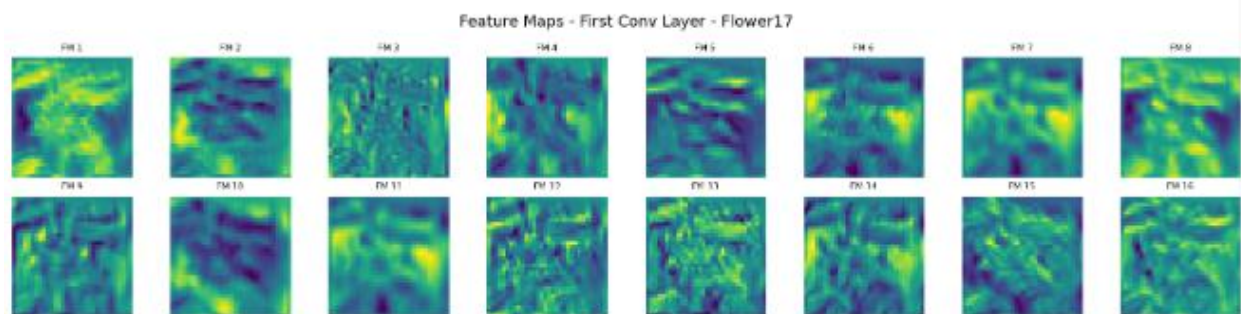
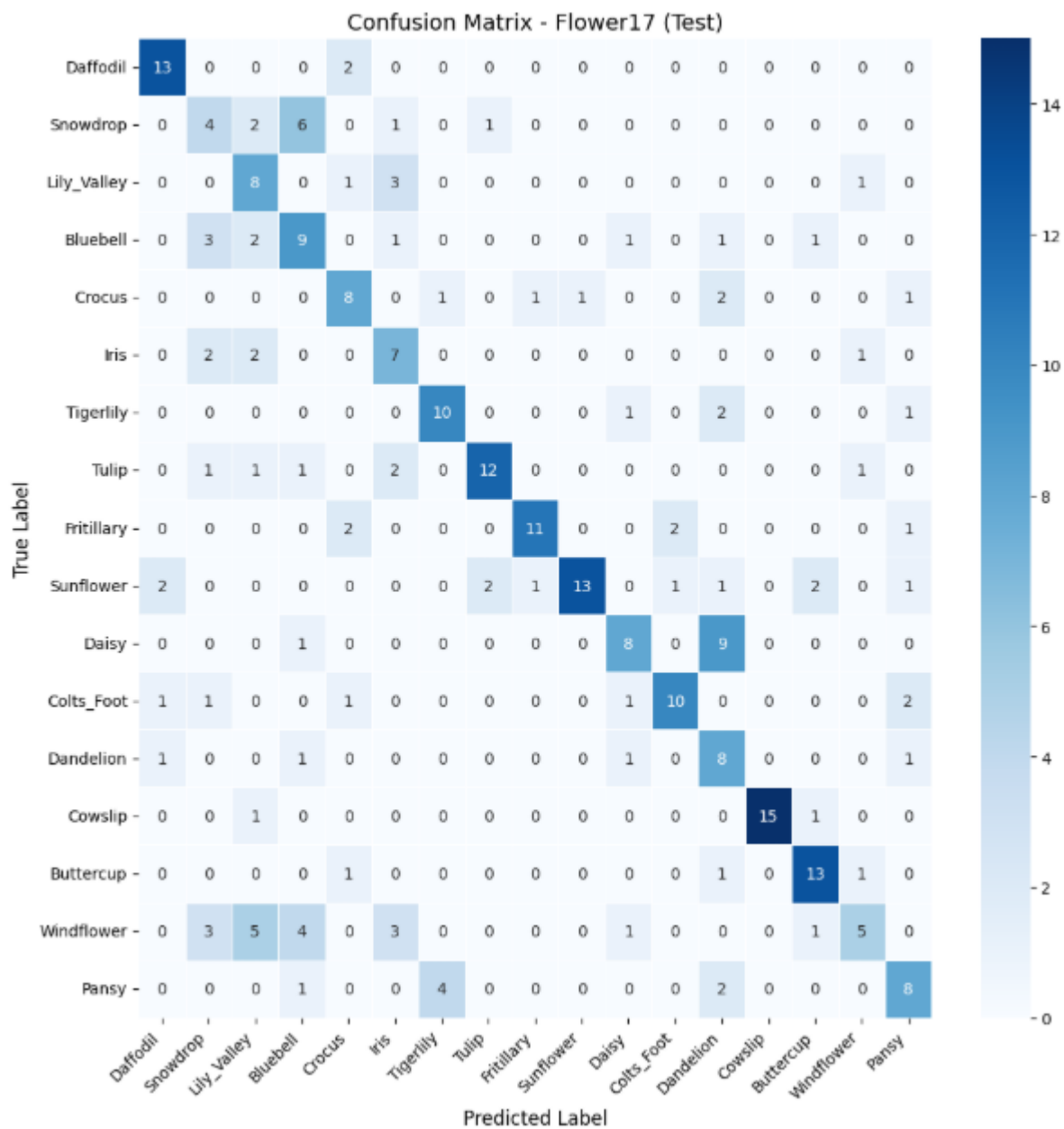
Accuracy: 59.34%

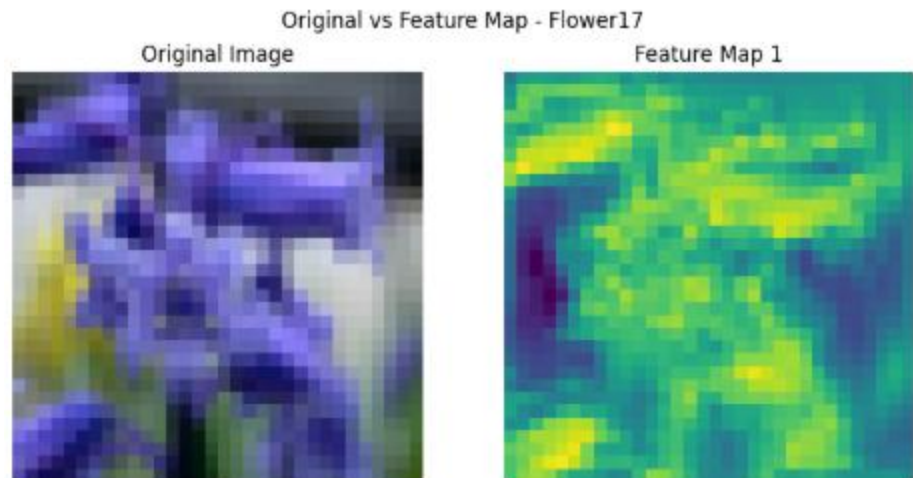
F1-score (Macro): 0.5944

F1-score (Weighted): 0.5986

گزارش کامل:

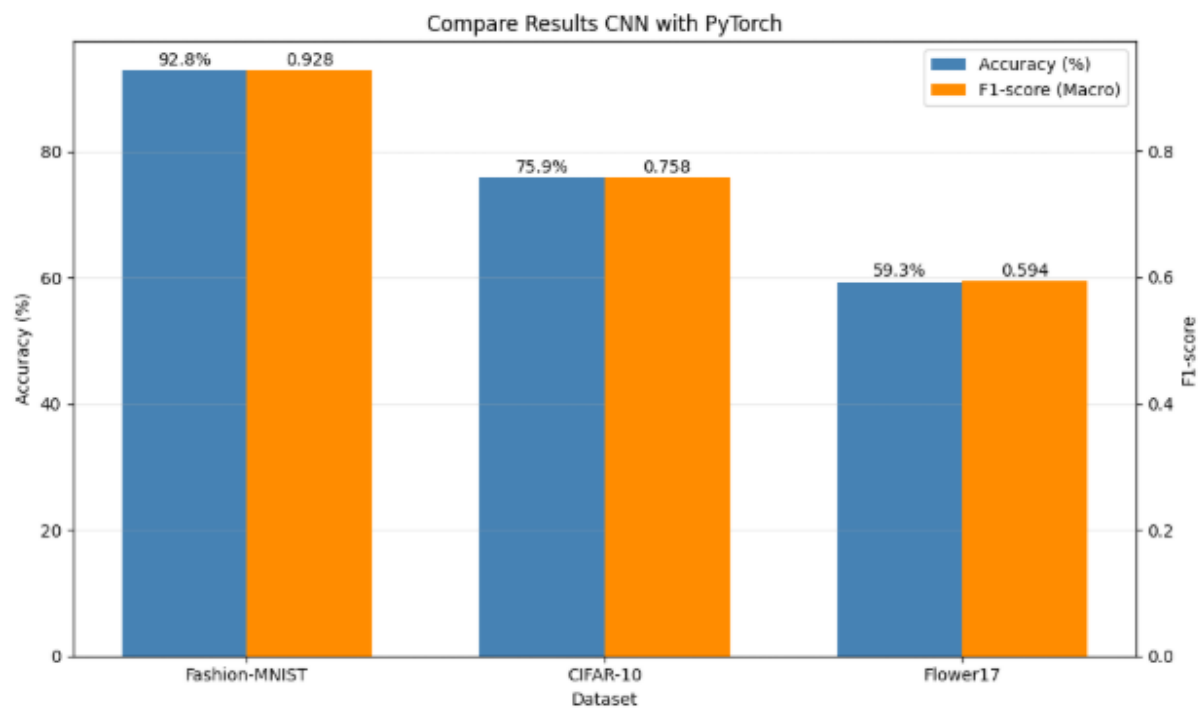
	precision	recall	f1-score	support
Daffodil	0.7647	0.8667	0.8125	15
Snowdrop	0.2857	0.2857	0.2857	14
Lily_Valley	0.3810	0.6154	0.4706	13
Bluebell	0.3913	0.5000	0.4390	18
Crocus	0.5333	0.5714	0.5517	14
Iris	0.4118	0.5833	0.4828	12
Tigerlily	0.6667	0.7143	0.6897	14
Tulip	0.8000	0.6667	0.7273	18
Fritillary	0.8462	0.6875	0.7586	16
Sunflower	0.9286	0.5652	0.7027	23
Daisy	0.6154	0.4444	0.5161	18
Colts_Foot	0.7692	0.6250	0.6897	16
Dandelion	0.3077	0.6667	0.4211	12
Cowslip	1.0000	0.8824	0.9375	17
Buttercup	0.7222	0.8125	0.7647	16
Windflower	0.5556	0.2273	0.3226	22
Pansy	0.5333	0.5333	0.5333	15
accuracy			0.5934	273
macro avg	0.6184	0.6028	0.5944	273
weighted avg	0.6387	0.5934	0.5986	273



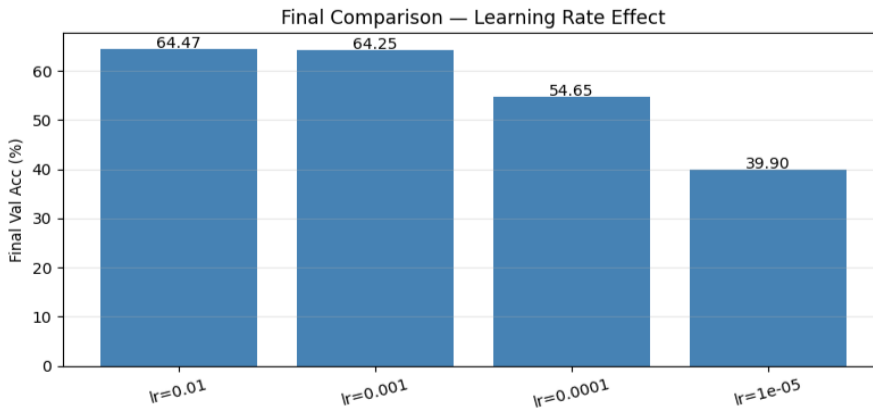
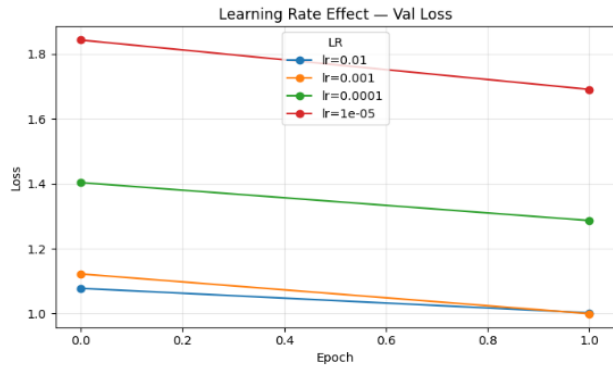
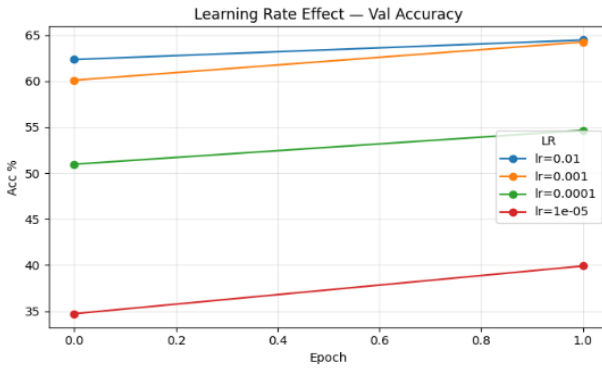


مقایسه نتایج بین 3 دیتاست - PyTorch

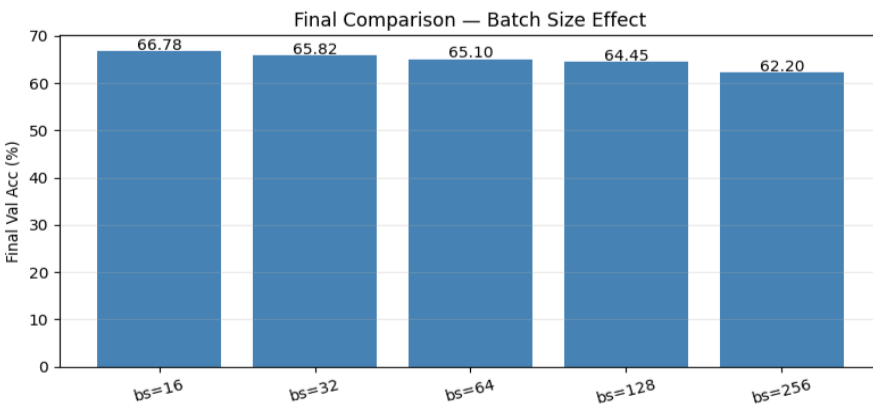
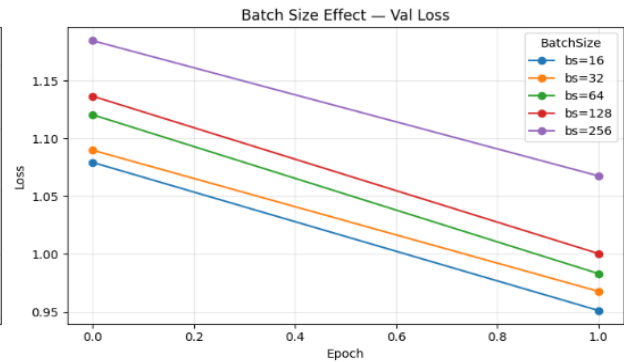
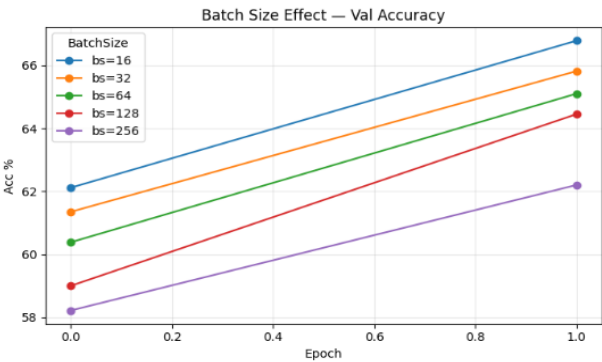
Results CNN with PyTorch			
Fashion-MNIST	Accuracy: 92.82%	F1-score: 0.9276	
CIFAR-10	Accuracy: 75.94%	F1-score: 0.7582	
Flower17	Accuracy: 59.34%	F1-score: 0.5944	

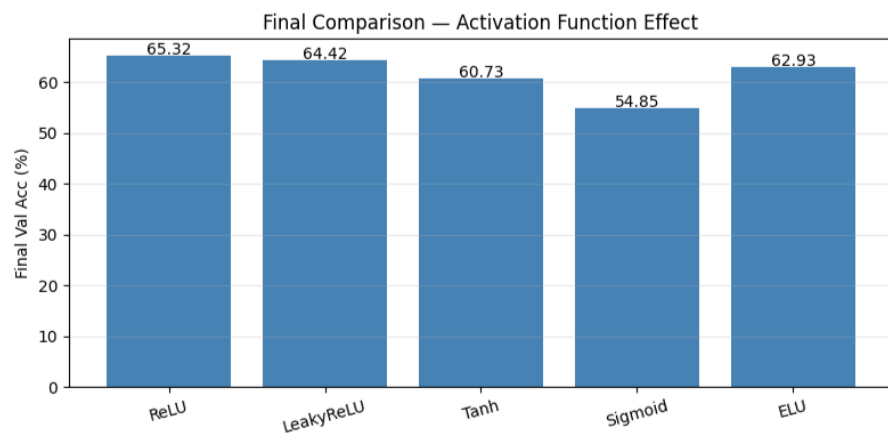
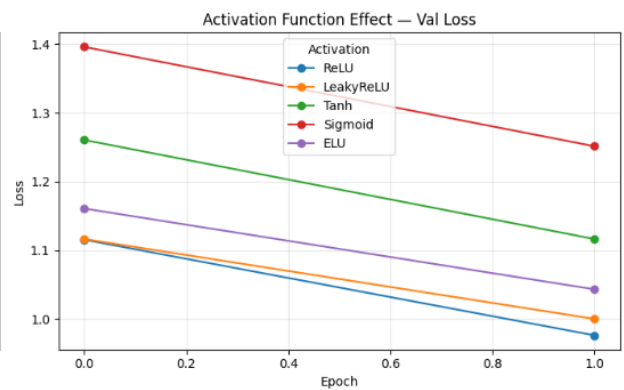
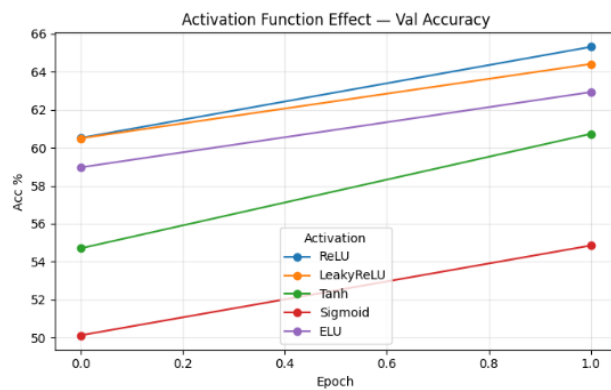


Hyper Sensitivity for CIFAR10

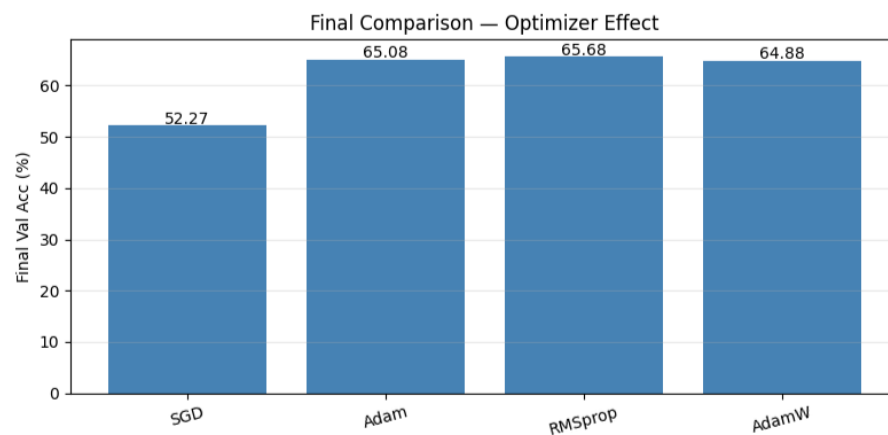
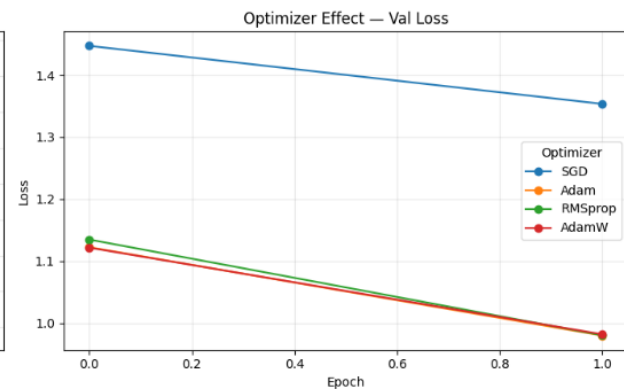
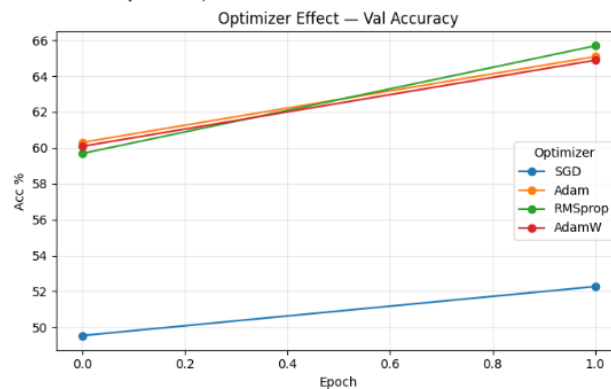


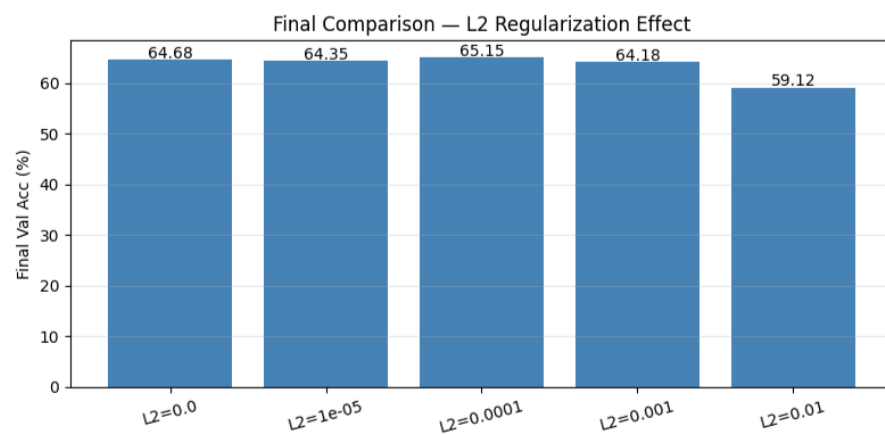
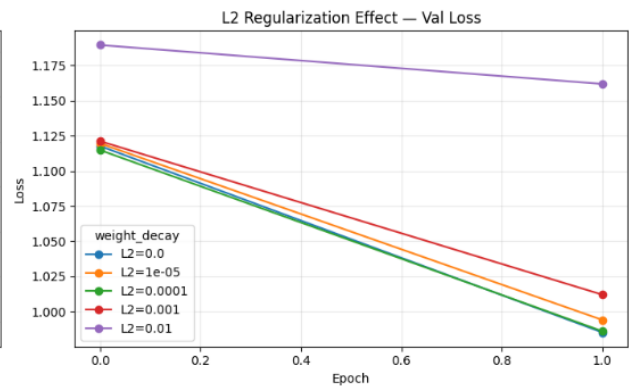
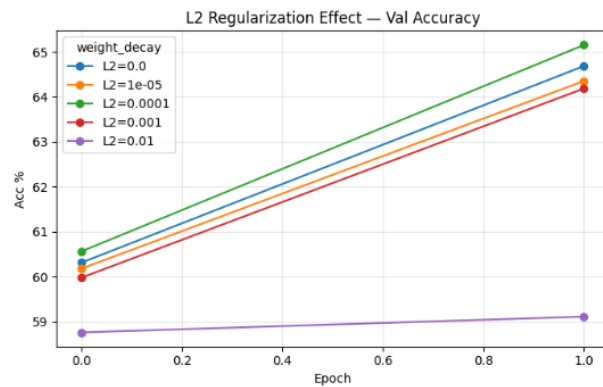
>>> Sensitivity 2/8 : Batch Size



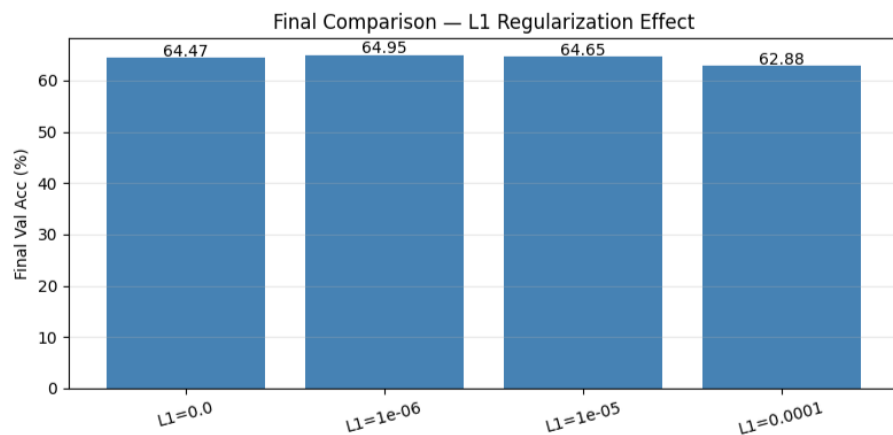
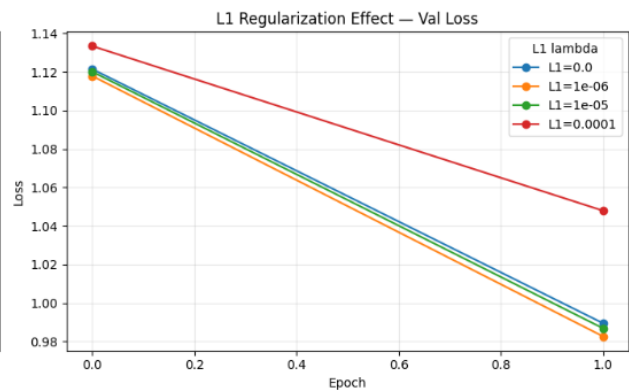
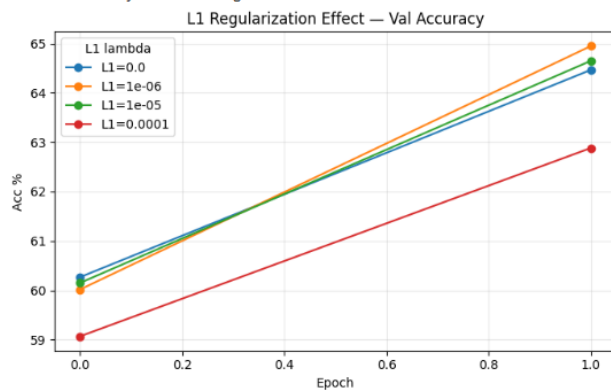


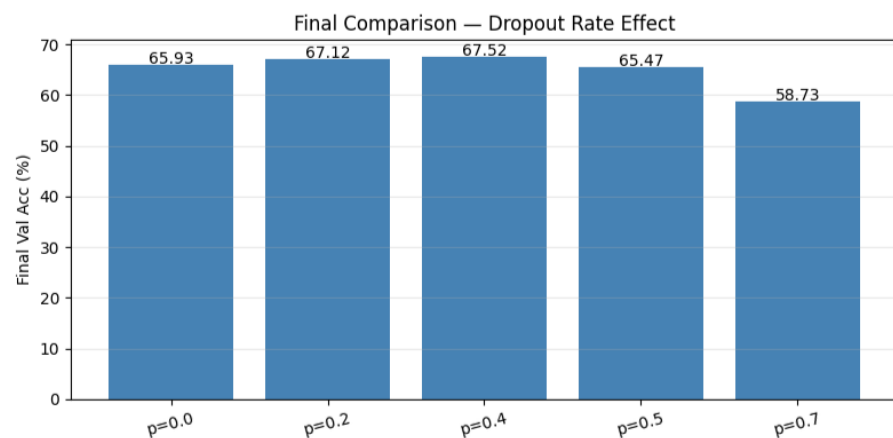
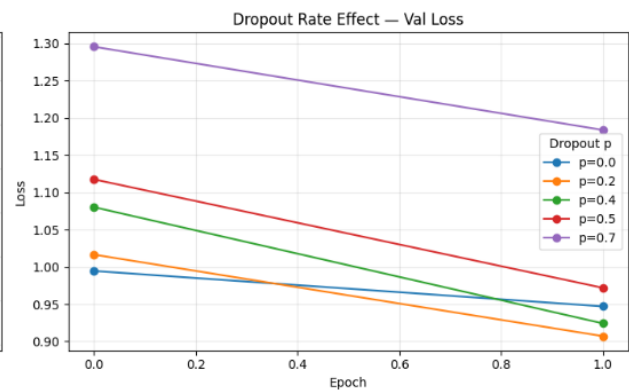
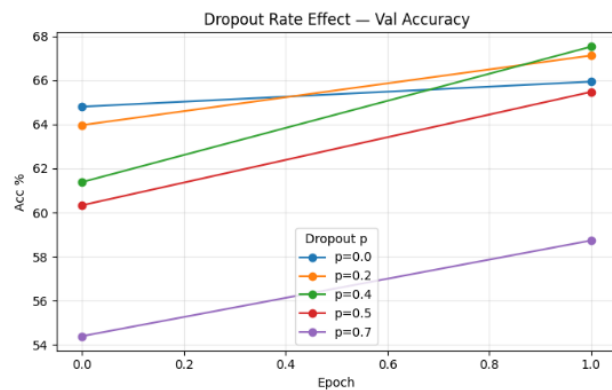
>>> Sensitivity 4/8 : Optimizer



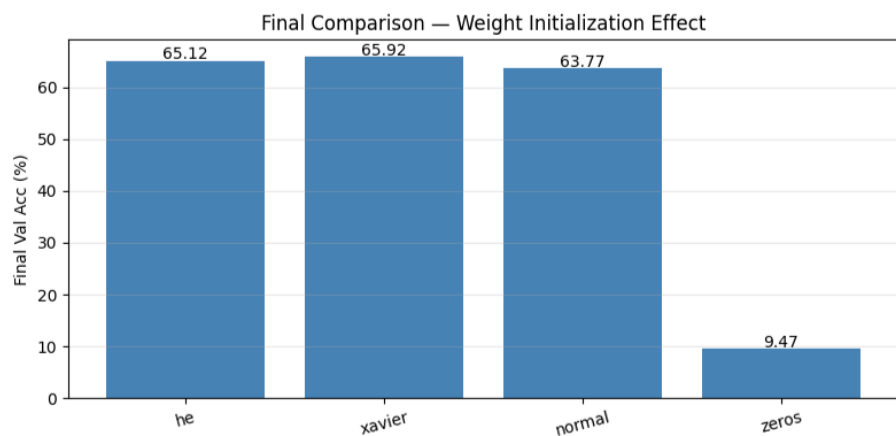
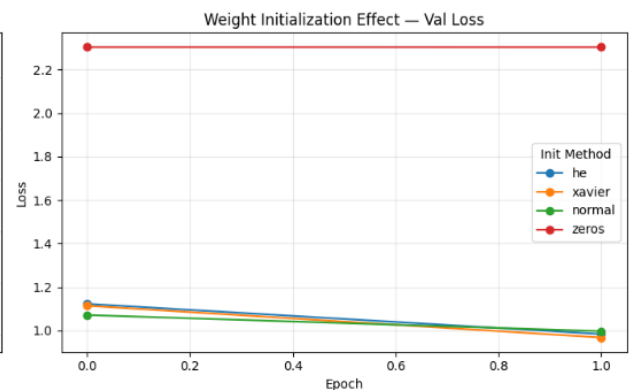
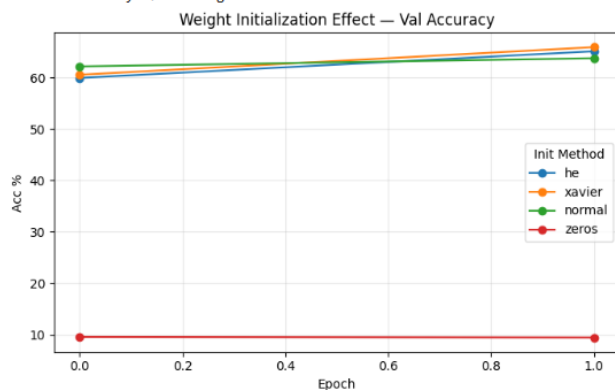


>>> Sensitivity 6/8 : L1 Regularization





>>> Sensitivity 8/8 : Weight Initialization



نتایج Hyper Sensitivity – CIFAR10

```
=====
val acc بر اساس بهترین مقدار های پیرامونها:
=====
Learning Rate      : lr=0.01
Batch Size         : bs=16
Activation          : ReLU
Optimizer           : RMSprop
L2 weight_decay    : L2=0.0001
Dropout             : p=0.4
Init Method        : xavier
=====
```

بخش سوم:

الف) مسئله‌ای که Residual Connection حل می‌کند

با عمیق‌تر شدن شبکه‌های CNN دو پدیده مخرب ظاهر می‌شوند:

1- محو شدن گرادیان (Vanishing Gradient): در پس‌انتشار گرادیان لایه ℓ به صورت ضرب لایه‌های بعدی محاسبه می‌شود:

$$\frac{\partial L}{\partial x_\ell} = \frac{\partial L}{\partial x_L} \prod_{i=\ell}^{L-1} \frac{\partial f_{i+1}}{\partial x_i}$$

اگر هر ضریب کوچک‌تر از ۱ باشد حاصل ضرب با طول شبکه به صورت نمایی به صفر میل می‌کند و لایه‌های اولیه عملاً آموزش نمی‌بینند.

2- Degradation Problem: حتی با وجود BatchNorm که گرادیان را زنده نگه می‌دارد امکان دارد یک شبکه ۵۶ لایه عملکرد بدتری از ۲۰ لایه دارد نه به دلیل overfitting ولی چون بهینه‌سازی شناسایی نگاشت همانی $H(x)=x$ توسط چند لایه غیرخطی سخت است. اتصال باقیمانده:

$$y = F(x; W) + x$$

به جای اینکه از شبکه بخواهد $H(x)$ را یاد بگیرد، از او می‌خواهد باقیمانده یعنی $F(x)=H(x)-x$ را یاد بگیرد. اگر نگاشت بهینه نزدیک به همانی باشد، فقط کافی است اوزان F به صفر میل کنند کاری بسیار آسان‌تر از یادگیری یک نگاشت همانی غیرخطی. هم‌زمان، گرادیان نسبت به ورودی به صورت زیر باز می‌شود:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(I + \frac{\partial F}{\partial x} \right)$$

به طور خلاصه برای حل مشکل degradation در شبکه‌های CNN عمیق یعنی پدیده‌ای که در آن افزایش عمق شبکه باعث افزایش خطای آموزش می‌شد، در حالی که از نظر ظرفیت مشکلی نباید وجود می‌داشت.

عامل کمکی: نرم کردن محو گرادیان و آسان‌سازی بهینه‌سازی نگاشت همانی. ResNet با تبدیل هدف یادگیری از $H(x)$ به $F(x)=H(x)-x$ این مشکل را حل کرد.

ب) چه زمانی طول اتصال بلند بهتر است و چه زمانی کوتاه؟

کوتاه (۲-۳ لایه):

- 1- شبکه‌های بسیار عمیق :** مسیرهای کوتاه باعث می‌شوند گرادیان به صورت تقریباً بدون اعوجاج به لایه‌های پایین برسد. هر بلاک ۲ یا ۳ لایه است تا ترم باقیمانده F همچنان کوچک و قابل یادگیری بماند.
- 2- داده‌های با ویژگی‌های محلی غالب (تصاویر طبیعی، بافت‌ها):** تغییرات ظریف ویژگی بین لایه‌های نزدیک کافی است و نیازی به جابه‌جایی اطلاعات در فواصل بزرگ نیست.
- 3- زمانی که می‌خواهیم BatchNorm و فعال‌سازی‌های متعدد، مسیر سیگنال را به شدت تغییر ندهند:** هرچه طول کوتاه‌تر باشد خطر اینکه $F(x)$ سیگنال هویتی را خراب کند کمتر است.
- 4- در شبکه‌های بسیار عمیق، اتصال کوتاه باعث گرادیان‌سنجش‌پذیر و پایدار می‌شود و عملاً همان اصل اصلی ResNet است.**

بلند:

- 1- معماری‌های Encoder&Decoder:** اطلاعات با وضوح فضایی بالا که در encoder نزدیک ورودی‌اند و باید به decoder منتقل شوند تا decoder بتواند جزئیات مکانی (لبه‌ها، مرزها) را در خروجی بازسازی کند. اینجا یک پرش بلند مستقیماً ویژگی‌های سطح پایین را به انتهای شبکه می‌رساند.
- 2- وظایف وابسته به اطلاعات سطح پایین در عمق زیاد مانند image-to-image, super-resolution, segmentation, image translation و denoising autoencoderها:** پرش بلند جلوی از دست رفتن ریزجزئیات هنگام عبور از pooling/downsampling را می‌گیرد.
- 3- زمانی که خطر محو گرادیان در عمق بسیار زیاد همچنان وجود دارد، حتی با وجود پرش‌های کوتاه:** ایده DenseNet با اتصال هر لایه به همه لایه‌های بعدی، عملاً پرش‌های با طول‌های متنوع را به شبکه می‌افزاید و گرادیان را در همه مقیاس‌ها زنده نگه می‌دارد.
- 4- زمانی که حافظه بلندمدت ویژگی‌ها (feature reuse) مهم است.** پرش بلند معادل ایجاد یک بزرگراه اطلاعاتی در شبکه است.

دیتاست 5 Linnaeus (128×128) شامل ۵ کلاس متوازن berry, bird, dog, flower, other است. تقسیم نهایی پس از اجرای کد:

بخش	تعداد نمونه	درصد
Train	5,250	۶۵.۶٪
Validation	750	۹.۴٪
Test	2,000	۲۵٪

Augmentation فقط روی Train

1. Resize(128,128)
 2. RandomHorizontalFlip(p=0.5)
 3. RandomRotation(10°)
 4. ColorJitter (brightness=0.1, contrast=0.1, saturation=0.1)
 5. Normalize با میانگین و انحراف معیار, ImageNet: mean=[0.485,0.456,0.406],
std=[0.229,0.224,0.225]
- روی Val/Test تنها Resize و Normalize اعمال می شود تا ارزیابی بدون نویز انجام شود.

روابط ریاضی Forward و Backward :

لایه کانولوشن (Forward): برای ورودی $x \in \mathbb{R}^{C_{in} \times H \times W}$ و کرنل $W \in \mathbb{R}^{C_{out} \times C_{in} \times k \times k}$ با فرض

: bias = False

$$y_{o,i,j} = \sum_{c=1}^{C_{in}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W_{o,c,u,v} \cdot x_{c,i \cdot s + u - p, j \cdot s + v - p}$$

لایه کانولوشن (Backward):

$$\frac{\partial L}{\partial W_{o,c,u,v}} = \sum_{i,j} \frac{\partial L}{\partial y_{o,i,j}} \cdot x_{c,i \cdot s + u - p, j \cdot s + v - p}$$

$$\frac{\partial L}{\partial x_{c,a,b}} = \sum_{o,u,v} \frac{\partial L}{\partial y_{o,(a-u+p)/s, (b-v+p)/s}} \cdot W_{o,c,u,v}$$

لایه نرمال سازی بچ (BatchNorm):

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

و در پس انتشار (Backpropagation):

$$\frac{\partial L}{\partial \hat{x}_i} = \frac{\partial L}{\partial y_i} \cdot \gamma$$

$$\frac{\partial L}{\partial \sigma_B^2} = \sum_{i=1}^m \frac{\partial L}{\partial \hat{x}_i} \cdot (x_i - \mu_B) \cdot \left(-\frac{1}{2}\right) (\sigma_B^2 + \epsilon)^{-3/2}$$

$$\frac{\partial L}{\partial \mu_B} = \left(\sum_{i=1}^m \frac{\partial L}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_B^2 + \epsilon}} \right) + \frac{\partial L}{\partial \sigma_B^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_B)}{m}$$

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{\partial L}{\partial \sigma_B^2} \cdot \frac{2(x_i - \mu_B)}{m} + \frac{\partial L}{\partial \mu_B} \cdot \frac{1}{m}$$

تابع فعال ساز ReLU:

$$y = \max(0, x)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \mathbb{I}[x > 0]$$

لایه MaxPool: در عبور رو به جلو در هر پنجره مقدار ماکزیمم انتخاب می‌شود. در عبور رو به عقب گرادیان فقط به موقعیتی که در forward ماکزیمم بود منتقل شده و برای بقیه موقعیت‌ها صفر قرار داده می‌شود.

لایه Global Average Pooling (GAP):

$$y_c = \frac{1}{H \cdot W} \sum_{i,j} x_{c,i,j}$$

$$\frac{\partial L}{\partial x_{c,i,j}} = \frac{1}{H \cdot W} \cdot \frac{\partial L}{\partial y_c}$$

ترکیب Softmax + Cross-Entropy:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad L = - \sum_{k=1}^K t_k \log p_k$$

$$\frac{\partial L}{\partial z_k} = p_k - t_k$$

بلوک باقی‌مانده (Residual Block):

$$y = F(x; W) + S(x)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(\frac{\partial F}{\partial x} + \frac{\partial S}{\partial x} \right)$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(\frac{\partial F}{\partial x} + I \right)$$

بلوک انسپشن (Inception Block):

$$y = \text{Concat}[B_1(x), B_2(x), B_3(x), B_4(x)]$$

مقایسه سه معماری

1) BaseLine CNN:

این مدل شامل چهار بلوک پشت سرهم به صورت زیر است:

$$[\text{Conv}3 \times 3 \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}3 \times 3 \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}2]$$

تغییر عرض کانال ها به صورت $256 \rightarrow 128 \rightarrow 64 \rightarrow 32$ انجام شده و در نهایت لایه GAP و یک لایه خطی (Linear) قرار دارد:

$$128 \times 128 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow \text{GAP} \rightarrow \text{Dropout}(0.3) \rightarrow \text{FC}(256 \rightarrow 5)$$

تعداد پارامترهای آموزش پذیر: ۱,۱۷۴,۵۰۱

2) Residual CNN:

این مدل $\text{FC} + \text{GAP} + 4 \times \text{ResidualBlock} + \text{stem}$ با همان ابعاد مدل پایه است. هر بلوک Residual باقی مانده به صورت زیر است:

$$\text{Conv}3 \times 3(\text{stride} = s) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}3 \times 3 \rightarrow \text{BN} \rightarrow [+ \text{shortcut}] \rightarrow \text{ReLU}$$

لایه shortcut در حالت عدم تطبیق ابعاد به صورت یک لایه $\text{BN} \rightarrow \text{Conv}1 \times 1(\text{stride} = s)$ پیاده سازی شده است. در این ساختار طول هر اتصال کوتاه (skip) برابر با ۲ لایه است که سبک کوتاه و کلاسیک ResNet را شکل می دهد.

تعداد پارامترها: ۱,۲۲۸,۷۷۳

3 Inception-Residual CNN :

درون هر بلوک باقی مانده، کانولوشن های 3x3 با یک InceptionBlock کامل جایگزین شده اند، اما اتصال کوتاه (skip connection) همچنان حفظ می شود. تخصیص کانال ها در هر بلوک به صورت زیر فرمول بندی شده است:

$$c_1 = c_3 = c_5 = \lfloor C_{out}/4 \rfloor, \quad c_p = C_{out} - 3\lfloor C_{out}/4 \rfloor, \quad c_{3r} = c_{5r} = \max(C_{out}/8, 4)$$

این طرح به طور هم زمان ویژگی چند مقیاسی بودن لایه های Inception و مسیر مستقیم جریان گرادیان در ResNet را ترکیب می کند.

پارامترهای شبکه (بین سه شبکه کاملاً یکسان هستند)

پارامتر	مقدار
Lost Function	Cross Entropy
نرخ یادگیری اولیه (Initial Learning Rate)	0.001
num_epochs	20
کاهش وزن (Weight Decay -L2)	0.0001
Batch size	64
لایه Dropout قبل از FC	0.3
optimizer	Adam
random_seed	42

خروجی ها:

Training Sample Dataset



معماري Baseline CNN

```
BaselineCNN(  
  (b1): Sequential(  
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (4): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (5): ReLU(inplace=True)  
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (b2): Sequential(  
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (5): ReLU(inplace=True)  
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (b3): Sequential(  
    (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (5): ReLU(inplace=True)  
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (b4): Sequential(  
    (0): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
    (4): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (5): ReLU(inplace=True)  
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (gap): AdaptiveAvgPool2d(output_size=1)  
  (dropout): Dropout(p=0.3, inplace=False)  
  (fc): Linear(in features=256, out features=5, bias=True)
```

Train و نتایج Baseline CNN

\nTotal trainable params: 1,174,501

[Baseline]	Ep01/20	TL 1.279 TA 46.93%	VL 1.268 VA 48.53%	F1 46.56%	∇	10.250	lr 1.0e-03
[Baseline]	Ep02/20	TL 1.176 TA 51.75%	VL 1.437 VA 42.80%	F1 40.09%	∇	10.043	lr 1.0e-03
[Baseline]	Ep03/20	TL 1.100 TA 56.25%	VL 1.201 VA 55.33%	F1 53.26%	∇	7.882	lr 1.0e-03
[Baseline]	Ep04/20	TL 1.044 TA 58.78%	VL 1.226 VA 51.87%	F1 48.71%	∇	10.982	lr 1.0e-03
[Baseline]	Ep05/20	TL 1.016 TA 60.21%	VL 1.153 VA 59.33%	F1 59.18%	∇	18.247	lr 1.0e-03
[Baseline]	Ep06/20	TL 1.026 TA 59.66%	VL 0.980 VA 62.80%	F1 63.58%	∇	4.253	lr 1.0e-03
[Baseline]	Ep07/20	TL 0.940 TA 64.23%	VL 1.136 VA 56.93%	F1 55.34%	∇	10.227	lr 1.0e-03
[Baseline]	Ep08/20	TL 0.898 TA 65.75%	VL 1.170 VA 53.07%	F1 51.22%	∇	16.473	lr 1.0e-03
[Baseline]	Ep09/20	TL 0.891 TA 66.34%	VL 1.461 VA 53.07%	F1 51.65%	∇	9.380	lr 5.0e-04
[Baseline]	Ep10/20	TL 0.790 TA 70.32%	VL 1.012 VA 64.00%	F1 64.04%	∇	7.094	lr 5.0e-04
[Baseline]	Ep11/20	TL 0.763 TA 71.22%	VL 0.760 VA 70.67%	F1 70.97%	∇	6.533	lr 5.0e-04
[Baseline]	Ep12/20	TL 0.711 TA 73.14%	VL 0.767 VA 70.67%	F1 70.55%	∇	7.784	lr 5.0e-04
[Baseline]	Ep13/20	TL 0.709 TA 73.45%	VL 0.835 VA 70.80%	F1 70.12%	∇	11.080	lr 5.0e-04
[Baseline]	Ep14/20	TL 0.697 TA 73.77%	VL 0.727 VA 73.33%	F1 73.29%	∇	14.426	lr 5.0e-04
[Baseline]	Ep15/20	TL 0.680 TA 75.07%	VL 0.667 VA 75.73%	F1 76.17%	∇	18.206	lr 5.0e-04
[Baseline]	Ep16/20	TL 0.651 TA 75.37%	VL 0.877 VA 70.27%	F1 70.27%	∇	16.136	lr 5.0e-04
[Baseline]	Ep17/20	TL 0.628 TA 76.17%	VL 0.758 VA 74.00%	F1 74.26%	∇	14.750	lr 5.0e-04
[Baseline]	Ep18/20	TL 0.621 TA 77.73%	VL 0.761 VA 72.53%	F1 72.65%	∇	46.033	lr 2.5e-04
[Baseline]	Ep19/20	TL 0.617 TA 76.80%	VL 0.717 VA 74.00%	F1 74.08%	∇	17.116	lr 2.5e-04
[Baseline]	Ep20/20	TL 0.546 TA 80.36%	VL 0.603 VA 77.87%	F1 78.09%	∇	3.931	lr 2.5e-04
[Baseline]	Test Loss = 0.5649 Test Acc = 79.35% Test F1 (macro) = 79.41%						

\nClassification report:

	precision	recall	f1-score	support
berry	0.8276	0.9000	0.8623	400
bird	0.6763	0.7050	0.6903	400
dog	0.9042	0.7550	0.8229	400
flower	0.8643	0.8600	0.8622	400
other	0.7188	0.7475	0.7328	400
accuracy			0.7935	2000
macro avg	0.7982	0.7935	0.7941	2000
weighted avg	0.7982	0.7935	0.7941	2000

Train و نتائج Residual CNN

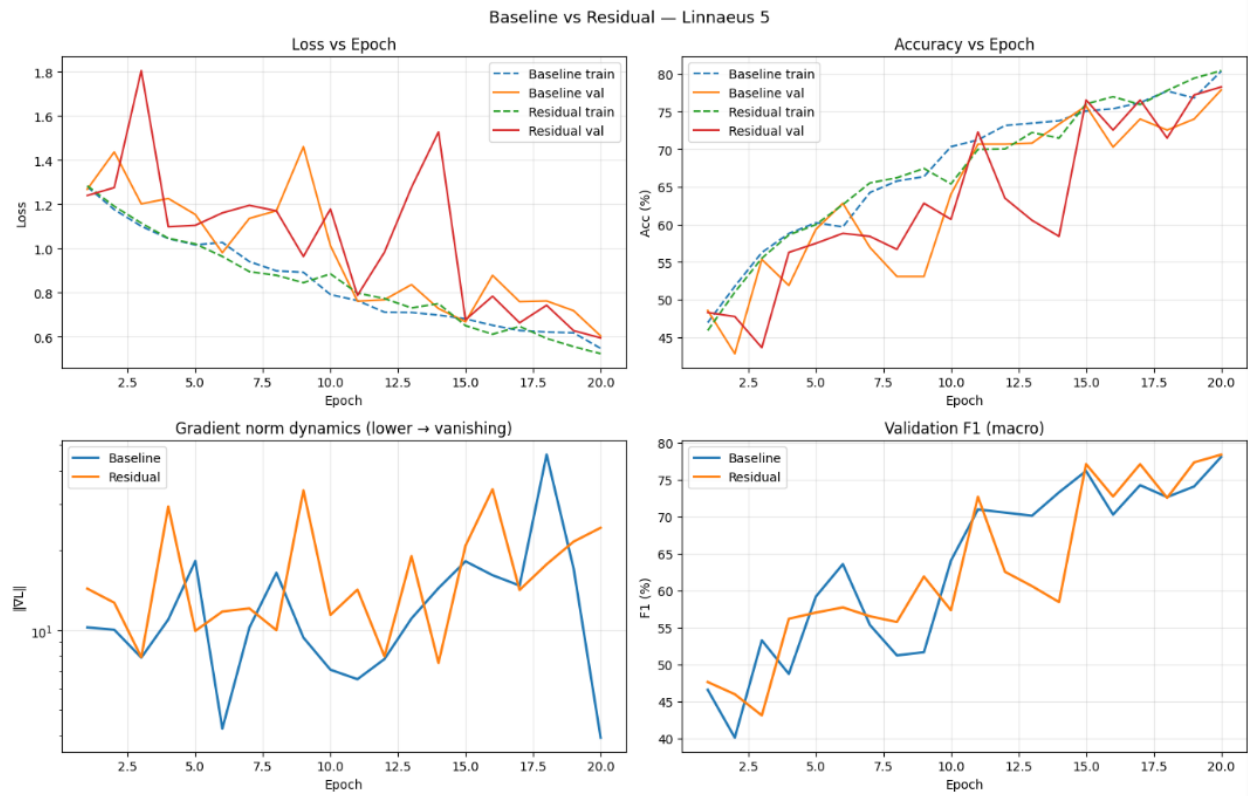
Residual params: 1,228,773

```
[Residual] Ep01/20 | TL 1.284 TA 45.89% | VL 1.239 VA 48.27% F1 47.61% | ||V|| 14.361 | lr 1.0e-03
[Residual] Ep02/20 | TL 1.190 TA 50.97% | VL 1.275 VA 47.73% F1 45.96% | ||V|| 12.705 | lr 1.0e-03
[Residual] Ep03/20 | TL 1.114 TA 55.49% | VL 1.805 VA 43.60% F1 43.10% | ||V|| 7.914 | lr 1.0e-03
[Residual] Ep04/20 | TL 1.045 TA 58.59% | VL 1.098 VA 56.27% F1 56.17% | ||V|| 29.299 | lr 1.0e-03
[Residual] Ep05/20 | TL 1.020 TA 59.96% | VL 1.104 VA 57.47% F1 57.01% | ||V|| 9.945 | lr 1.0e-03
[Residual] Ep06/20 | TL 0.963 TA 62.63% | VL 1.160 VA 58.80% F1 57.71% | ||V|| 11.764 | lr 1.0e-03
[Residual] Ep07/20 | TL 0.894 TA 65.50% | VL 1.195 VA 58.40% F1 56.51% | ||V|| 12.093 | lr 1.0e-03
[Residual] Ep08/20 | TL 0.878 TA 66.19% | VL 1.170 VA 56.67% F1 55.74% | ||V|| 10.002 | lr 1.0e-03
[Residual] Ep09/20 | TL 0.844 TA 67.43% | VL 0.963 VA 62.80% F1 61.89% | ||V|| 33.738 | lr 1.0e-03
[Residual] Ep10/20 | TL 0.884 TA 65.37% | VL 1.178 VA 60.67% F1 57.33% | ||V|| 11.419 | lr 1.0e-03
[Residual] Ep11/20 | TL 0.797 TA 69.96% | VL 0.786 VA 72.27% F1 72.68% | ||V|| 14.239 | lr 1.0e-03
[Residual] Ep12/20 | TL 0.772 TA 70.02% | VL 0.982 VA 63.47% F1 62.53% | ||V|| 7.964 | lr 1.0e-03
[Residual] Ep13/20 | TL 0.730 TA 72.21% | VL 1.276 VA 60.53% F1 60.59% | ||V|| 19.055 | lr 1.0e-03
[Residual] Ep14/20 | TL 0.749 TA 71.47% | VL 1.527 VA 58.40% F1 58.43% | ||V|| 7.529 | lr 5.0e-04
[Residual] Ep15/20 | TL 0.649 TA 76.00% | VL 0.676 VA 76.53% F1 77.11% | ||V|| 20.860 | lr 5.0e-04
[Residual] Ep16/20 | TL 0.611 TA 76.97% | VL 0.783 VA 72.53% F1 72.72% | ||V|| 34.056 | lr 5.0e-04
[Residual] Ep17/20 | TL 0.646 TA 75.92% | VL 0.662 VA 76.53% F1 77.10% | ||V|| 14.192 | lr 5.0e-04
[Residual] Ep18/20 | TL 0.592 TA 77.79% | VL 0.742 VA 71.47% F1 72.54% | ||V|| 17.741 | lr 2.5e-04
[Residual] Ep19/20 | TL 0.554 TA 79.43% | VL 0.627 VA 77.20% F1 77.34% | ||V|| 21.610 | lr 2.5e-04
[Residual] Ep20/20 | TL 0.522 TA 80.44% | VL 0.594 VA 78.27% F1 78.39% | ||V|| 24.376 | lr 2.5e-04
\n[Residual] Test Loss = 0.5729 | Test Acc = 80.00% | Test F1 (macro) = 79.97%
```

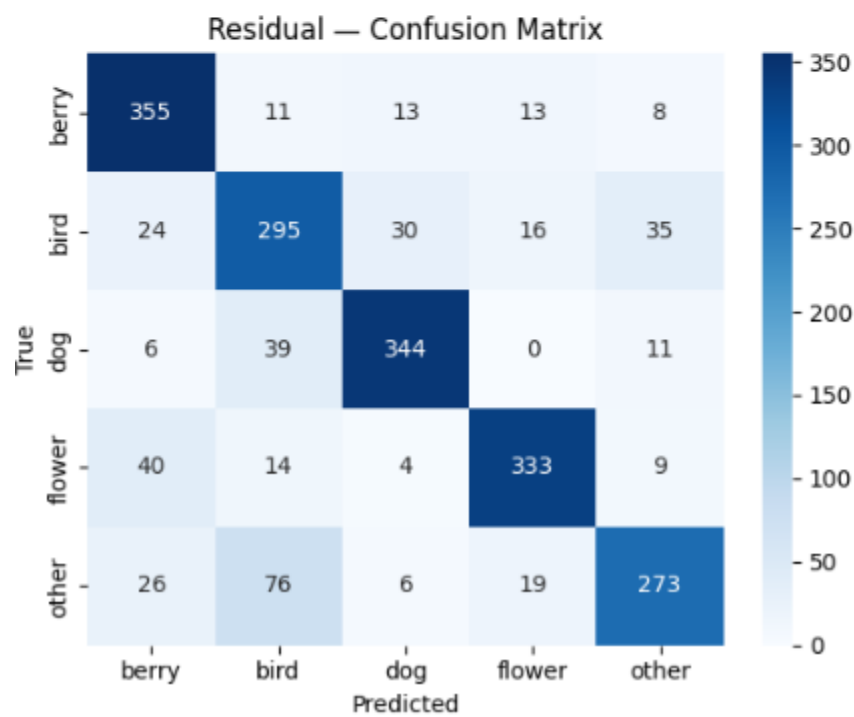
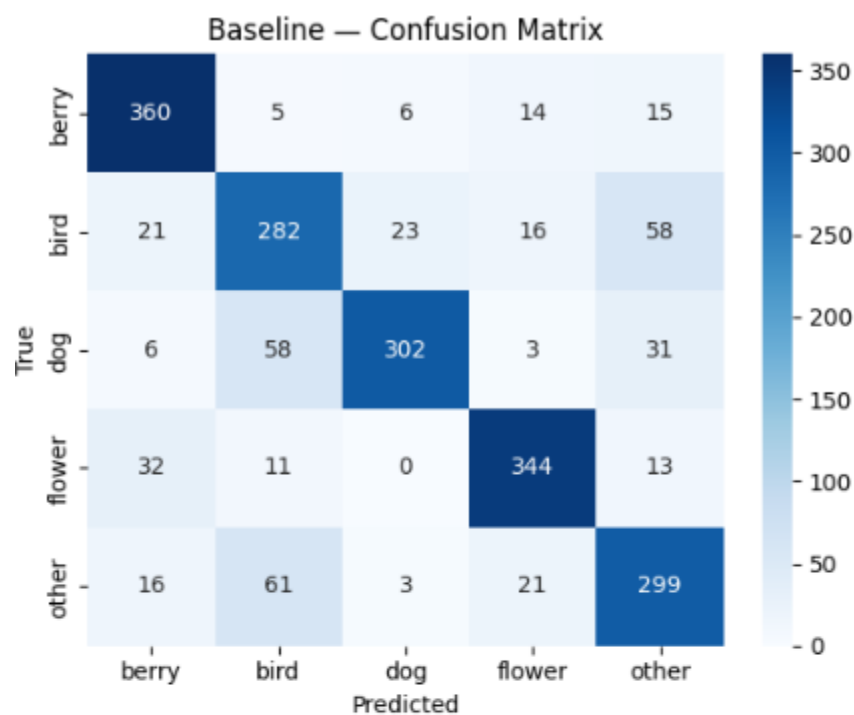
\nClassification report:

	precision	recall	f1-score	support
berry	0.7871	0.8875	0.8343	400
bird	0.6782	0.7375	0.7066	400
dog	0.8665	0.8600	0.8632	400
flower	0.8740	0.8325	0.8528	400
other	0.8125	0.6825	0.7418	400
accuracy			0.8000	2000
macro avg	0.8037	0.8000	0.7997	2000
weighted avg	0.8037	0.8000	0.7997	2000

مقایسه Baseline and Residual

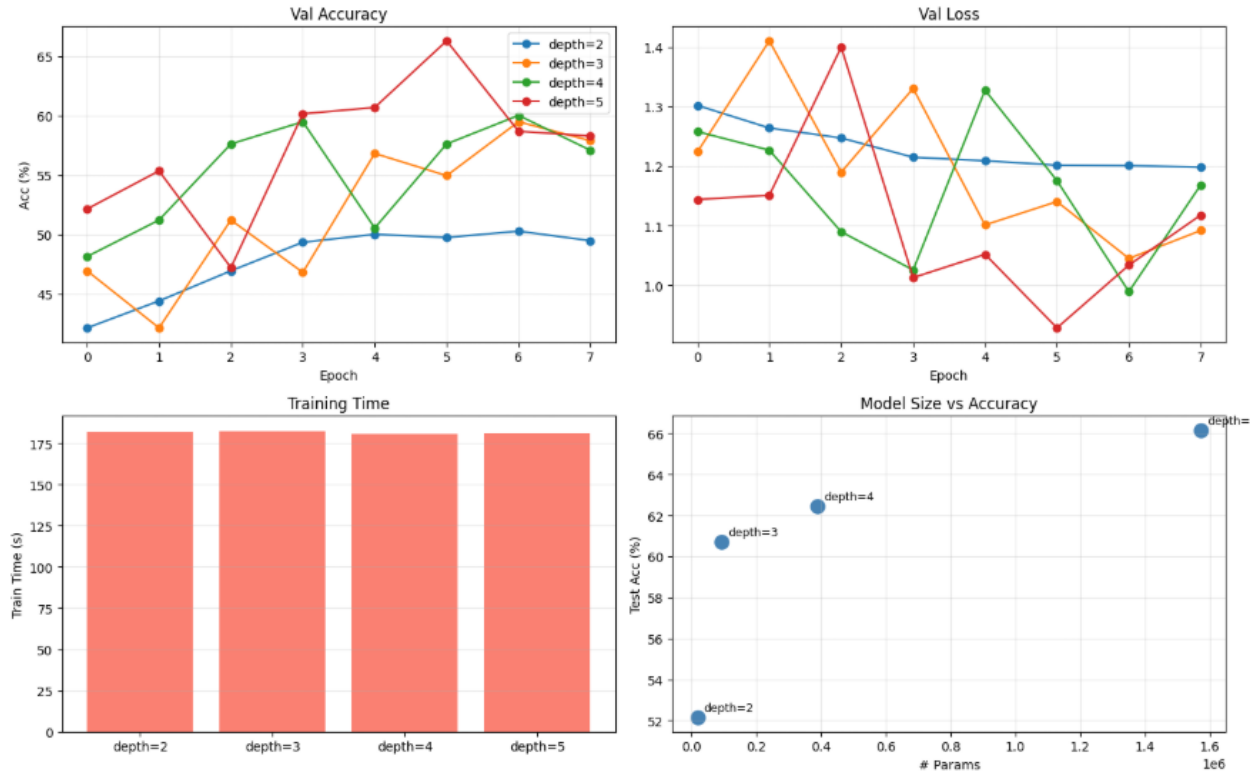


ماتریس های BaseLine و Residual

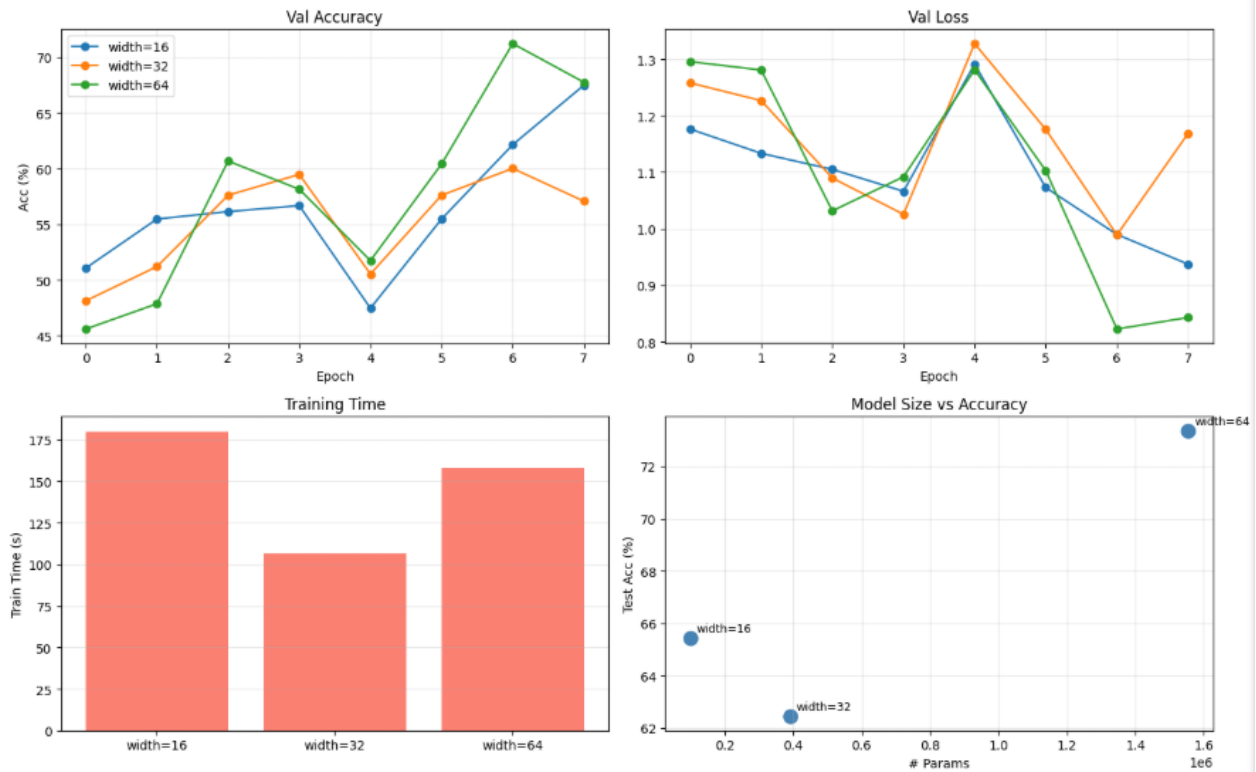


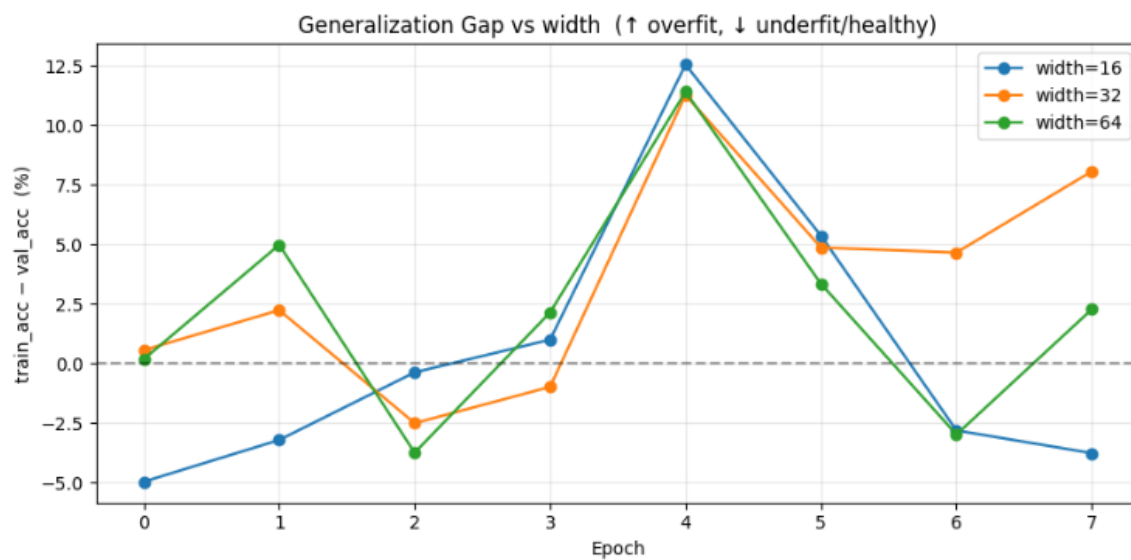
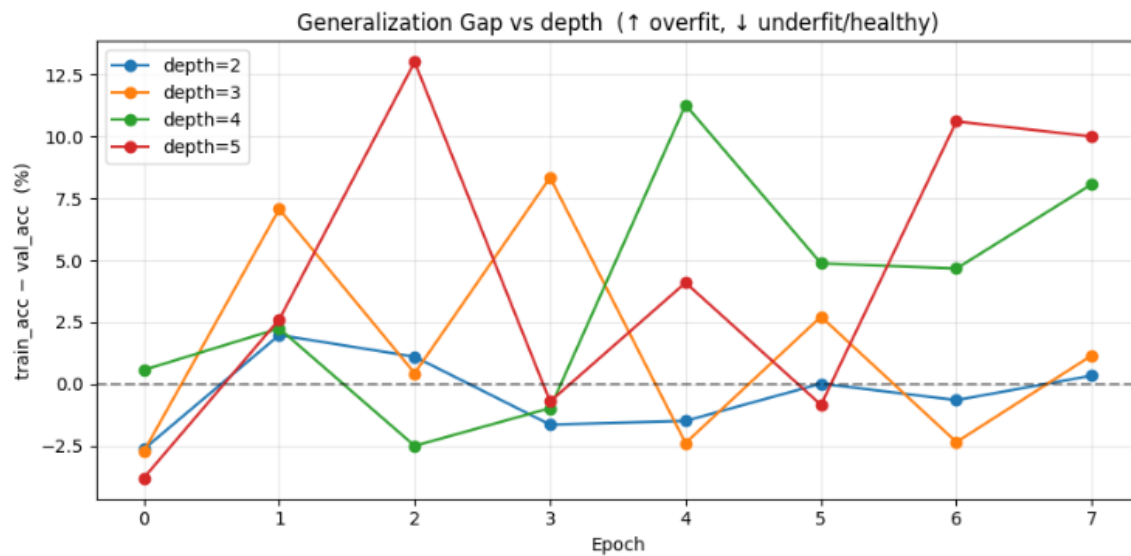
Effect Depths

Effect of Network Depth on Linnaeus 5



Effect of Network Width on Linnaeus 5





Train & Result Inception- Residual

Inception-Res params: 206,757

[Sanity] forward OK, output shape = (2, 5)

[InceptionRes]	Ep01/20	TL 1.269	TA 47.16%	VL 1.089	VA 57.07%	F1 57.17%	v 10.728	lr 1.0e-03
[InceptionRes]	Ep02/20	TL 1.073	TA 57.45%	VL 0.964	VA 62.80%	F1 62.46%	v 7.039	lr 1.0e-03
[InceptionRes]	Ep03/20	TL 0.991	TA 60.53%	VL 0.891	VA 64.93%	F1 64.81%	v 8.941	lr 1.0e-03
[InceptionRes]	Ep04/20	TL 0.919	TA 64.21%	VL 0.842	VA 68.53%	F1 68.37%	v 15.093	lr 1.0e-03
[InceptionRes]	Ep05/20	TL 0.872	TA 66.88%	VL 0.877	VA 66.00%	F1 67.10%	v 10.994	lr 1.0e-03
[InceptionRes]	Ep06/20	TL 0.824	TA 68.21%	VL 0.833	VA 67.60%	F1 68.37%	v 17.636	lr 1.0e-03
[InceptionRes]	Ep07/20	TL 0.789	TA 68.97%	VL 0.722	VA 73.60%	F1 73.94%	v 11.116	lr 1.0e-03
[InceptionRes]	Ep08/20	TL 0.777	TA 70.21%	VL 0.699	VA 76.13%	F1 76.06%	v 12.963	lr 1.0e-03
[InceptionRes]	Ep09/20	TL 0.725	TA 72.80%	VL 0.725	VA 71.87%	F1 72.37%	v 12.842	lr 1.0e-03
[InceptionRes]	Ep10/20	TL 0.702	TA 73.89%	VL 0.684	VA 75.20%	F1 75.54%	v 10.411	lr 1.0e-03
[InceptionRes]	Ep11/20	TL 0.684	TA 74.61%	VL 0.691	VA 76.53%	F1 76.58%	v 10.941	lr 1.0e-03
[InceptionRes]	Ep12/20	TL 0.652	TA 75.30%	VL 0.655	VA 78.53%	F1 78.46%	v 9.973	lr 1.0e-03
[InceptionRes]	Ep13/20	TL 0.637	TA 75.73%	VL 0.609	VA 79.20%	F1 79.22%	v 2.866	lr 1.0e-03
[InceptionRes]	Ep14/20	TL 0.592	TA 78.04%	VL 0.641	VA 77.73%	F1 77.89%	v 5.463	lr 1.0e-03
[InceptionRes]	Ep15/20	TL 0.581	TA 78.34%	VL 0.640	VA 79.33%	F1 79.44%	v 12.500	lr 1.0e-03
[InceptionRes]	Ep16/20	TL 0.585	TA 78.40%	VL 0.598	VA 80.67%	F1 80.83%	v 5.024	lr 1.0e-03
[InceptionRes]	Ep17/20	TL 0.549	TA 79.75%	VL 0.579	VA 81.20%	F1 81.27%	v 10.393	lr 1.0e-03
[InceptionRes]	Ep18/20	TL 0.543	TA 79.98%	VL 0.646	VA 78.13%	F1 78.57%	v 7.698	lr 1.0e-03
[InceptionRes]	Ep19/20	TL 0.514	TA 81.45%	VL 0.648	VA 79.47%	F1 79.68%	v 11.151	lr 1.0e-03
[InceptionRes]	Ep20/20	TL 0.506	TA 80.74%	VL 0.561	VA 80.13%	F1 80.14%	v 11.558	lr 5.0e-04

\n[InceptionRes] Test Loss = 0.5602 | Test Acc = 79.50% | Test F1 (macro) = 79.41%

\nClassification report:

	precision	recall	f1-score	support
berry	0.8028	0.8650	0.8327	400
bird	0.7368	0.7000	0.7179	400
dog	0.7800	0.8950	0.8335	400
flower	0.8935	0.7550	0.8184	400
other	0.7755	0.7600	0.7677	400
accuracy			0.7950	2000
macro avg	0.7977	0.7950	0.7941	2000
weighted avg	0.7977	0.7950	0.7941	2000

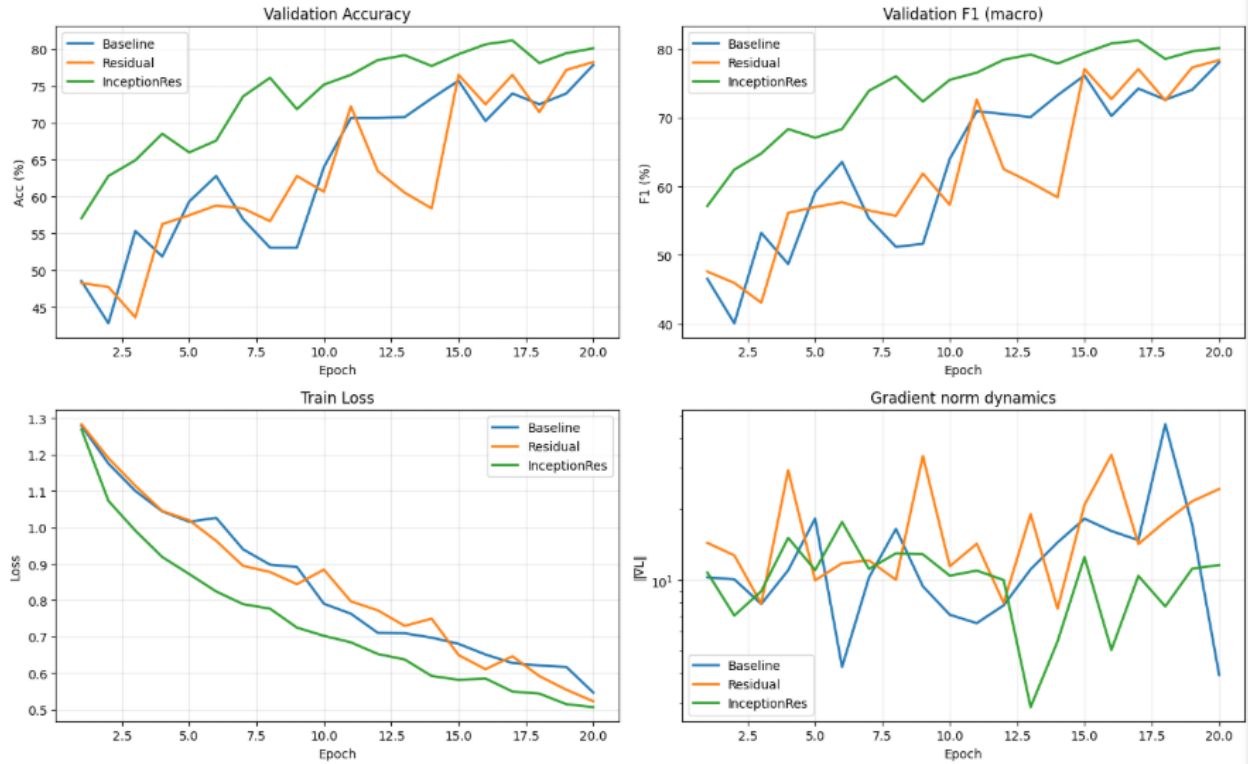
\n===== FINAL TEST METRICS =====

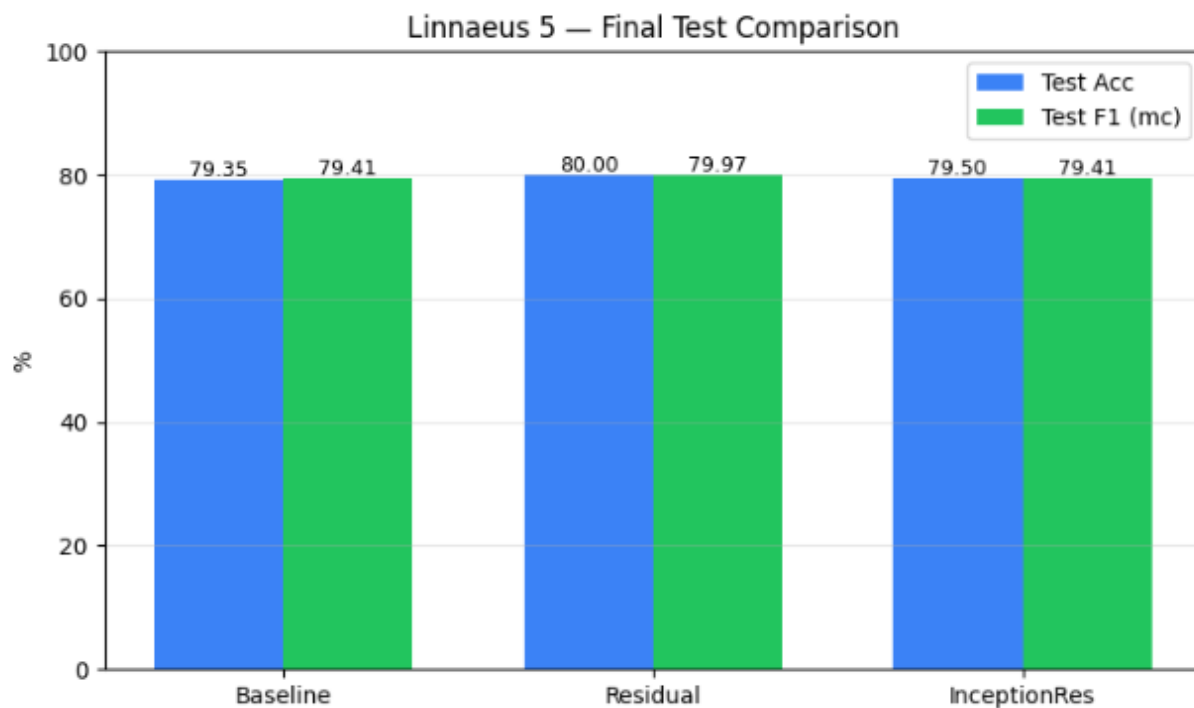
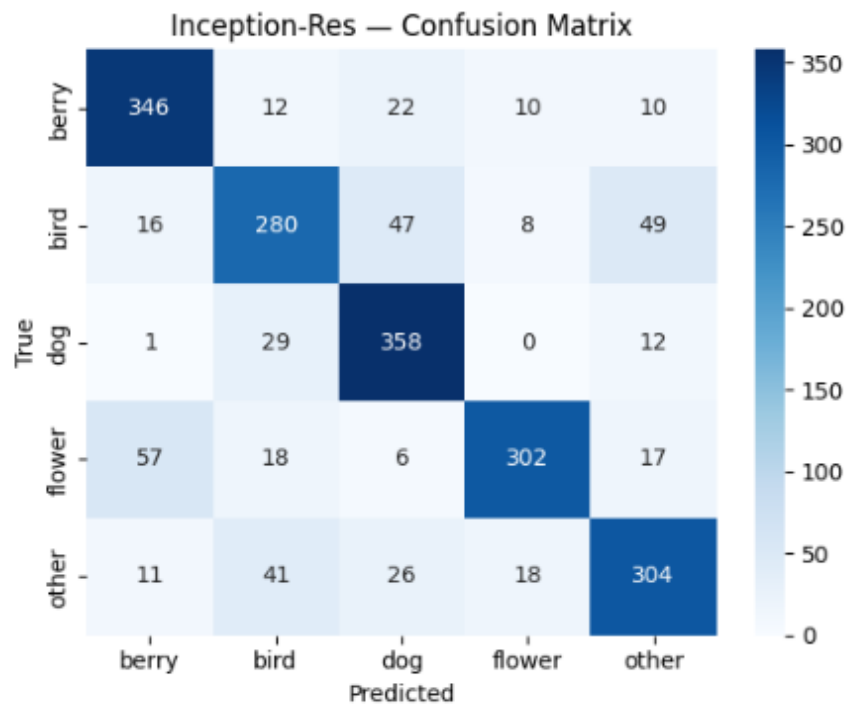
Model	Test Acc	Test F1	Test Prec	Test Rec
Baseline CNN	79.350	79.410	79.822	79.350
Residual CNN	80.000	79.975	80.366	80.000
Inception-Res CNN	79.500	79.406	79.772	79.500

مقایسه سه مدل

```
\n===== FINAL TEST METRICS =====\n      Model  Test Acc  Test F1  Test Prec  Test Rec\nBaseline CNN    79.350    79.410    79.822    79.350\nResidual CNN    80.000    79.975    80.366    80.000\nInception-Res CNN 79.500    79.406    79.772    79.500
```

Three-way comparison on Linnaeus 5





بخش چهارم : Transfer Learning

در این بخش هدف پیاده‌سازی یادگیری انتقالی (Transfer Learning) بر روی مدل پیش‌آموزته‌ی به‌دست‌آمده در بخش سوم قسمت ب (شبکه‌ی Inception-ResNet آموزش‌دیده روی دیتاست Linnaeus 5) برای دسته‌بندی مجموعه‌ی تصاویر flower_photos است

الف) چرا وزن‌های استخراج‌کننده‌ی ویژگی در گام اول باید منجمد باشند؟

1- حفاظت از وزن‌های پیش‌آموزته در برابر گرادیان‌های مخرب اولیه :

دسته‌بند جدید با وزن‌های تصادفی (مثلاً Xavier یا Kaiming) مقداردهی اولیه می‌شود و در ابتدای کار خطای آن بسیار بزرگ است. اگر در همان ابتدا کل شبکه را به‌روزرسانی کنیم، گرادیان‌های بزرگ تولیدشده توسط این دسته‌بند آموزش‌نندیده، طی الگوریتم پس‌انتشار به لایه‌های ابتدایی منتقل می‌شوند و وزن‌های ارزشمند پیش‌آموزته را مخدوش (catastrophic forgetting) می‌کنند.

$$\Delta W_{\text{backbone}} = -\eta \cdot \frac{\partial L}{\partial W_{\text{backbone}}}$$

در ابتدای آموزش به دلیل خطای زیاد دسته‌بند بسیار بزرگ است و یک گام به‌روزرسانی می‌تواند ساختار ویژگی‌های آموزخته‌شده را تخریب کند.

2- ایجاد یک خط‌مبنا (Baseline) و گرم‌کردن دسته‌بند:

با منجمد کردن backbone شبکه به‌صورت یک استخراج‌کننده‌ی ویژگی ثابت عمل می‌کند و دسته‌بند فرصت می‌یابد ابتدا یاد بگیرد چگونه فضای ویژگی موجود را به کلاس‌های جدید نگاشت کند. این کار یک baseline قابل‌اعتماد فراهم می‌کند که در فاز بعدی بتوان بهبود آن را سنجید.

3- کاهش هزینه‌ی محاسباتی و خطر بیش‌برازش:

در فاز اول تنها حدود ۳۳,۷۹۷ پارامتر آموزش می‌بیند (به جای کل ۲۰۷,۳۸۱ پارامتر). این کاهش چشمگیر پارامترهای آموزش‌پذیر:

1- مصرف حافظه و زمان آموزش را به شدت پایین می‌آورد

2- ریسک بیش‌برازش روی دیتاست هدف (که اغلب کوچک‌تر از دیتاست مبدا است) را کم می‌کند.

4- ثابت نگه داشتن آماره‌های Batch Normalization:

در backbone به حالت eval تنظیم شدند تا آماره‌های running mean و running variance که در حین پیش‌آموزش روی Linnaeus 5 یاد گرفته شده‌اند، با آماره‌های دیتاست flower بازنویسی نشوند. این کار از یک منبع پنهان نشأت آماری جلوگیری می‌کند.

ب) پیاده‌سازی یادگیری انتقالی و تاثیر پیچیدگی دسته‌بند

مدل پایه Inception-ResNet:

مدل پایه یک شبکه‌ی Inception-Residual CNN است که قبلاً روی دیتاست Linnaeus 5 آموزش دیده است. اجزای آن:

بخش	جزئیات
Stem	<code>Conv2d(3 → 32, kernel=3×3) → BatchNorm → ReLU</code>
Layer 1	InceptionResBlock با ۳۲ کانال (وضوح $۱۲۸ \times ۱۲۸ \rightarrow ۶۴ \times ۶۴$)
Layer 2	InceptionResBlock با ۶۴ کانال ($۶۴ \rightarrow ۳۲$)
Layer 3	InceptionResBlock با ۱۲۸ کانال ($۳۲ \rightarrow ۱۶$)
Layer 4	InceptionResBlock با ۲۵۶ کانال ($۱۶ \rightarrow ۸$)
GAP	Global Average Pooling
Classifier	معماری جدید جایگزین می‌شود

ساختار بلوک Inception-Residual

هر InceptionBlock شامل چهار شاخه‌ی موازی است:

- شاخه ۱: کانولوشن ۱×۱
- شاخه ۲: $۳ \times ۳ \rightarrow ۱ \times ۱$
- شاخه ۳: $۵ \times ۵ \rightarrow ۱ \times ۱$
- شاخه ۴: MaxPool $۳ \times ۳ \rightarrow ۱ \times ۱$

خروجی این چهار شاخه با عملگر `torch.cat` در بعد کانال پیوسته می‌شوند و سپس یک اتصال میان‌بر (Residual Shortcut) ورودی بلوک را به این خروجی اضافه می‌کند. در صورت عدم تطابق ابعاد Shortcut از یک کانولوشن ۱×۱ + BatchNorm برای هم‌تراز کردن استفاده می‌نماید.

: Classifier Head

(5,128)Flatten → Linear(256, 128) → BatchNorm1d(128) → ReLU → Dropout(p=0.4) → Linear

یک لایه ی پنهان ۲۸ انورونی: ایجاد یک ظرفیت غیرخطی میانی برای نگاشت ویژگی های ۲۵۶ بعدی خروجی GAP به فضای کلاس ها.

BatchNorm1d: پایدارسازی توزیع فعال سازی ها در حین آموزش و تسریع همگرایی.

Dropout با نرخ ۰.۴: منظم سازی قوی برای مقابله با بیش برارزش، که در دیتاست های کوچک (۳۶۷۰ تصویر) اهمیت ویژه دارد.

دیتاست flower_photos شامل ۳۶۷۰ تصویر در ۵ کلاس است:

تعداد تصاویر	کلاس
۶۳۳	daisy
۸۹۸	dandelion
۶۴۱	roses
۶۹۹	sunflowers
۷۹۹	tulips

تقسیم بندی به نسبت ۲۰/۱۰/۷۰ انجام شد:

- آموزش: ۲۵۶۹ تصویر
- اعتبارسنجی: ۳۶۷ تصویر
- آزمون: ۷۳۴ تصویر

: Hyper Parameter

پارامتر	مقدار
Optimizer	Adam
Learning Rate	0.001
Weight Decay	0.0001
Batch Size	۶۴
Epochs	۱۰
Loss	CrossEntropyLoss
Scheduler	(factor=0.5, patience=2)

نتایج پایان فاز ۱:

معيار	آموزش	اعتبارسنجی	آزمون
Accuracy	۶۹,۵۲%	۷۰,۳۰%	۷۲,۲۱%
Loss	۰,۷۹۵	۰,۸۰۸	۰,۷۷۱

فاز ۲ (Fine-tuning تدریجی):

Epochs	پارامترهای آموزش پذیر	لایه های آزاد شده	Stage
۴	۱۶۲,۴۳۷	layer4	A
۴	۱۹۵,۰۱۳	layer4 + layer3	B
۴	۲۰۳,۳۶۵	layer4 + layer3 + layer2	C
۴	۲۰۷,۳۸۱	کل شبکه (stem + layer1...layer4)	D

نتایج پایانی روی مجموعه ی آزمون پس از Fine-tuning:

بهبود	فاز ۲	فاز ۱	معیار
۳,۶۸%+	۷۵,۸۹%	۷۲,۲۱%	Test Accuracy
۳,۳۶%+	۷۵,۱۰%	۷۱,۷۴%	Test F1-macro
۳,۵۰%+	۷۵,۵۷%	۷۲,۰۷%	Test Precision
۳,۰۴%+	۷۵,۲۲%	۷۲,۱۸%	Test Recall
۰,۰۸۰-	۰,۶۹۱	۰,۷۷۱	Test Loss

گزارش طبقه‌بندی به تفکیک کلاس (Phase 2)

کلاس	Precision	Recall	F1-score	Support
daisy	۰,۷۰۲۱	۰,۸۰۴۹	۰,۷۵۰۰	۱۲۳
dandelion	۰,۷۴۸۸	۰,۸۶۸۶	۰,۸۰۴۲	۱۷۵
roses	۰,۶۶۹۸	۰,۶۱۲۱	۰,۶۳۹۶	۱۱۶
sunflowers	۰,۸۵۹۳	۰,۷۸۳۸	۰,۸۱۹۸	۱۴۸
tulips	۰,۷۹۸۷	۰,۶۹۱۹	۰,۷۴۱۴	۱۷۲

بهترین کلاس: Sunflowers (بالاترین Precision = ۸۵,۹۳٪) و Dandelion (بالاترین Recall = ۸۶,۸۶٪).

تأثیر بر فاز انجماد: هر چه دسته‌بند پیچیده‌تر باشد، ظرفیت بیشتری برای استخراج روابط غیرخطی از ویژگی‌های ثابت backbone دارد. در این پروژه، دولایه‌ای توانست بدون Fine-tuning به دقت ۷۲,۲۱٪ برسد اگر صرفاً یک Linear ساده می‌بود احتمالاً نتیجه به مراتب پایین‌تر می‌شد.

تأثیر بر فاز Fine-tuning: اگر دسته‌بند بیش از حد پیچیده باشد و تعداد پارامترهای آن به نسبت داده‌ها زیاد باشد:

1- ریسک بیش‌برازش روی دیتاست هدف افزایش می‌یابد

2- در گام آزادسازی backbone، گرادین‌های اشباع‌شده یا نوسانی به سمت لایه‌های عمیق منتقل می‌شود.

اگر دسته‌بند بسیار ساده باشد (تک Linear) ظرفیت کافی برای نگاشت ویژگی‌های سطح بالا به کلاس‌های جدید را ندارد و Fine-tuning مجبور است این کمبود ظرفیت را جبران کند.

روابط پیشرو و پسرو (Forward & Backward Propagation)

لایه ی کانولوشن ۲ بعدی

:Forward

$$Y_{c_o, i, j} = b_{c_o} + \sum_{c=1}^{c_{in}} \sum_{u=1}^{k_h} \sum_{v=1}^{k_w} K_{c_o, c, u, v} \cdot X_{c, i+u-1, j+v-1}$$

:Backward

$$\frac{\partial L}{\partial K_{c_o, c, u, v}} = \sum_{i, j} \delta_{c_o, i, j} (Y) \cdot X_{c, i+u-1, j+v-1}$$

$$\frac{\partial L}{\partial b_{c_o}} = \sum_{i, j} \delta_{c_o, i, j} (Y)$$

$$\frac{\partial L}{\partial X_{c, m, n}} = \sum_{c_o=1}^{c_{out}} \sum_{u, v} K_{c_o, c, u, v} \cdot \delta_{c_o, m-u+1, n-v+1} (Y)$$

لایه Batch Normalization

:Forward

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

:Backward

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^m \delta_i \hat{x}_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^m \delta_i, \quad \frac{\partial L}{\partial \hat{x}_i} = \delta_i \cdot \gamma$$

$$\frac{\partial L}{\partial \sigma_B^2} = \sum_{i=1}^m \frac{\partial L}{\partial \hat{x}_i} \cdot (x_i - \mu_B) \cdot \left(-\frac{1}{2}\right) (\sigma_B^2 + \epsilon)^{-3/2}$$

$$\frac{\partial L}{\partial \mu_B} = \left(\sum_{i=1}^m \frac{\partial L}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_B^2 + \epsilon}} \right) + \frac{\partial L}{\partial \sigma_B^2} \cdot \frac{\sum_{i=1}^m (x_i - \mu_B)}{m}$$

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{\partial L}{\partial \sigma_B^2} \cdot \frac{2(x_i - \mu_B)}{m} + \frac{\partial L}{\partial \mu_B} \cdot \frac{1}{m}$$

$$y_i = \gamma \cdot \frac{x_i - \mu_{\text{running}}}{\sqrt{\sigma_{\text{running}}^2 + \epsilon}} + \beta$$

تابع فعالساز :

:Forward

$$y_i = \max(0, x_i)$$

:Backward

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y_i} \cdot \mathbb{I}[x_i > 0]$$

:Max Pooling

Forward

$$y_{i,j} = \max_{(u,v) \in R_{i,j}} x_{u,v}$$

Backward

$$\frac{\partial L}{\partial x_{u,v}} = \begin{cases} \frac{\partial L}{\partial y_{i,j}} & \text{if } (u,v) = \operatorname{argmax}_{R_{i,j}} x \\ 0 & \text{otherwise} \end{cases}$$

:GAP

:Forward

$$y_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W X_{c,i,j}$$

:Backward

$$\frac{\partial L}{\partial X_{c,i,j}} = \frac{1}{H \cdot W} \cdot \frac{\partial L}{\partial y_c}$$

: Residual Shortcut

:Forward

$$y = \text{ReLU}(F(x) + s(x))$$

:Backward

$$\frac{\partial L}{\partial F(x)} = \delta(z), \quad \frac{\partial L}{\partial s(x)} = \delta(z)$$
$$\frac{\partial L}{\partial x} = \frac{\partial F(x)}{\partial x} \cdot \delta(z) + \frac{\partial s(x)}{\partial x} \cdot \delta(z)$$

: Inception Concat

:Forward

$$y = \text{Concat}[b_1, b_2, b_3, b_4]$$

:Backward

$$\frac{\partial L}{\partial b_k} = \frac{\partial L}{\partial y}$$

: Fully Connected

:Forward

$$y = Wx + b$$

:Backward

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot x^T, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial y}, \quad \frac{\partial L}{\partial x} = W^T \cdot \frac{\partial L}{\partial y}$$

: Dropout

:Forward

$$y_i = \frac{1}{1-p} \cdot m_i \cdot x_i$$

:Backward

$$\frac{\partial L}{\partial x_i} = \frac{\partial L}{\partial y_i} \cdot \frac{1}{1-p} \cdot m_i$$

:SoftMax + CrossEntropy

:Forward

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad L = - \sum_{k=1}^K t_k \log(p_k)$$

:Backward

$$\frac{\partial L}{\partial z_k} = p_k - t_k$$

:ADAM

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

:Bias Correction

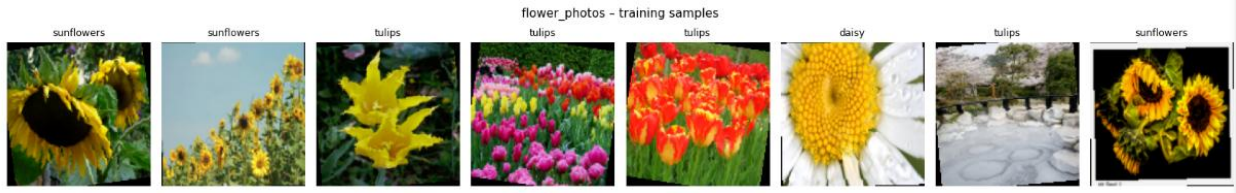
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

به روزرسانی نهایی پارامترها:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \cdot \hat{m}_t$$

خروجی ها:

Training Sample Dataset



Train Phase 1 - Freeze

```
[Phase1-Freeze] Ep01/10 | TL 1.192 TA 53.76% | VL 1.040 VA 59.40% F1 58.76% | ||∇|| 1.702 | lr 1.0e-03
[Phase1-Freeze] Ep02/10 | TL 0.998 TA 62.24% | VL 0.970 VA 61.58% F1 60.79% | ||∇|| 1.452 | lr 1.0e-03
[Phase1-Freeze] Ep03/10 | TL 0.932 TA 64.07% | VL 0.905 VA 67.03% F1 65.20% | ||∇|| 1.526 | lr 1.0e-03
[Phase1-Freeze] Ep04/10 | TL 0.877 TA 66.95% | VL 0.885 VA 68.12% F1 66.92% | ||∇|| 1.576 | lr 1.0e-03
[Phase1-Freeze] Ep05/10 | TL 0.851 TA 68.70% | VL 0.857 VA 68.12% F1 66.88% | ||∇|| 1.566 | lr 1.0e-03
[Phase1-Freeze] Ep06/10 | TL 0.820 TA 68.43% | VL 0.870 VA 66.76% F1 65.79% | ||∇|| 1.576 | lr 1.0e-03
[Phase1-Freeze] Ep07/10 | TL 0.809 TA 69.64% | VL 0.846 VA 69.48% F1 68.42% | ||∇|| 1.641 | lr 1.0e-03
[Phase1-Freeze] Ep08/10 | TL 0.800 TA 69.72% | VL 0.859 VA 66.76% F1 66.01% | ||∇|| 1.765 | lr 1.0e-03
[Phase1-Freeze] Ep09/10 | TL 0.799 TA 69.56% | VL 0.826 VA 70.03% F1 68.38% | ||∇|| 1.690 | lr 1.0e-03
[Phase1-Freeze] Ep10/10 | TL 0.795 TA 69.52% | VL 0.808 VA 70.30% F1 69.24% | ||∇|| 1.672 | lr 1.0e-03
\n[After Phase 1] Test Loss = 0.7710 | Test Acc = 72.21% | Test F1 = 71.74%
\n===== Stage-A unfreeze layer4 =====
Trainable params now: 162,437
[Stage-A] Ep01/4 | TL 0.741 TA 71.93% | VL 0.806 VA 70.30% F1 69.37% | ||∇|| 2.332 | lr 1.0e-04
[Stage-A] Ep02/4 | TL 0.727 TA 73.34% | VL 0.782 VA 72.75% F1 71.68% | ||∇|| 2.375 | lr 1.0e-04
[Stage-A] Ep03/4 | TL 0.696 TA 74.78% | VL 0.792 VA 71.66% F1 71.01% | ||∇|| 2.463 | lr 1.0e-04
[Stage-A] Ep04/4 | TL 0.695 TA 73.96% | VL 0.783 VA 70.84% F1 69.98% | ||∇|| 2.397 | lr 1.0e-04
\n===== Stage-B unfreeze layer4+layer3 =====
Trainable params now: 195,013
[Stage-B] Ep01/4 | TL 0.716 TA 73.37% | VL 0.771 VA 73.02% F1 71.90% | ||∇|| 3.072 | lr 1.0e-04
[Stage-B] Ep02/4 | TL 0.683 TA 74.39% | VL 0.764 VA 73.30% F1 72.30% | ||∇|| 3.007 | lr 1.0e-04
[Stage-B] Ep03/4 | TL 0.682 TA 74.70% | VL 0.754 VA 73.84% F1 72.55% | ||∇|| 3.036 | lr 1.0e-04
[Stage-B] Ep04/4 | TL 0.657 TA 75.94% | VL 0.767 VA 72.75% F1 71.86% | ||∇|| 3.136 | lr 1.0e-04
\n===== Stage-C unfreeze layer4..layer2 =====
Trainable params now: 203,365
[Stage-C] Ep01/4 | TL 0.662 TA 74.62% | VL 0.761 VA 72.48% F1 71.50% | ||∇|| 3.605 | lr 1.0e-04
[Stage-C] Ep02/4 | TL 0.632 TA 77.50% | VL 0.780 VA 72.75% F1 71.97% | ||∇|| 3.603 | lr 1.0e-04
[Stage-C] Ep03/4 | TL 0.641 TA 76.88% | VL 0.736 VA 72.21% F1 71.23% | ||∇|| 3.636 | lr 1.0e-04
[Stage-C] Ep04/4 | TL 0.624 TA 77.31% | VL 0.737 VA 74.11% F1 73.15% | ||∇|| 3.650 | lr 1.0e-04
\n===== Stage-D unfreeze ALL (incl. stem) =====
Trainable params now: 207,381
[Stage-D] Ep01/4 | TL 0.609 TA 77.89% | VL 0.716 VA 74.66% F1 73.68% | ||∇|| 5.250 | lr 1.0e-04
[Stage-D] Ep02/4 | TL 0.581 TA 78.82% | VL 0.712 VA 74.11% F1 73.04% | ||∇|| 5.317 | lr 1.0e-04
[Stage-D] Ep03/4 | TL 0.586 TA 78.08% | VL 0.747 VA 71.93% F1 70.73% | ||∇|| 5.434 | lr 1.0e-04
[Stage-D] Ep04/4 | TL 0.573 TA 78.79% | VL 0.711 VA 74.66% F1 73.60% | ||∇|| 6.061 | lr 5.0e-05
\n[After Fine-tuning] Test Loss = 0.6905 | Test Acc = 75.89% | Test F1 = 75.10%
```

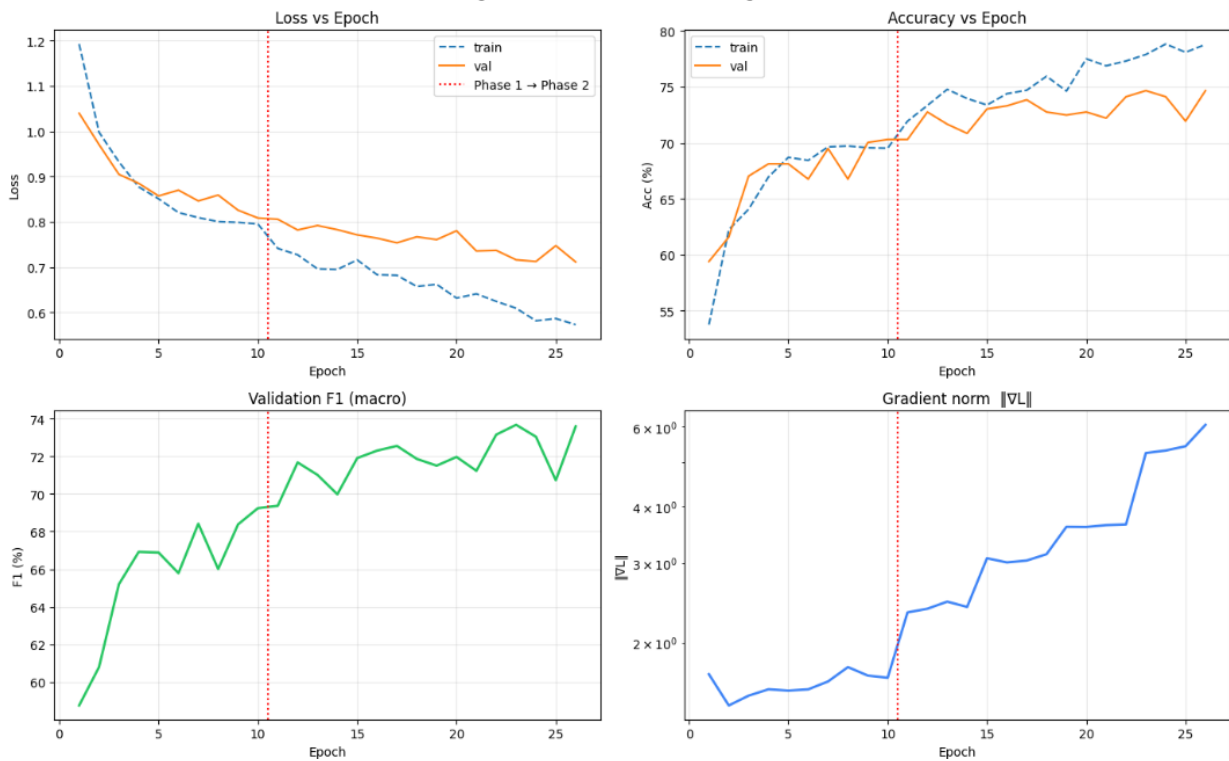
Classification Report

\nClassification report:

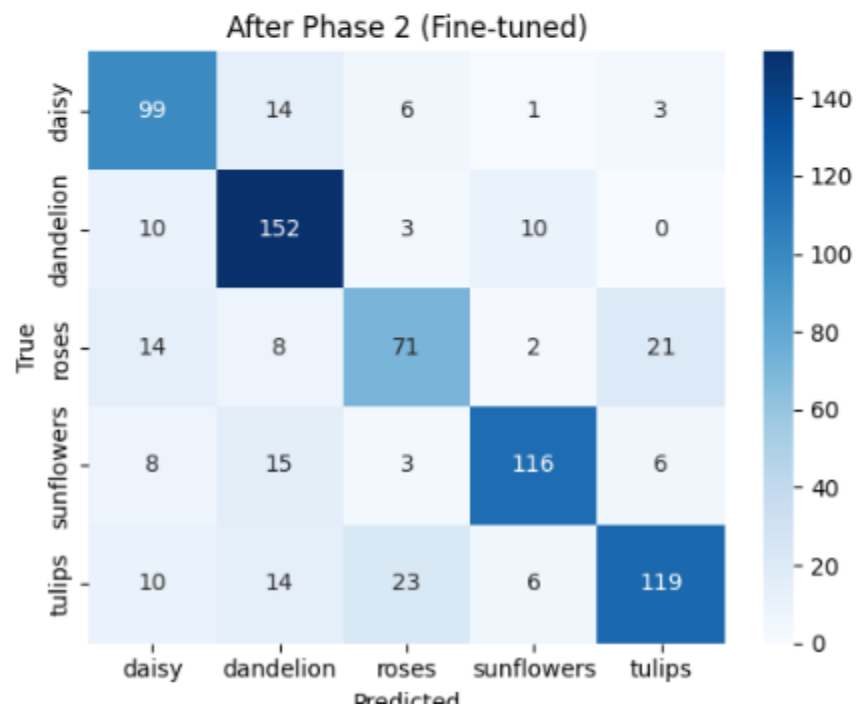
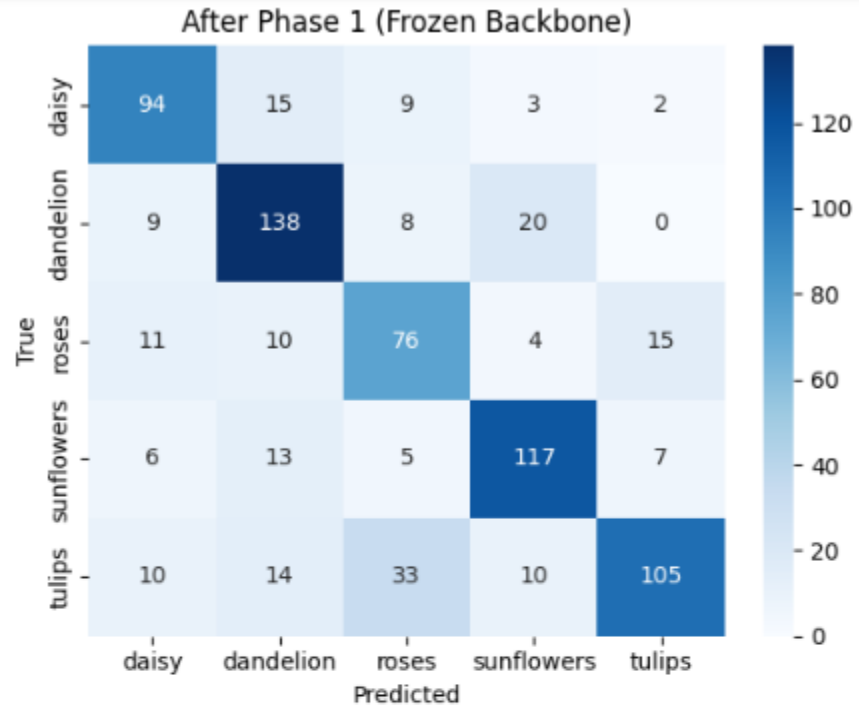
	precision	recall	f1-score	support
daisy	0.7021	0.8049	0.7500	123
dandelion	0.7488	0.8686	0.8042	175
roses	0.6698	0.6121	0.6396	116
sunflowers	0.8593	0.7838	0.8198	148
tulips	0.7987	0.6919	0.7414	172
accuracy			0.7589	734
macro avg	0.7557	0.7522	0.7510	734
weighted avg	0.7624	0.7589	0.7576	734

Transfer Learning phase 1 → 2

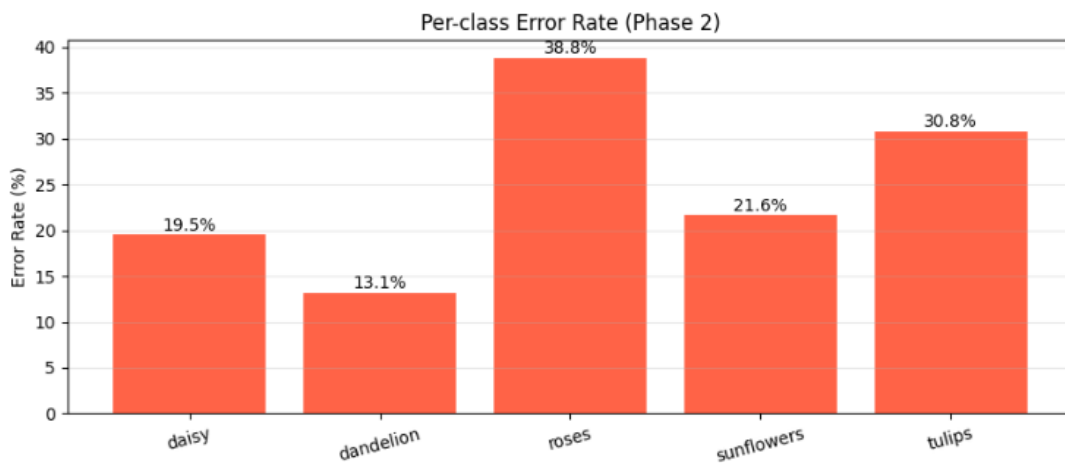
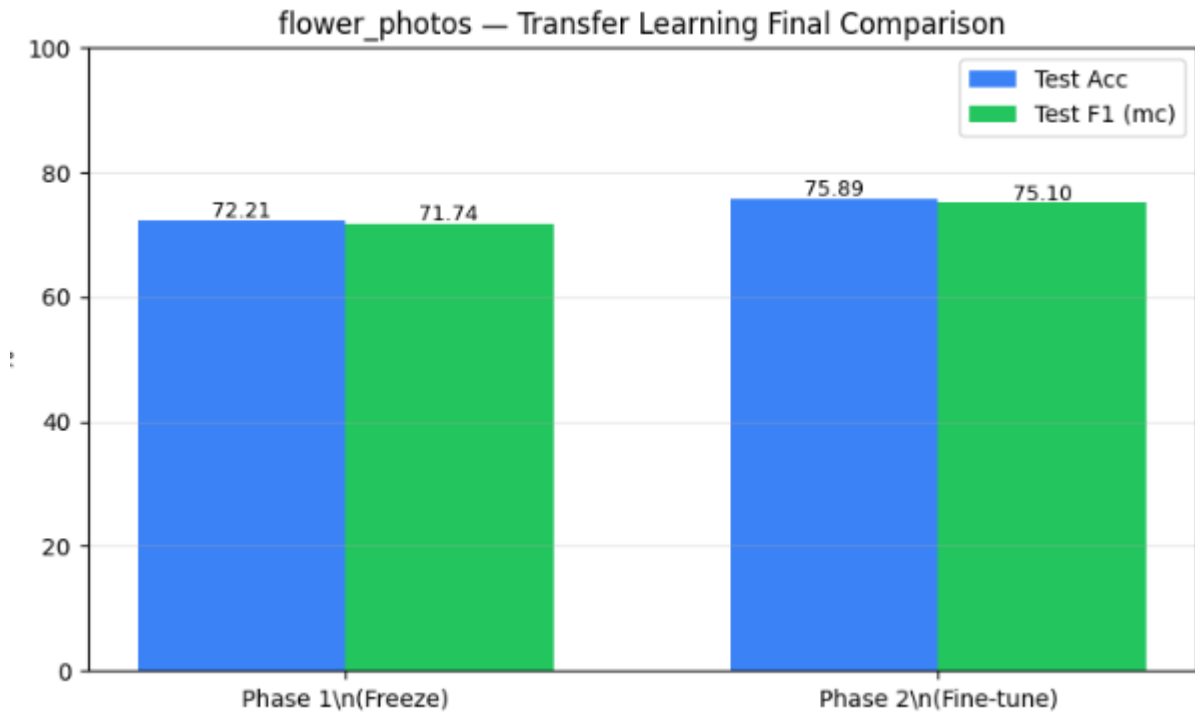
Transfer Learning - Phase 1 (freeze) → Phase 2 (gradual fine-tune)



Confusion Matrixes



مقایسه نتایج



Top most-confused class pairs (true → predicted):

tulips	→ roses	: 23 samples
roses	→ tulips	: 21 samples
sunflowers	→ dandelion	: 15 samples
daisy	→ dandelion	: 14 samples
roses	→ daisy	: 14 samples
tulips	→ dandelion	: 14 samples
dandelion	→ daisy	: 10 samples

MisClassified Examples

Misclassified Test Samples (Phase 2 — Fine-tuned)

true: tulips
pred: dandelion
conf: 0.47



true: sunflowers
pred: sunflowers
conf: 0.45



true: daisy
pred: dandelion
conf: 0.60



true: roses
pred: tulips
conf: 0.55

true: daisy
pred: dandelion
conf: 0.42



true: roses
pred: sunflowers
conf: 0.62



true: tulips
pred: dandelion
conf: 0.44



true: daisy
pred: roses
conf: 0.50

true: sunflowers
pred: tulips
conf: 0.61



true: roses
pred: dandelion
conf: 1.00



true: daisy
pred: dandelion
conf: 0.49



true: dandelion
pred: roses
conf: 0.72

true: roses
pred: dandelion
conf: 0.99



true: dandelion
pred: daisy
conf: 0.51



true: sunflowers
pred: dandelion
conf: 0.58



true: dandelion
pred: sunflowers
conf: 0.43

بخش پنجم: (Deep Convolutional Denoising Autoencoder)

روند کلی کار این بخش:

در این بخش یک AE عمیق (DCDAE) را به همراه یک طبقه بند MLP در انتهای آن طراحی و آموزش دهیم.

برای استفاده از همان مجموعه داده‌ی تمرین قبلی داده‌ی FashionMNIST با همان پیش‌پردازش [۱۰،۱] بارگذاری شد. نویز گاوسی با $\text{mean}=0.0$, $\text{std}=0.3$ و clip در $[0,1]$ کاملاً مطابق یک‌بار از پیش روی train/val/test محاسبه و کش شد.

برای عدم Flatten کردن ورودی و وارد کردن تصویر به صورت تانسور دو بعدی ورودی Encoder تانسور است و Flatten فقط در ابتدای کلاسیفایر MLP رخ می‌دهد.

برای طراحی Encoder بر پایه‌ی لایه‌های پیچشی و Decoder بر پایه‌ی Transposed-Convolution

Encoder شامل ۳ بلوک Conv2d + BN + ReLU + MaxPool و Decoder شامل ۳ بلوک:

ConvTranspose2d + BN + ReLU

برای چسباندن یک Classifier شامل دو لایه‌ی MLP به انتهای شبکه کلاسیفایر دارای

$(10 \rightarrow 128) \text{ fc1 (latent} \rightarrow 128) + \text{BN} + \text{ReLU} + \text{Dropout}(0.4) + \text{fc2}$

ابتدا اتوکدکننده به صورت محلی pre-train شود سپس آموزش در دو مسیر ادامه یابد به صورت مرحله‌ی

pre-training A: اتوکدکننده. مرحله‌ی B: دو نسخه‌ی deepcopy از مدل (model_p1, model_p2).

ساختار کلی:

ساختار کلی به این صورت است:

Encoder $\rightarrow$ Latent Tensor $(7 \times 7 \times 128) \rightarrow \begin{cases} \text{Decoder (Transposed-Conv)} & \rightarrow \text{برای بازسازی} \\ \text{Classifier (MLP (دو لایه))} & \rightarrow \text{برای طبقه‌بندی} \end{cases}$

تقسیم مجموعه داده: تقسیم دسته‌ای 20/10/70

نویز:

$$x_{\text{noisy}} = \text{clip}(x + \mathcal{N}(0, 0.3^2), 0, 1)$$

ابریارامترهای مشترک آزمایش:

پارامتر	مقدار
نرخ یادگیری (Learning Rate)	0.001
اندازه بچ (Batch Size)	۱۲۸
اپاک‌های پیش-آموزش اتوکدکننده (AE Pre-training Epochs)	۱۵
اپاک‌های دسته‌بند (Classifier Epochs)	در هر مسیر ۱۰
بهینه‌ساز (Optimizer)	Adam
تابع فعال‌سازی	ReLU
لایه حذف تصادفی (Dropout)	0.4
آرایه فیلترهای انگدر	$32 \rightarrow 64 \rightarrow 128$
ابعاد فضای پنهان (Latent Size)	$128 \times 7 \times 7 = 6272$

پارامترهای بخش رمزگذار

Encoder = 93,120

بخش رمزگشا

Decoder = 41,537

و بخش دسته‌بند

Classifier = 804,490

: Encoder

ابعاد ورودی → خروجی	لایه‌های تشکیل دهنده	بلوک
(1,28,28) → (32,14,14)	Conv2d(1 → 32, $k = 3$, $p = 1$) + BN + ReLU + MaxPool(2)	۱
(32,14,14) → (64,7,7)	Conv2d(32 → 64, $k = 3$, $p = 1$) + BN + ReLU + MaxPool(2)	۲
(64,7,7) → (128,7,7) = z	Conv2d(64 → 128, $k = 3$, $p = 1$) + BN + ReLU	۳

:Decoder

بلوک	لایه‌های تشکیل دهنده	ابعاد ورودی → خروجی
۱	$\text{ConvTranspose2d}(128 \rightarrow 64, k = 2, s = 2) + \text{BN} + \text{ReLU}$	$(128, 7, 7) \rightarrow (64, 14, 14)$
۲	$\text{ConvTranspose2d}(64 \rightarrow 32, k = 2, s = 2) + \text{BN} + \text{ReLU}$	$(64, 14, 14) \rightarrow (32, 28, 28)$
۳	$\text{ConvTranspose2d}(32 \rightarrow 1, k = 3, p = 1) + \text{Sigmoid}$	$(32, 28, 28) \rightarrow (1, 28, 28) = \hat{x}$

:Classifier

ردیف	لایه عملیاتی	ابعاد ورودی → خروجی
۱	Flatten	$(128, 7, 7) \rightarrow 6272$
۲	$\text{Linear}(6272 \rightarrow 128) + \text{BN} + \text{ReLU} + \text{Dropout}(0.4)$	$6272 \rightarrow 128$
۳	$\text{Linear}(128 \rightarrow 10)$	$128 \rightarrow 10$ (logits)

:A-Pre Training

اپاک (Epoch)	Train MSE	Val MSE
۱	0.01903	0.01082
۵	0.00869	0.00877
۱۰	0.00813	0.00830
۱۵	0.00784	0.00801

:B Training

مدل و فرآیند محاسباتی	Train Acc	Test Acc	Train F1	Test F1 (macro)
Path-1 / رمزگذار منجمد (Frozen)	92.78%	86.44%	0.9271	0.8630
Path-2 / تنظیم دقیق کامل (Fine-tuning)	98.84%	85.81%	0.9884	0.8578

:Path-1

شناسه کلاس	F1-Score شاخص امتیاز
T-shirt/top	0.8154
Trouser	0.9750
Pullover	0.7708
Dress	0.8831
Coat	0.7591
Sandal	0.9546
Shirt	0.6279
Sneaker	0.9262
Bag	0.9714
Ankle boot	0.9463

تحلیل مقایسه‌ای تطبیقی با شبکه تمرین ۲

مدل مورد بررسی	Test Acc شاخص تفکیکی
مدل ساده تمرین ۲ (طراحی دستی از صفر)	92.50%
(PyTorch با توابع بومی) مدل ساده تمرین ۲	92.70%
(Path-1) مسیر اول — DCDAE مدل	86.44%
(Path-2) مسیر دوم — DCDAE مدل	85.81%

دلایل تفاوت عملکرد :

شبکه ساده تمرین ۲ مستقیماً با هدف ریاضی تمایز کلاس‌ها آموزش دیده و کل ظرفیت فیلترها صرف در گام اول صرفاً برای هدف بازسازی استخراج ویژگی‌های متمایزکننده شده است. در حالی که رمزگذار پیکسل‌ها آموزش می‌بیند. در نتیجه فضای پنهان مکلف به حفظ اطلاعات فرکانس پایین و جزئیات پیکسلی است که لزوماً برای مرزبندی کلاس‌ها بهینه نیست

همچنین ورودی مدل دسته‌بند در تمرین ۲ کاملاً تمیز و بدون نویز بود، در حالی که مدل فعلی ورودی را تحت شرایط نویز شدید معدل‌گیری می‌کند

نقش لایه نرمال سازی و افزایش داده

تنظیمات آزمایش	Test Acc	Test F1 (macro)
بدون BN / بدون Augmentation	86.77%	0.8651
معماری اصلی (بدون BN / بدون Augmentation)	87.37%	0.8710
بدون BN / دارای Augmentation	83.64%	0.8322
دارای BN / دارای Augmentation	84.01%	0.8385

روابط پیشرو و پسرو

: Forward

مدل سازی نویز ورودی

$$\tilde{x} = \text{clip}(x + \epsilon, 0, 1), \quad \epsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 0.3$$

Encoder

$$a^{(\ell)} = W^{(\ell)} \star h^{(\ell-1)} + b^{(\ell)}$$

$$\hat{a}^{(\ell)} = \gamma^{(\ell)} \cdot \frac{a^{(\ell)} - \mu_B^{(\ell)}}{\sqrt{(\sigma_B^{(\ell)})^2 + \varepsilon}} + \beta^{(\ell)} \quad (\text{BatchNorm})$$

$$h^{(\ell)} = \text{ReLU}(\hat{a}^{(\ell)}) = \max(0, \hat{a}^{(\ell)})$$

$$p^{(\ell)} = \text{MaxPool}_{2 \times 2}(h^{(\ell)}) \quad (\text{برای } \ell \in \{1, 2\})$$

Decoder

$$\begin{aligned}u^{(m)} &= W_T^{(m)} \star^T r^{(m-1)} + b_T^{(m)} \\ \hat{u}^{(m)} &= \gamma^{(m)} \cdot \frac{u^{(m)} - \mu_B^{(m)}}{\sqrt{(\sigma_B^{(m)})^2 + \varepsilon}} + \beta^{(m)} \\ r^{(m)} &= \text{ReLU}(\hat{u}^{(m)})\end{aligned}$$

در لایه نهایی خروجی از تابع فعال ساز سیگموید عبور می کند

$$\hat{x} = \sigma(u^{(3)}) = \frac{1}{1 + e^{-u^{(3)}}}$$

:Path A

$$L_{\text{AE}} = \frac{1}{N} \sum_{i=1}^N \frac{1}{H \cdot W \cdot C} \sum_{j=1}^{H \cdot W \cdot C} (\hat{x}_{i,j} - x_{i,j})^2$$

:Classifier

$$\begin{aligned}v &= \text{vec}(z) \in \mathbb{R}^{6272} \\ s^{(1)} &= W_{\text{fc}}^{(1)} v + b_{\text{fc}}^{(1)} \\ \hat{s}^{(1)} &= \gamma_{\text{fc}} \cdot \frac{s^{(1)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} + \beta_{\text{fc}} \\ h_{\text{fc}} &= \text{ReLU}(\hat{s}^{(1)}) \\ \tilde{h}_{\text{fc}} &= m \odot h_{\text{fc}}, \quad m_i \sim \frac{1}{1-p} \text{Bernoulli}(1-p), \quad p = 0.4 \\ o &= W_{\text{fc}}^{(2)} \tilde{h}_{\text{fc}} + b_{\text{fc}}^{(2)} \in \mathbb{R}^{10}\end{aligned}$$

(مرحله B) تابع هزینه دسته بندی نهایی

$$\hat{p}_{i,k} = \frac{e^{o_k}}{\sum_{j=1}^{10} e^{o_j}} \Rightarrow L_{\text{CLS}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^{10} y_{i,k} \log \hat{p}_{i,k}$$

: Backward

از تابع هزینه تا خروجی رمزگشا

$$\frac{\partial L_{AE}}{\partial \hat{x}_{i,j}} = \frac{2}{N \cdot H \cdot W \cdot C} (\hat{x}_{i,j} - x_{i,j})$$

مشتق عبور از تابع سیگموید لایه نهایی

$$\delta_T^{(3)} = \frac{\partial L}{\partial u^{(3)}} = \frac{\partial L}{\partial \hat{x}} \odot \sigma(u^{(3)}) \odot (1 - \sigma(u^{(3)}))$$

مشتق عبور از تابع فعال ساز

$$\delta_{\hat{u}}^{(m)} = \delta_r^{(m)} \odot \mathbb{I}[\hat{u}^{(m)} > 0]$$

BatchNorm عبور گرادیان از لایه

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^N \delta_{\hat{u},i}^{(m)} \odot \tilde{u}_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^N \delta_{\hat{u},i}^{(m)}$$

$$\frac{\partial L}{\partial u_i} = \frac{\gamma}{N\sqrt{\sigma_B^2 + \varepsilon}} \left(N\delta_{\hat{u},i}^{(m)} - \sum_{j=1}^N \delta_{\hat{u},j}^{(m)} - \tilde{u}_i \sum_{j=1}^N \delta_{\hat{u},j}^{(m)} \odot \tilde{u}_j \right)$$

پس انتشار از لایه کانولوشن ترانهاد

$$\frac{\partial L}{\partial r^{(m-1)}} = W_T^{(m)} \star \delta_u^{(m)}$$

$$\frac{\partial L}{\partial W_T^{(m)}} = \delta_u^{(m)} \star^T r^{(m-1)}, \quad \frac{\partial L}{\partial b_T^{(m)}} = \sum_{n,h,w} \delta_{u,(n,h,w)}^{(m)}$$

انکدر Max Pooling پس انتشار از لایه

$$\frac{\partial L}{\partial h_{i,j}^{(\ell)}} = \begin{cases} \frac{\partial L}{\partial p_{i,j}^{(\ell)}} & \text{if } (i,j) = \text{argmax}(h^{(\ell)}) \\ 0 & \text{otherwise} \end{cases}$$

پس انتشار از لایه های کانولوشن

$$\frac{\partial L}{\partial h^{(\ell-1)}} = W^{(\ell)} \star^T \delta_a^{(\ell)}$$
$$\frac{\partial L}{\partial W^{(\ell)}} = \delta_a^{(\ell)} \star h^{(\ell-1)}, \quad \frac{\partial L}{\partial b^{(\ell)}} = \sum_{n,h,w} \delta_{a,(n,h,w)}^{(\ell)}$$

از تابع هزینه انتروپی متقاطع تالوجیت های خروجی دسته بند

$$\frac{\partial L_{\text{CLS}}}{\partial o_k} = \hat{p}_k - y_k$$

پس انتشار از لایه خطی دوم

$$\frac{\partial L}{\partial \tilde{h}_{\text{fc}}} = \left(W_{\text{fc}}^{(2)}\right)^T \frac{\partial L}{\partial o}, \quad \frac{\partial L}{\partial W_{\text{fc}}^{(2)}} = \frac{\partial L}{\partial o} (\tilde{h}_{\text{fc}})^T, \quad \frac{\partial L}{\partial b_{\text{fc}}^{(2)}} = \frac{\partial L}{\partial o}$$

(Dropout) عبور گرادیان از لایه حذف تصادفی

$$\frac{\partial L}{\partial h_{\text{fc}}} = m \odot \frac{\partial L}{\partial \tilde{h}_{\text{fc}}}$$

عبور گرادیان از لایه فعال ساز خطی اول دسته بند

$$\frac{\partial L}{\partial \hat{s}^{(1)}} = \frac{\partial L}{\partial h_{fc}} \odot \mathbb{I}[\hat{s}^{(1)} > 0]$$

پس انتشار از لایه خطی اول

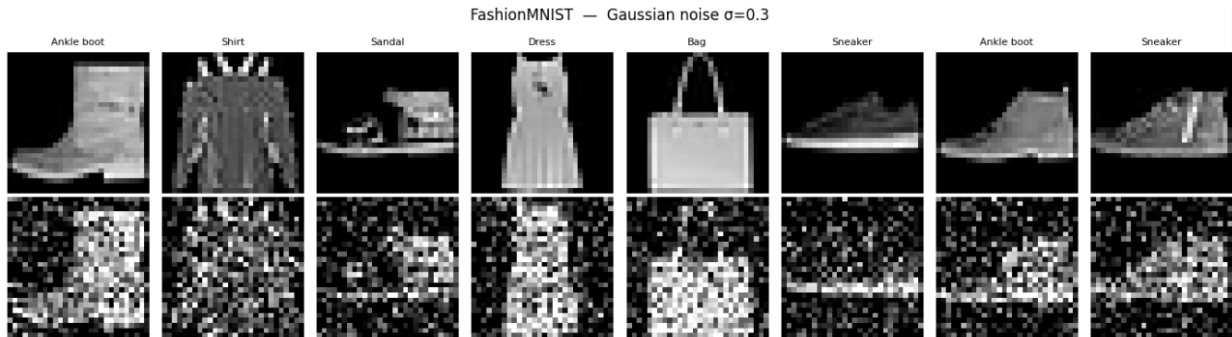
$$\frac{\partial L}{\partial v} = \left(W_{fc}^{(1)}\right)^T \frac{\partial L}{\partial s^{(1)}}, \quad \frac{\partial L}{\partial W_{fc}^{(1)}} = \frac{\partial L}{\partial s^{(1)}} v^T$$

تغییر آرایه ساختاری از بردار به تنسور فضای پنهان

$$\frac{\partial L}{\partial z} = \text{reshape}\left(\frac{\partial L}{\partial v}, 128 \times 7 \times 7\right)$$

خروجی ها:

FashionMNIST with Noise



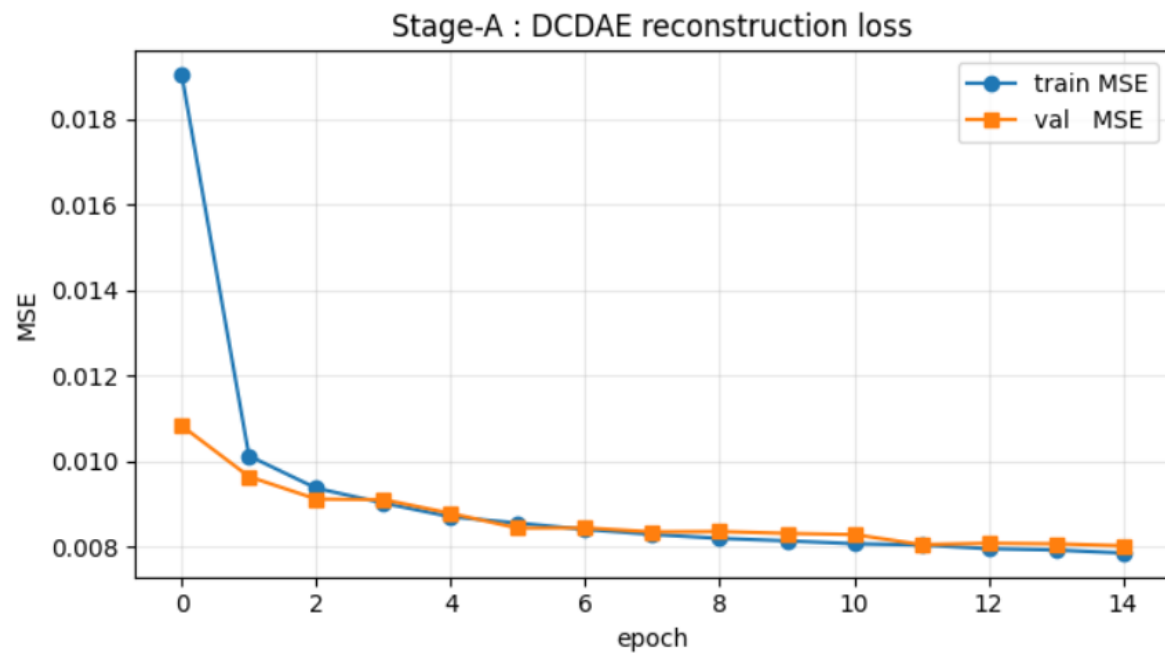
Train شبکه

Reconstruction shape : `torch.Size([4, 1, 28, 28])`
Latent shape : `torch.Size([4, 128, 7, 7])`
Logits shape : `torch.Size([4, 10])`

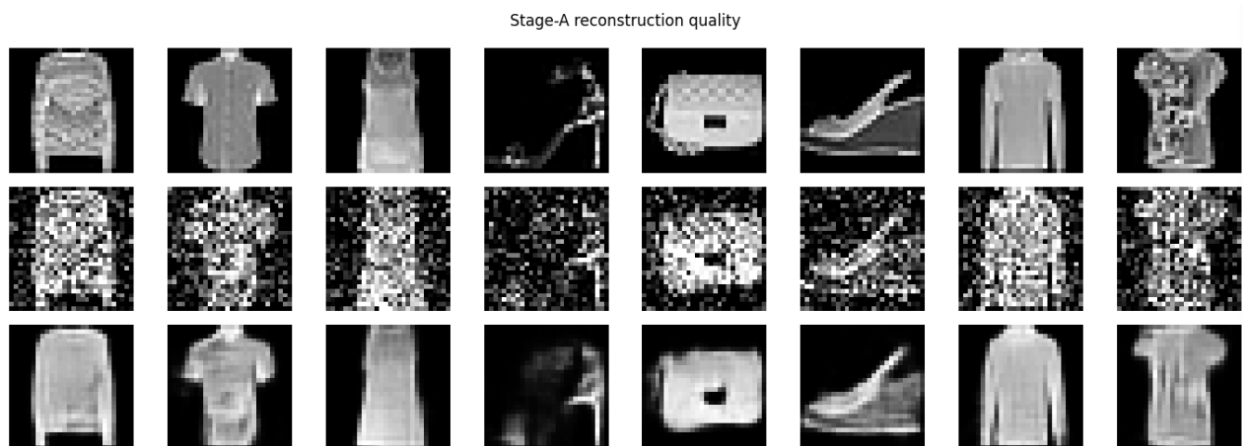
Encoder params : 93,120
Decoder params : 41,537
Classifier params : 804,490
TOTAL : 939,147

Epoch	1/15	train MSE = 0.01903	val MSE = 0.01082
Epoch	2/15	train MSE = 0.01012	val MSE = 0.00963
Epoch	3/15	train MSE = 0.00936	val MSE = 0.00911
Epoch	4/15	train MSE = 0.00901	val MSE = 0.00909
Epoch	5/15	train MSE = 0.00869	val MSE = 0.00877
Epoch	6/15	train MSE = 0.00854	val MSE = 0.00843
Epoch	7/15	train MSE = 0.00840	val MSE = 0.00843
Epoch	8/15	train MSE = 0.00828	val MSE = 0.00834
Epoch	9/15	train MSE = 0.00819	val MSE = 0.00835
Epoch	10/15	train MSE = 0.00813	val MSE = 0.00830
Epoch	11/15	train MSE = 0.00806	val MSE = 0.00828
Epoch	12/15	train MSE = 0.00803	val MSE = 0.00804
Epoch	13/15	train MSE = 0.00795	val MSE = 0.00807
Epoch	14/15	train MSE = 0.00791	val MSE = 0.00806
Epoch	15/15	train MSE = 0.00784	val MSE = 0.00801

Cost Function



Reconstruction



Path A , Path B

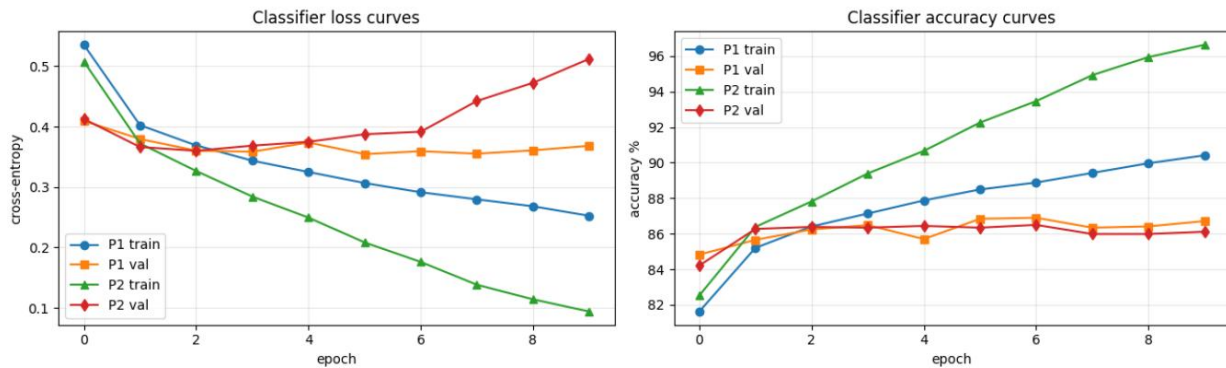
Path-1 trainable parameters : 846,027

Epoch	train_loss	train_acc	val_loss	val_acc
1/10	0.5355	81.63%	0.4093	84.83%
2/10	0.4024	85.21%	0.3796	85.66%
3/10	0.3689	86.41%	0.3602	86.24%
4/10	0.3437	87.14%	0.3584	86.49%
5/10	0.3248	87.88%	0.3737	85.70%
6/10	0.3067	88.49%	0.3545	86.84%
7/10	0.2913	88.88%	0.3593	86.90%
8/10	0.2796	89.42%	0.3552	86.34%
9/10	0.2681	89.97%	0.3606	86.41%
10/10	0.2524	90.41%	0.3682	86.71%

Path-2 trainable parameters : 897,610

Epoch	train_loss	train_acc	val_loss	val_acc
1/10	0.5078	82.53%	0.4126	84.21%
2/10	0.3721	86.39%	0.3662	86.27%
3/10	0.3266	87.82%	0.3599	86.39%
4/10	0.2842	89.39%	0.3685	86.34%
5/10	0.2494	90.66%	0.3749	86.44%
6/10	0.2083	92.26%	0.3874	86.34%
7/10	0.1760	93.46%	0.3915	86.50%
8/10	0.1382	94.92%	0.4424	85.99%
9/10	0.1140	95.94%	0.4725	85.99%
10/10	0.0939	96.63%	0.5122	86.11%

Classifier Acc and Loss Curves



Path-1 (frozen)

Train : acc =	92.78%	F1(macro) =	0.9271
Test : acc =	86.44%	F1(macro) =	0.8630

Path-2 (finetune)

Train : acc =	98.84%	F1(macro) =	0.9884
Test : acc =	85.81%	F1(macro) =	0.8578

Path 1 and Path 2 – Train

Path-1 — Train

True	T-shirt/top	4628	0	64	26	8	0	139	0	3	0	
	Trouser	9	4889	1	23	2	0	2	0	1	0	
	Pullover	33	2	4571	15	131	0	144	0	0	0	
	Dress	137	15	16	4591	76	0	59	0	0	0	
	Coat	8	1	570	73	4067	0	193	0	0	0	
	Sandal	0	0	0	0	0	4873	0	65	0	13	
	Shirt	652	1	467	50	237	0	3529	0	1	0	
	Sneaker	0	0	0	0	0	40	0	4659	1	166	
	Bag	5	1	9	3	11	1	5	2	4859	0	
	Ankle boot	0	0	0	0	0	7	0	49	0	4797	
		T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot	True

Predicted

Path-2 — Train

True	T-shirt/top	4733	0	7	18	0	0	108	0	2	0	
	Trouser	0	4923	0	3	1	0	0	0	0	0	
	Pullover	5	0	4740	4	92	0	55	0	0	0	
	Dress	0	0	1	4877	15	0	1	0	0	0	
	Coat	0	0	29	6	4872	0	5	0	0	0	
	Sandal	0	0	0	0	0	4945	0	6	0	0	
	Shirt	33	0	27	15	75	0	4786	0	1	0	
	Sneaker	0	0	0	0	0	4	0	4862	0	0	
	Bag	0	0	0	1	0	0	3	0	4892	0	
	Ankle boot	0	0	0	0	0	1	0	51	0	4801	
		T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot	True

Predicted

Path 1 and Path 2 – Test

True \ Predicted	T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot
T-shirt/top	1219	2	48	20	1	1	107	0	10	0
Trouser	8	1325	6	24	1	0	5	0	1	1
Pullover	14	0	1219	12	90	0	93	0	2	0
Dress	78	15	14	1205	52	1	36	0	1	0
Coat	6	1	238	42	1007	1	106	0	2	0
Sandal	1	0	1	0	0	1292	0	52	4	18
Shirt	250	4	196	21	89	0	783	0	5	0
Sneaker	0	0	0	0	0	26	0	1298	4	80
Bag	6	0	11	3	10	4	16	2	1378	0
Ankle boot	0	0	0	0	0	14	0	43	0	1375

	Predicted									
	T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot
T-shirt/top	1072	1	33	79	5	1	207	0	10	0
Trouser	3	1323	3	29	3	0	8	0	1	1
Pullover	10	1	1073	17	168	1	158	0	2	0
Dress	27	8	9	1249	59	0	48	0	2	0
Coat	4	0	118	51	1133	0	94	0	3	0
Sandal	1	0	1	0	0	1303	1	41	3	18
Shirt	151	4	120	61	167	1	835	0	9	0
Sneaker	0	0	0	0	0	44	0	1326	1	37
Bag	8	0	7	5	5	8	21	4	1371	1
Ankle boot	0	0	0	0	0	15	0	89	0	1328

نتائج

===== Path-1 (frozen) - test classification report =====

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

T-shirt/top	0.7705	0.8658	0.8154	1408
Trouser	0.9837	0.9664	0.9750	1371
Pullover	0.7034	0.8524	0.7708	1430
Dress	0.9081	0.8595	0.8831	1402
Coat	0.8056	0.7177	0.7591	1403
Sandal	0.9649	0.9444	0.9546	1368
Shirt	0.6832	0.5809	0.6279	1348
Sneaker	0.9305	0.9219	0.9262	1408
Bag	0.9794	0.9636	0.9714	1430
Ankle boot	0.9328	0.9602	0.9463	1432
accuracy			0.8644	14000
macro avg	0.8662	0.8633	0.8630	14000
weighted avg	0.8664	0.8644	0.8636	14000

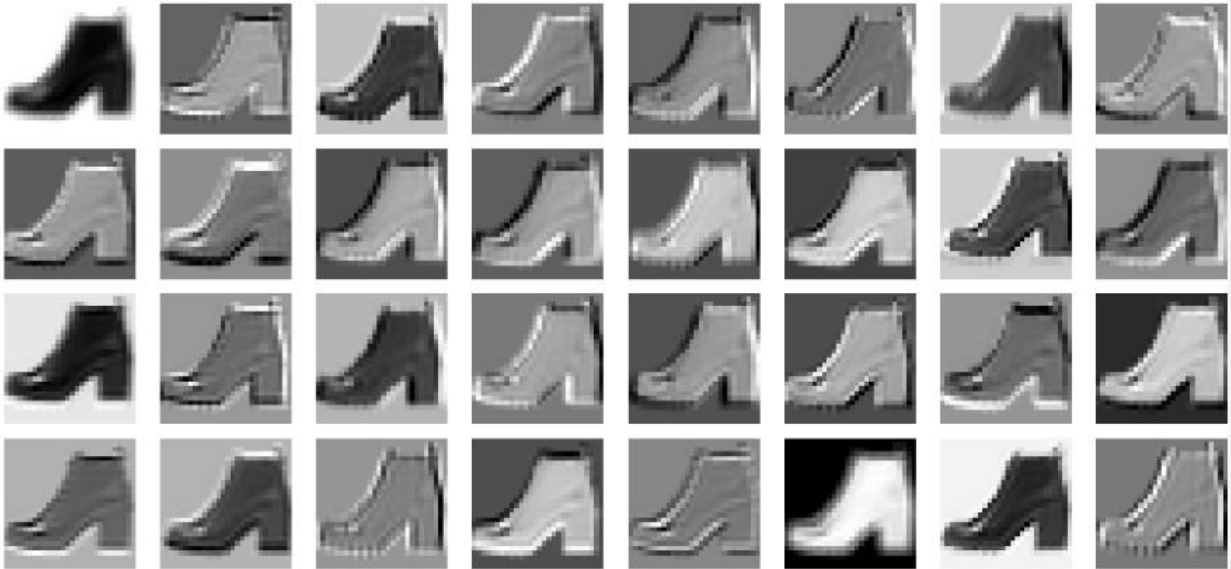
===== Path-2 (finetune) - test classification report =====

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

T-shirt/top	0.8401	0.7614	0.7988	1408
Trouser	0.9895	0.9650	0.9771	1371
Pullover	0.7867	0.7503	0.7681	1430
Dress	0.8377	0.8909	0.8635	1402
Coat	0.7357	0.8076	0.7700	1403
Sandal	0.9490	0.9525	0.9507	1368
Shirt	0.6086	0.6194	0.6140	1348
Sneaker	0.9082	0.9418	0.9247	1408
Bag	0.9779	0.9587	0.9682	1430
Ankle boot	0.9588	0.9274	0.9428	1432
accuracy			0.8581	14000
macro avg	0.8592	0.8575	0.8578	14000
weighted avg	0.8600	0.8581	0.8585	14000

Feature Maps

First convolution layer feature maps (Path-2 encoder)

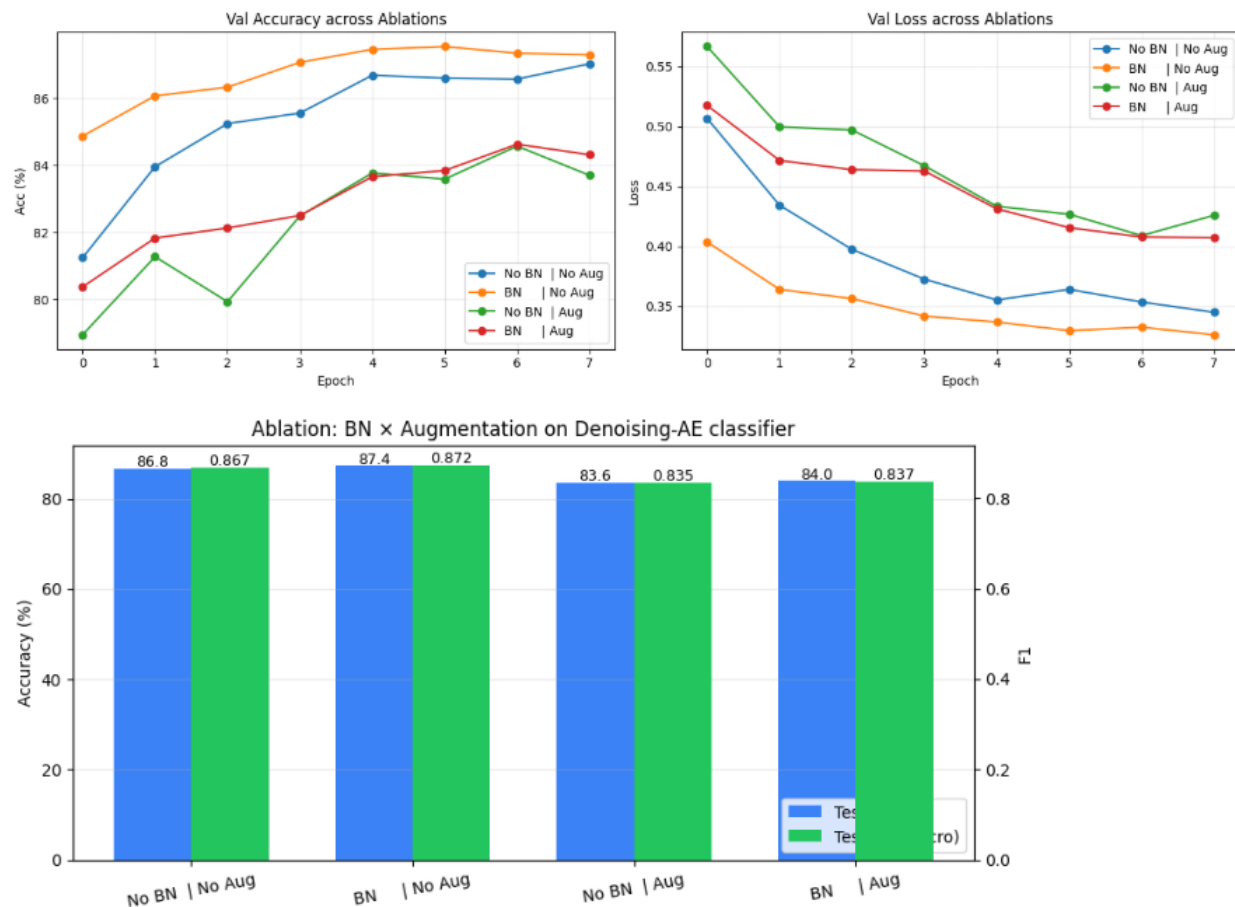


Ablation

===== ABLATION =====

```
[no_BN_no_Aug] Ep 1/8 train_acc=79.95% val_acc=81.24%
[no_BN_no_Aug] Ep 2/8 train_acc=83.35% val_acc=83.96%
[no_BN_no_Aug] Ep 3/8 train_acc=84.18% val_acc=85.24%
[no_BN_no_Aug] Ep 4/8 train_acc=84.88% val_acc=85.56%
[no_BN_no_Aug] Ep 5/8 train_acc=85.42% val_acc=86.69%
[no_BN_no_Aug] Ep 6/8 train_acc=85.98% val_acc=86.60%
[no_BN_no_Aug] Ep 7/8 train_acc=86.16% val_acc=86.57%
[no_BN_no_Aug] Ep 8/8 train_acc=86.36% val_acc=87.03%
[BN_only] Ep 1/8 train_acc=81.07% val_acc=84.87%
[BN_only] Ep 2/8 train_acc=84.60% val_acc=86.07%
[BN_only] Ep 3/8 train_acc=85.52% val_acc=86.33%
[BN_only] Ep 4/8 train_acc=86.08% val_acc=87.07%
[BN_only] Ep 5/8 train_acc=86.53% val_acc=87.46%
[BN_only] Ep 6/8 train_acc=87.03% val_acc=87.54%
[BN_only] Ep 7/8 train_acc=87.10% val_acc=87.34%
[BN_only] Ep 8/8 train_acc=87.48% val_acc=87.30%
[Aug_only] Ep 1/8 train_acc=71.23% val_acc=78.94%
[Aug_only] Ep 2/8 train_acc=75.74% val_acc=81.27%
[Aug_only] Ep 3/8 train_acc=77.50% val_acc=79.93%
[Aug_only] Ep 4/8 train_acc=78.35% val_acc=82.49%
[Aug_only] Ep 5/8 train_acc=79.21% val_acc=83.77%
[Aug_only] Ep 6/8 train_acc=80.00% val_acc=83.59%
[Aug_only] Ep 7/8 train_acc=80.33% val_acc=84.57%
[Aug_only] Ep 8/8 train_acc=80.53% val_acc=83.70%
[BN_+_Aug] Ep 1/8 train_acc=72.63% val_acc=80.37%
[BN_+_Aug] Ep 2/8 train_acc=77.02% val_acc=81.83%
[BN_+_Aug] Ep 3/8 train_acc=78.47% val_acc=82.13%
[BN_+_Aug] Ep 4/8 train_acc=79.15% val_acc=82.50%
[BN_+_Aug] Ep 5/8 train_acc=79.94% val_acc=83.66%
[BN_+_Aug] Ep 6/8 train_acc=80.35% val_acc=83.84%
[BN_+_Aug] Ep 7/8 train_acc=80.56% val_acc=84.63%
[BN_+_Aug] Ep 8/8 train_acc=80.98% val_acc=84.31%
```

Ablation و مقایسه



===== Ablation Summary =====		
Setting	Test Acc (%)	Test F1 (macro)
No BN No Aug	86.77	0.8674
BN No Aug	87.37	0.8723
No BN Aug	83.64	0.8349
BN Aug	84.01	0.8375

ممنون و خسته نباشید.